

**ANALISIS KOMPARASI MODEL *MACHINE LEARNING*
DAN *DEEP LEARNING* UNTUK KLASIFIKASI SENTIMEN
ISU GENOSIDA PALESTINA PADA MEDIA SOSIAL X**

SKRIPSI

diajukan untuk menempuh ujian sarjana
pada Fakultas Matematika dan Ilmu Pengetahuan Alam
Universitas Padjadjaran

WAFA TSABITA
NPM 140810200055



UNIVERSITAS PADJADJARAN
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
PROGRAM STUDI TEKNIK INFORMATIKA
SUMEDANG
2025

SKRIPSI

ANALISIS KOMPARASI MODEL *MACHINE LEARNING* DAN *DEEP LEARNING* UNTUK KLASIFIKASI SENTIMEN ISU GENOSIDA PALESTINA PADA MEDIA SOSIAL X

COMPARATIVE ANALYSIS OF MACHINE LEARNING AND DEEP LEARNING MODELS FOR SENTIMENT CLASSIFICATION OF THE PALESTINIAN GENOCIDE ISSUE ON SOCIAL MEDIA X

Telah dipersiapkan dan disusun oleh

WAFA TSABITA
NPM 140810200055

Telah dipertahankan di depan Tim Penguji
pada tanggal

Susunan Tim Penguji

- | | | | |
|----|---------------------------------------|-------------------|-------|
| 1. | <u>Dr. Akmal, S.Si., MT.</u> | Ketua Tim Penguji | |
| | NIP. 19700615 199803 1 003 | | |
| 2. | <u>Dr. Afrida Helen, S.T., M.Kom.</u> | Pembimbing | |
| | NIP. 19650128 199703 2 001 | | |
| 3. | <u>Dr. Mira Suryani, S.Pd, M.Kom.</u> | Co-Pembimbing | |
| | NIP. 19891230 201504 2 001 | | |
| 4. | | Penguji | |
| | NIP. | | |
| 5. | | Penguji | |
| | NIP. | | |
| 6. | | Penguji | |
| | NIP. | | |

KATA PENGANTAR

Bismillahirrahmanirrahim. Dengan menyebut nama Allah yang Maha Pengasih dan Maha Penyayang. Segala puji dan syukur penulis panjatkan ke hadirat Allah SWT yang telah memberikan rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan penyusunan skripsi yang berjudul “ANALISIS KOMPARASI MODEL *MACHINE LEARNING* DAN *DEEP LEARNING* UNTUK KLASIFIKASI SENTIMEN ISU GENOSIDA PALESTINA PADA MEDIA SOSIAL X” sebagai salah satu syarat menempuh sarjana pada Program Studi S-1 Teknik Informatika Fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Padjadjaran.

Dalam proses penyusunan dan penulisan skripsi ini tidak terlepas dari bantuan, bimbingan, serta dukungan dari berbagai pihak. Oleh karena itu dalam kesempatan ini penulis mengucapkan terima kasih kepada Dr. Afrida Helen, S.T., M.Kom., sebagai pembimbing utama, dan Dr. Mira Suryani, S.Pd, M.Kom. sebagai pembimbing pendamping yang telah meluangkan waktu dan pikirannya sehingga penulis dapat menyelesaikan skripsi ini. Ucapan terima kasih juga diberikan kepada keluarga penulis yang selalu memberikan motivasi dan doa yang menjadi pendorong dalam penyelesaian skripsi ini. Penulis juga mengucapkan terima kasih sebanyak-banyaknya kepada:

1. Prof. Dr. Iman Rahayu, S.Si., M.Si, selaku Dekan Fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Padjadjaran.

2. Dr. Setiawan Hadi, M.Sc.CS, selaku Kepala Departemen Ilmu Komputer Fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Padjadjaran.
3. Dr. Akmal, S.Si., M.T., selaku Ketua Program Studi S-1 Teknik Informatika Fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Padjadjaran
4. Dosen-dosen Teknik Informatika Unpad yang telah mengajar dan memberikan ilmu kepada penulis selama masa perkuliahan yang membawa penulis pada posisi sekarang ini.
5. Teman-teman angkatan seperjuangan yang telah membantu dalam proses penelitian ini.
6. Sahabat Cita Syurga yang terus menemani dalam perjalanan penelitian ini.

Akhir kata, penulis berharap semoga skripsi ini dapat bermanfaat bagi pembaca.

Jatinangor, Desember 2025

Penulis

ABSTRAK

Media sosial X telah menjadi ruang utama dalam diskusi publik mengenai isu-isu global, termasuk tindakan genosida dilakukan oleh Israel. Yang kemudian memunculkan arus informasi terorganisir, atau sering disebut sebagai *Hasbara* project, yang memicu media dalam penyebaran konten yang menyesatkan dan bias. Banyaknya data unggahan pengguna menjadikan analisis secara manual tidak lagi efektif, sehingga dibutuhkan teknik klasifikasi sentimen secara otomatis.

Penelitian ini bertujuan untuk membandingkan kinerja model *machine learning* dan *deep learning* dalam mengklasifikasikan sentimen terkait isu genosida Palestina pada media sosial X. Terkumpul sebanyak 1.702 tweet dengan jangka waktu dari Oktober 2023 sampai dengan April 2025. Data tersebut diklasifikasikan ke dalam tiga kelas sentimen, yaitu pro, netral dan kontra-genosida. Tahapan *preprocess* data meliputi *case folding*, menghapus noise, menghapus stopword, tokenisasi, lematisasi dan stemming. Kemudian proses ekstraksi fitur dilakukan menggunakan TF-IDF dan Word2vec. Model yang dievaluasi meliputi *Naive Bayes*, *Support Vector Machine*, *Long Short-Term Memory* dan *Bidirectional Long Short-Term Memory*, dengan penerapan *hyperparameter tuning*.

Hasil penelitian menunjukkan bahwa model SVM memperoleh performa kinerja tertinggi dengan nilai akurasi sebesar 85,97% dan *f1-score* sebesar 85,88%. Namun pada model *deep learning*, khususnya LSTM dan BiLSTM, menunjukkan kemampuan prediksi yang lebih baik dalam menangkap konteks dan pola kata pada teks. Selain itu, penelitian ini juga mengembangkan aplikasi berbasis web menggunakan Streamlit untuk mendiseminasikan dan menampilkan visualisasi hasil klasifikasi sentimen,

Kata Kunci: Analisis Sentimen, *Deep Learning*, Genosida, *Machine Learning*, Media Sosial X

ABSTRACT

Social media platforms such X have become major spaces for public discussion on global humanitarian, including the genocide action commits by Israel. The emergence of organized information campaigns, often referred to as hasbara projects, has intensified online discourse and contributed to information warfare, including the spread of misleading or biased content. The large volume and complexity of such user-generated data make manual analysis impractical, thus requiring automated sentiment classification techniques.

This study aims to compare the performance of machine learning and deep learning models in classifying sentiments of tweets related to the genocide issue on social media X. With total 1.702 twweet collected between October 2023 and April 2025 were categorized into three sentiment classes: pro, neutral dan contra-genocide. Data preprocessing involved case folding, noise removal, stopword removal, tokenization, lemmatization and stemming. While feature extraction was performed using TF-IDF and Word2vec. The evaluated models included Naive Bayes, Support Vector Machine, Long Short-Term Memory, and Bidirectional Long Short-Term Memory with hyperparameter tuning applied.

The result indicate that the SVM model achieved the highest overall performance, with an accuracy of 85.97% and an f1-score of 85.88%. however, deep learning models, particularly LSTM and BiLSTM, has more capability in predicting sentiment. Also a web-based application was developed using Streamlit to disseminate and visualize sentiment classification results.

Keywords: *Sentiment Analysis, Deep Learning, Genocide, Machine Learning, Social Media X,*

DAFTAR ISI

ABSTRAK	v
<i>ABSTRACT</i>	vi
DAFTAR ISI	vii
DAFTAR GAMBAR	xi
DAFTAR TABEL	x
DAFTAR LAMPIRAN	xiii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Identifikasi Masalah	4
1.3 Batasan Masalah	4
1.4 Maksud dan Tujuan Penelitian	5
1.5 Manfaat Penelitian	6
1.6 Metodologi Penelitian	6
1.7 Sistematika Penulisan	7
BAB II TINJAUAN PUSTAKA	9
2.1 Analisis Sentimen	9
2.2 <i>Natural Language Processing (NLP)</i>	10
2.3 <i>Text Preprocessing</i>	11
2.4 <i>Machine Learning</i>	13
2.4.1 <i>Naive Bayes</i>	13
2.4.2 <i>Support Vector Machine (SVM)</i>	14
2.5 <i>Deep Learning</i>	14
2.5.1 <i>Long Short-Term Memory</i>	14
2.5.2 <i>Bidirectional Long Short-Term Memory (BiLSTM)</i>	16
2.6 <i>Unified Modeling Language (UML)</i>	17
2.6.1 <i>Use Case Diagram</i>	18
2.6.2 <i>Activity Diagram</i>	19
2.6.3 <i>Deployment Diagram</i>	21

2.7	Penelitian Terdahulu	23
BAB III ANALISIS DAN PERANCANGAN		26
3.1	Tahapan Penelitian.....	26
3.2	Objek Penelitian.....	27
3.3	Pengumpulan Data.....	29
3.4	Pelabelan Data	32
3.5	Data Balancing.....	35
3.6	Data Preprocessing	36
3.6.1	Case Folding	36
3.6.2	Noise Removal.....	37
3.6.3	Stopword Removal	39
3.6.4	Tokenization	40
3.6.5	Lemmatization	41
3.7	Data Training, Validation dan Prediction	42
3.8	Ekstraksi Fitur.....	43
3.8.1	<i>Term Frecuency-Inverse Document Frequency</i>	43
3.8.2	Word Embedding.....	45
3.9	Pembangunan Model	50
3.9.1	Naive Bayes	50
3.9.2	Support Vector Machine	52
3.9.3	Long Short-Term Memeory.....	53
3.9.4	Bidirectional Long Short-Term Memory.....	56
3.10	Perancangan Aplikasi	58
3.10.1	Use Case Diagram	58
3.10.2	Activity Diagram	60
3.10.3	Deployment Diagram.....	63
3.10.4	Perancangan Antarmuka	64
3.11	Analisis Kebutuhan.....	70
3.11.1	Kebutuhan Perangkat Lunak.....	70
3.11.2	Kebutuhan Perangkat Keras.....	70
BAB IV HASIL DAN PEMBAHASAN		71

4.1	Persiapan Data	71
4.1.1	Term Frequency-Inverse Document Frequency	72
4.1.2	<i>Word Embedding</i>	75
4.2	Pengujian Model.....	80
4.2.1	Naive Bayes	81
4.2.2	Support Vector Machine	83
4.2.3	Long Short-Term Memory.....	86
4.2.4	Bidirectional Long Short-Term Memory.....	94
4.3	Perbandingan Evaluasi Model	97
4.4	Perbandingan Klasifikasi Model.....	98
4.5	Implementasi Aplikasi	100
4.5.1	Halaman Utama	100
4.5.2	Halaman Dataset Penelitian	102
4.5.3	Halaman <i>Data Preprocessing</i>	103
4.5.4	Halaman <i>Data Balancing</i>	105
4.5.5	Halaman Ekstraksi Fitur	106
4.5.6	Halaman <i>Model Evaluation</i>	109
4.5.7	Halaman <i>Model Comparison</i>	110
4.5.8	Halaman <i>Sentiment Prediction</i>	112
4.6	Limitasi Penelitian.....	113
	BAB V KESIMPULAN DAN SARAN.....	115
5.1	Kesimpulan.....	115
5.1	Saran	116
	DAFTAR PUSTAKA	117
	LAMPIRAN.....	119
	RIWAYAT HIDUP.....	128

DAFTAR GAMBAR

Gambar 1.1 Grafik Frekuensi Kata Kunci <i>Antisemitic</i> (<i>Institute for Strategic Dialogue</i> , 2023).....	2
Gambar 2.1 Arsitektur Model LSTM (Young et al., 2018)	15
Gambar 2.2 Arsitektur BiLSTM	16
Gambar 2.3 Contoh <i>Use Case Diagram</i>	19
Gambar 2.4 Contoh <i>Activity Diagram</i> (Ahmad et al., 2019)	21
Gambar 2.5 Contoh <i>Deployment Diagram</i> (Bhatt & Nandu, 2021)	23
Gambar 3.1 Alur Tahapan Penelitian.....	27
Gambar 3.2 Contoh <i>Tweet</i> Sentimen Pro-genosida	28
Gambar 3.3 Contoh <i>Tweet</i> Sentimen Kontra-genosida.....	28
Gambar 3.4 Contoh <i>Tweet</i> Sentimen Netral	28
Gambar 3.5 <i>Flowchart</i> Proses <i>Case Folding</i>	37
Gambar 3.6 <i>Flowchart</i> Proses <i>Noise Removal</i>	38
Gambar 3.7 <i>Flowchart</i> Proses <i>Stopword Removal</i>	39
Gambar 3.8 <i>Flowchart</i> Proses <i>Tokenization</i>	40
Gambar 3.9 <i>Flowchart</i> Proses <i>Lemmatization</i>	41
Gambar 3.10 Arsitektur <i>Skip-Gram</i> (Mikolov et al, 2013).....	45
Gambar 3.11 <i>Flowchat</i> Model LSTM.....	53
Gambar 3.12 <i>Use Case Diagram</i> Aplikasi.....	58
Gambar 3.13 <i>Activity Diagram</i> Halaman Utama	60
Gambar 3.14 <i>Activity Diagram</i> Halaman <i>Text Preprocessing</i>	61
Gambar 3.15 <i>Activity Diagram</i> Halaman <i>Word Embedding</i>	62
Gambar 3.16 <i>Activity Diagram</i> Halaman <i>Sentiment Prediction</i>	63
Gambar 3.17 <i>Deployment Diagram</i>	64
Gambar 3.18 Desain Halaman Utama.....	65
Gambar 3.19 Desain Halaman <i>Data Scraping</i>	65
Gambar 3.20 Desain Halaman <i>Data Preprocessing</i>	66
Gambar 3.21 Desain Halaman <i>Data Balancing</i>	67
Gambar 3.22 Desain Halaman <i>Word Embedding</i>	67
Gambar 3.23 Desain Halaman <i>Model Evaluation</i>	68
Gambar 3.24 Desain Halaman <i>Model Comparison</i>	68
Gambar 3.25 Desain Halaman <i>Sentiment Prediction</i>	69
Gambar 4.1 Visualisasi Bobot TF-IDF Tertinggi	73
Gambar 4.2 Visualisasi 10 Kata dengan Bobot TF-IDF Tertinggi pada Kelas sentimen Pro-Genosida	74
Gambar 4.3 Visualisasi 10 Kata dengan Bobot TF-IDF Tertinggi pada Kelas sentimen Netral	75

Gambar 4.4 Visualisasi 10 Kata dengan Bobot TF-IDF Tertinggi pada Kelas sentimen Kontra-Genosida.....	75
Gambar 4.5 Visualisasi Frekuensi Kata	77
Gambar 4.6 Visualisasi Kata pada Kelas sentimen Pro-Genosida.....	78
Gambar 4.7 Visualisasi Kata pada Kelas sentimen Netral.....	79
Gambar 4.8 Visualisasi Kata pada Kelas sentimen Kontra-Genosida	79
Gambar 4.9 Vektor Kata “ <i>genocide</i> ” dari Model Word2Vec	79
Gambar 4.10 <i>Confusion Matrix</i> Pengujian Model Naive Bayes.....	82
Gambar 4.11 Grafik ROC Hasil Pengujian Model Naive Bayes	83
Gambar 4.12 <i>Confusion Matrix</i> Pengujian Model SVM	85
Gambar 4.13 Grafik ROC Hasil Pengujian Model SVM.....	86
Gambar 4.14 Grafik Akurasi dan Loss Model LSTM	92
Gambar 4.15 <i>Confusion Matrix</i> Pengujian Model LSTM	93
Gambar 4.16 Grafik ROC Hasil Pengujian Model LSTM.....	93
Gambar 4.17 Grafik Akurasi dan <i>Loss</i> Model BiLSTM.....	95
Gambar 4.18 <i>Confusion Matrix</i> Model BiLSTM.....	96
Gambar 4.19 Grafik ROC Model BiLSTM	97
Gambar 4.20 Halaman Utama.....	102
Gambar 4.21 Halaman Dataset Penelitian	103
Gambar 4.22 Halaman <i>Data Preprocessing</i>	104
Gambar 4.23 Halaman <i>Data Balancing</i>	106
Gambar 4.24 Halaman Ekstraksi Fitur TF-IDF	107
Gambar 4.25 Halaman Ekstraksi Fitur Word2vec	108
Gambar 4.26 Halaman Evaluasi Model	110
Gambar 4.27 Halaman Komparasi Model	111
Gambar 4.28 Halaman Prediksi Sentimen	112

DAFTAR TABEL

Tabel 2.1 Simbol-simbol pada <i>Use Case Diagram</i> (Sundaramoorthy, 2022)	18
Tabel 2.2 Simbol-Simbol pada <i>Activity Diagram</i> (Sundaramoorthy, 2022).....	20
Tabel 2.3 Simbol-simbol pada <i>Deployment Diagram</i> (Sundaramoorthy, 2022) ..	22
Tabel 2.4 Penelitian Terdahulu	23
Tabel 3.1 Hasil Sampel Data dari <i>Crawling Tweet</i>	29
Tabel 3.2 Hasil Label Data Manual	32
Tabel 3.3 Hasil Akhir Data Label	33
Tabel 3.4 Jumlah Dataset Berdasarkan Kategori Label	35
Tabel 3.5 Jumlah Data Berdasarkan Kategori Label setelah <i>Oversampling</i>	36
Tabel 3.6 Data Hasil Tahap <i>Case Folding</i>	37
Tabel 3.7 <i>Regular Expression</i>	38
Tabel 3.8 Data Hasil Tahap <i>Noise Removal</i>	38
Tabel 3.9 Data Hasil Tahap <i>Stopword Removal</i>	40
Tabel 3.10 Data Hasil Tahap <i>Tokenization</i>	40
Tabel 3.11 Data Hasil Tahap <i>Lemmatization</i>	41
Tabel 3.12 <i>Sliding Window</i>	46
Tabel 3.13 <i>One Hot Encoding</i>	46
Tabel 3.14 <i>Hidden Layer Word2Vec</i>	47
Tabel 3.15 <i>Output Layer Word2Vec</i>	48
Tabel 3.16 Komponen Tuning Hyperparameter Model Naive Bayes.....	51
Tabel 3.17 Komponen Tuning Hyperparameter Model SVM	52
Tabel 3.18 Komponen Tuning <i>Hyperparameter</i> Model LSTM.....	56
Tabel 3.19 Komponen Tuning Hyperparameter Model BiLSTM	57
Tabel 4.2 Hasil Pengujian Hyperparameter Nilai <i>Max Feature</i> TF-IDF	72
Tabel 4.3 Hasil Pengujian Hyperparameter <i>Ngram Range</i> TF-IDF	73
Tabel 4.4 20 Kata dengan Bobot TF-IDF Tertinggi	74
Tabel 4.5 Hasil Pengujian <i>Hyperparameter</i> Ukuran Dimensi Word2vec	76
Tabel 4.6 Hasil Pengujian <i>Hyperparameter</i> Ukuran <i>Window</i> Word2vec.....	76
Tabel 4.7 Frekuensi Kata Tertinggi	77
Tabel 4.8 Hasil Kemiripan Antar Kata	80
Tabel 4.9 Hasil Tuning Hyperparameter Naive Bayes Menggunakan <i>Grid Search Cross-Validation</i>	81
Tabel 4.10 Evaluasi Model Naive Bayes	81
Tabel 4.11 Hasil Tuning Hyperparameter SVM Menggunakan <i>Grid Search Cross-Validation</i>	84
Tabel 4.12 Evaluasi Model SVM.....	84
Tabel 4.13 Hasil Pengujian Jumlah Ukuran <i>Embedding Dimension</i>	87

Tabel 4.14 Hasil Pengujian Jumlah Unit LSTM.....	87
Tabel 4.15 Hasil Pengujian <i>Dropout</i>	88
Tabel 4.16 Hasil Pengujian <i>Batch Size</i>	89
Tabel 4.17 Hasil Pengujian <i>Learning Rate</i>	90
Tabel 4.18 Hasil Pengujian Ukuran <i>Epoch</i>	90
Tabel 4.19 Hasil Akhir Pengujian <i>Hyperparameter</i> Model LSTM.....	91
Tabel 4.20 Evaluasi Model LSTM.....	91
Tabel 4.21 Hasil Pengujian Hyperparameter <i>Merge Mode</i> pada Model BiLSTM	94
Tabel 4.22 Evaluasi Model BiLSTM	95
Tabel 4.23 Perbandingan Evaluasi Model	97
Tabel 4.24 Perbandingan Hasil Klasifikasi Model	98

DAFTAR LAMPIRAN

Lampiran 1 Daftar Stopword	119
Lampiran 2 Dataset Mentah	119
Lampiran 3 Kode Preprocessing Dataset	120
Lampiran 4 Source Code GridsearchCV Model Naive Bayes	122
Lampiran 5 Source Code GridsearchCV Model SVM	123
Lampiran 6 <i>Source Code Training</i> LSTM.....	124
Lampiran 7 <i>Source Code Training</i> BiLSTM	126

BAB I

PENDAHULUAN

Pada Bab ini menerangkan tentang Latar Belakang yang mendasari penelitian ini, kemudian identifikasi masalah dan batasan masalah yang dijabarkan dari latar belakang. Selain itu, Bab ini membahas tentang mandat dan tujuan dari penelitian, serta menjabarkan metodologi penelitian dan sistematika penulisan.

1.1 Latar Belakang

Tindakan kekerasan agresi Israel terhadap Palestina mengalami peningkatan secara signifikan sejak 7 Oktober 2023, agresi ini telah berlangsung selama lebih dari 77 tahun sejak tanggal 14 Mei 1948. Penjajahan tersebut diawali dengan perpecahan ideologi antara pemukim Israel dan Palestina, yang kemudian berkembang menjadi isu global dan geopolitik Internasional bahkan termasuk dalam kemanusiaan dan media.

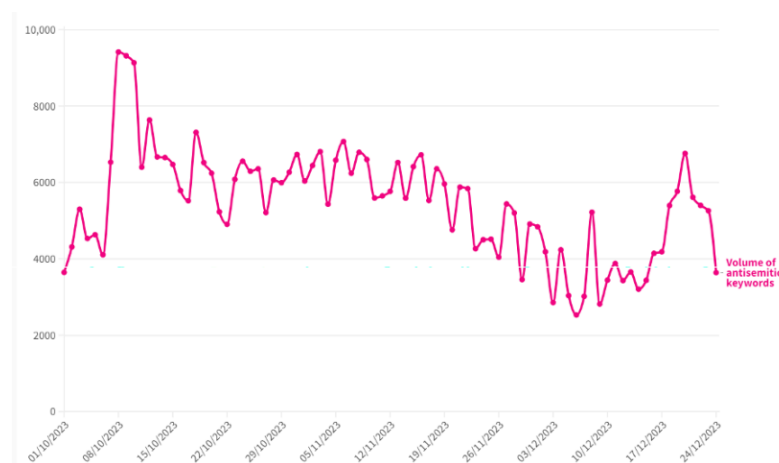
Serangan Isreal pada tanggal 7 Oktober 2023 disebut sebagai Operasi Banjir Al-Aqsa, yang merupakan salah satu faktor pemicu eskalasi kekerasan terbesar dalam sejarah penjajahan Israel terhadap Palestina. Serangan dimulai dengan pengeboman pada pemukiman sipil di wilayah Gaza oleh Israel, yang kemudian dibalas oleh kelompok pejuang kemerdekaan Palestina. Pada hari berikutnya, Israel meluncurkan lebih dari 2.000 bom ke wilayah Palestina dalam satu hari, dengan dalih membela diri.

Narasi “pembelaan diri” tersebut menyebar luas melalui media, Di mana banyak pemberitaan yang menyebutkan pejuang Palestina sebagai teroris

sedangkan Israel digambarkan sebagai pihak yang hanya membela diri setelah melakukan pengeboman tersebut. Kondisi ini mencerminkan fenomena yang disebut sebagai *Hasbara Project*, yaitu strategi propaganda digital yang dilakukan Israel untuk menyetir opini publik melalui media informasi (Jędrzejewska, 2020)

Sejak peristiwa 7 Oktober 2023, aktivitas opini publik pada media sosial meningkat secara drastis. Menurut laporan *Arab Center for the Advancement of Social Media* tercatat lebih dari 10 juta konten kekerasan diunggah dalam bahasa Ibrani pada tiga *platform* media sosial sejak tahun 2023, dan mengalami peningkatan konten sejak serangan 7 Oktober 2023 sebanyak 23 postingan setiap satu menit (*Arab Center*, 2023).

Selain itu, analisis dari *Institute for Strategic Dialogue* (2023), menunjukkan bahwa dalam rentang waktu dari 7 Oktober 2023 hingga 24 Desember 2023 terdapat lebih dari 460.000 unggahan yang mengandung kata-kunci “*antisemitic*”. Istilah “*antisemitic*” merujuk pada segala bentuk sikap, ujaran, ataupun tindakan yang menunjukkan diskriminasi terhadap individu maupun komunitas Yahudi.



Gambar 1.1 Grafik Frekuensi Kata Kunci *Antisemitic* (*Institute for Strategic Dialogue*, 2023)

Dengan demikian, media sosial menjadi wadah untuk mengungkapkan banyak hal, termasuk isu yang terjadi di Palestina saat ini. Terutama pada media sosial *Twitter* atau yang disebut juga sebagai X, pengguna twitter lebih masif dalam menyuarakan keberpihakan serta lebih vokal dalam menyampaikan opini mereka.

Namun dengan meningkatnya aktivitas opini publik pada media sosial tidak selalu diringi dengan informasi yang valid. Dalam isu genosida Palestina, ruang digital menjadi ruang pertarungan narasi yang berisi propaganda dan disinformasi, salah satunya adalah melalui *Hasbara Project* yang menggiring opini publik. Kondisi ini menyulitkan pengguna dalam memilah informasi berdasarkan fakta dan yang sudah dipengaruhi oleh penggiringan informasi. Oleh karena itu, diperlukan pendekatan berbasis data yang mampu mengidentifikasi sikap publik secara terstruktur, salah satunya adalah melalui analisis sentimen.

Penelitian ini bertujuan untuk mengklasifikasi sentimen pengguna X terhadap isu genosida di Palestina sejak 7 Oktober 2023 menjadi tiga kategori yaitu pro, netral dan kontra terhadap genosida. Melalui pendekatan analisis sentimen menggunakan model *machine learning* dengan harapan penelitian ini dapat memberikan pemahaman terkait komparasi setiap model yang digunakan.

Model *machine learning* sudah sering digunakan oleh beberapa penelitian untuk mengklasifikasikan sentimen. salah satu model yang sering digunakan adalah Naive Bayes (Junaedy et al., 2025) dan *Support Vector Machine* (Ryandi et al., 2025). Dan pada model deep learning seperti LSTM dan BiLSTM menjadi salah

satu model paling optimal dalam mengklasifikasikan sentimen dengan dataset yang besar (Eka Putra, 2024).

Penelitian ini melakukan analisis sentimen terhadap isu genosida di Palestina pada media sosial X dengan judul “**Komparasi Model *Machine Learning* dan *Deep Learning* untuk Analisis Sentimen Isu Genosida Palestina pada Media Sosial X**”. Penelitian ini bertujuan untuk mengetahui sentimen pengguna media sosial X terhadap isu genosida dan mendapatkan model yang paling optimal dalam mengklasifikasikan sentimen dalam konteks isu genosida.

1.2 Identifikasi Masalah

Berdasarkan latar belakang yang telah dijelaskan sebelumnya, identifikasi masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana tahapan analisis sentimen menggunakan model *machine learning* yang diterapkan untuk mengklasifikasikan *tweet* terkait isu genosida Palestina pada media sosial X?
2. Bagaimana mendapatkan kinerja algoritma *machine learning* yang optimal dalam mengklasifikasikan sentimen *tweet* terhadap isu genosida Palestina?
3. Bagaimana mengembangkan aplikasi berbasis web untuk mediseminasikan hasil klasifikasi sentimen *tweet* isu genosida Israel terhadap Palestina menggunakan *Unified Modelling Language*?

1.3 Batasan Masalah

Dari identifikasi masalah tersebut, maka dalam penelitian ini terdapat beberapa batasan masalah pada sebagai berikut:

1. Mempelajari cara kerja empat algoritma *machine learning* yaitu *Naive Bayes*, *Support Vector Machine (SVM)*, *Long Short-Term Memory (LSTM)*, dan *Bidirectional Long Short-Term Memory (BiLSTM)* untuk analisis sentimen *tweet* terhadap isu genosida Palestina.
2. Memperoleh performa model yang optimal untuk mengklasifikasikan sentimen berdasarkan *hyperparameter* yang diuji.
3. Dataset diambil dari *tweet* dengan kata kunci berupa '*genocide*', '*gaza*', '*israel*', dan '*palestine*' dalam bahasa Inggris dengan data yang dikumpulkan dari tanggal 7 Oktober 2023 sampai dengan 7 April 2025.
4. Implementasi aplikasi berbasis *web* menggunakan Python dan Streamlit untuk mendiseminasikan hasil klasifikasi sentimen.

1.4 Maksud dan Tujuan Penelitian

Berdasarkan identifikasi masalah, penelitian ini memiliki maksud untuk menganalisis sentimen dan membandingkan performa beberapa model *machine learning* dan *deep learning* dalam mengklasifikasikan sentimen *tweet* terhadap isu genosida Israel kepada Palestina.

Tujuan yang ingin dicapai oleh penulis dari penelitian ini adalah:

1. Mengembangkan empat model *machine learning* dan *deep learning*, yaitu *Naive Bayes*, *SVM*, *LSTM*, dan *BiLSTM*. untuk melakukan klasifikasi sentimen yang berkaitan dengan isu genosida Palestina.
2. Menentukan model dengan performa yang paling optimal melalui pengujian dan evaluasi model menggunakan beberapa *hyperparameter*, serta

membandingkan nilai akurasi, *precision*, *recall* dan *f1-score* pada setiap model.

3. Mengumpulkan dataset *tweet* yang berkaitan dengan isu genosida Palestina menggunakan kata kunci “*genocide*”, “*gaza*”, “*israel*”, dan “*palestine*” dalam bahasa Inggris untuk periode 7 Oktober 2023 sampai 7 April 2025 sebagai dasar analisis sentimen.
4. Mengembangkan aplikasi berbasis web yang berfungsi menampilkan hasil dari penelitian ini dan menggunakan *Unified Modelling Language* sebagai bentuk visualisasi dalam merancang sistem.

1.5 Manfaat Penelitian

Manfaat yang diharapkan dari penelitian ini adalah untuk mengetahui opini masyarakat terhadap isu genosida Israel kepada Palestina melalui hasil analisis sentimen. Penelitian ini juga dapat dimanfaatkan oleh pihak lain dalam penelitian lebih lanjut. Selain itu, penelitian ini dapat dimanfaatkan oleh pihak maupun komunitas aktivis Palestina sebagai referensi dalam mengklasifikasi sentimen berbasis teks.

1.6 Metodologi Penelitian

Jenis penelitian yang akan digunakan adalah Kuantitatif Eksperimental dengan pendekatan *text mining* dan *machine learning*, khususnya pada model *machine learning* dan *deep learning*. Penelitian ini menggunakan metodologi CRISP-DM dengan rincian tahapan sebagai berikut:

1. *Business Understanding*, yaitu proses memahami tujuan dan persyaratan proyek dari prespektif bisnis. Proses ini terdiri dari menentukan tujuan, mengumpulkan persyaratan dan menyusun rancangan proyek.
2. *Data Understanding*, yaitu proses memahami data yang akan digunakan. Proses ini terdiri dari pengumpulan data, mempelajari data dan memastikan kualitas dari data yang telah dikumpulkan.
3. *Data Preparation*, yaitu proses mempersiapkan data sebelum digunakan untuk membuat model. Proses ini terdiri dari menyeleksi atribut dan data yang akan digunakan, membersihkan data dan mentransformasi data ke dalam format yang sama.
4. *Modeling*, yaitu proses membangun model-model dari algoritma yang telah ditentukan dengan mencari parameter yang optimal.
5. *Evaluation*, yaitu proses yang dilakukan untuk mengevaluasi model yang telah dibangun, seperti menguji model yang dibangun kemudian melakukan analisis terhadap hasil pengujian model tersebut.
6. *Deployment*, yaitu proses melakukan *deploy* dari model optimal terpilih sehingga dapat digunakan pada proses bisnis.

1.7 Sistematika Penulisan

Untuk memberi gambaran yang jelas tentang penelitian ini, maka disusunlah sistematika penulisan yang berisi materi yang akan dibahas pada setiap bab. Sistematika dalam penulisan tugas akhir ini adalah sebagai berikut:

BAB I PENDAHULUAN

Pada bab ini dijelaskan tentang latar belakang dari topik penulisan skripsi, pokok permasalahan berupa identifikasi dan batasan masalah, tujuan dan manfaat yang diharapkan dari penulisan skripsi, metodologi yang digunakan serta sistematika penulisan.

BAB II TINJAUAN PUSTAKA

Pada bab ini berisi penjelasan mengenai teori dan penelitian terkait yang menjadi dasar penelitian ini, seperti mengenai analisis sentimen, *data preprocessing*, dan model *machine learning* dan *deep learning*.

BAB III ANALISIS DAN PERANCANGAN

Pada bab ini dijelaskan secara rinci mengenai tahapan penelitian, seperti tahap menyiapkan data, komputasi model dan analisis kebutuhan aplikasi.

BAB IV HASIL DAN PEMBAHASAN

Pada bab ini dibahas hasil dari penelitian yang dilakukan dan analisa terhadap hasil implementasi tersebut.

BAB V KESIMPULAN DAN SARAN

Bab ini merupakan penutup yang berisi kesimpulan dan saran dari penelitian yang sudah dilakukan.

BAB II

TINJAUAN PUSTAKA

Bab ini menjelaskan landasan teori terhadap topik dan variabel yang digunakan pada penelitian ini. Yaitu berupa teori terkait analisis sentimen, dan teori dasar terkait *text mining*, *machine learning*, *deep learning*, dan *unified modelling language*.

2.1 Analisis Sentimen

Sentimen adalah perasaan yang mendasari, sikap, evaluasi, atau emosi yang terkait dengan opini. Orientasi sentimen bisa positif, negatif, atau netral. Target sentimen atau dikenal juga sebagai target opini, adalah entitas atau aspek entitas tempat sentimen tersebut diungkapkan (Liu, 2010). Analisis sentimen atau *opinion mining* adalah studi komputasional tentang opini orang, sentimen, penilaian, sikap, dan emosi terhadap entitas dan aspek-aspeknya yang diekspresikan melalui teks (Bounds, 2017).

Secara umum terdapat dua metode yang biasa digunakan dalam melakukan analisis sentiment, yaitu dengan pendekatan *rule based/lexicon based* dan pendekatan dengan *machine learning/supervised learning* (Liu, 2010). Keuntungan utama dari pendekatan *supervised learning* untuk klasifikasi sentimen adalah dapat belajar secara otomatis dari semua jenis fitur untuk klasifikasi melalui optimasi. Sebagian besar fitur ini sulit digunakan dengan metode *lexicon based*. Namun, metode *supervised learning* bergantung pada data pelatihan, yang perlu diberi label secara manual untuk setiap domain. Kelemahan dari pendekatan ini adalah

supervised learning yang dilatih dari data berlabel di satu domain sering kali tidak berfungsi di domain lainnya. Sehingga untuk setiap domain, data pelatihan baru perlu diberi label yang memakan waktu lama (Bounds, 2017).

2.2 *Natural Language Processing (NLP)*

Natural Language Processing (NLP) merupakan salah satu bidang dalam *artificial intelligence* yang berfokus pada interaksi komputer dan bahasa manusia. NLP memungkinkan komputer untuk memproses, memahami dan menghasilkan bahasa secara otomatis melalui serangkaian teknik komputasional (Jurafsky & Martin, 2025).

NLP juga digunakan untuk mengekstrak informasi dan makna dari data berbasis bahasa, seperti opini, emosi, atau fakta yang terdapat pada teks tersebut. NLP menjadi salah satu landasan utama dalam berbagai penelitian berbasis analisis teks. Seperti analisis sentimen, klasifikasi dokumen, sistem tanya jawab, *speech recognition*, dan *machine translation*

Proses NLP dalam implementasi model mencakup menjadi 2 tahapan (Young et al., 2018)

1. *Natural Language Understanding (NLU)*

Tahapan untuk memahami maksud pesan dalam teks yang diberikan, termasuk analisis tata bahasa, semantik dan konteks.

2. *Natural Language Generation (NLG)*

Tahapan untuk menghasilkan teks atau ucapan dengan struktur bahasa yang alami dan dapat dipahami oleh manusia.

Dalam pengembangan sistem analisis sentimen, NLP berperan penting sebagai sistem yang mengonversi teks menjadi representasi numerik yang dapat dipelajari oleh model *machine learning* atau *deep learning*. Berbagai teknik dalam NLP berupa *tokenizing*, *stemming*, dan *stopword removal* menjadi salah satu proses penting pada tahap preprocessing sebelum dianalisis lebih lanjut menggunakan model *machine learning* maupun *deep learning*.

2.3 Text Preprocessing

Text preprocessing adalah sebuah tahap awal dari *text mining* untuk mempersiapkan data teks, seperti membersihkan data dari *noise*, ataupun mengubah format data sehingga mempermudah dalam melakukan pemrosesan data. *Text preprocessing* merupakan salah satu bagian yang penting dalam melakukan analisis sentimen, karena hasilnya akan berpengaruh terhadap performa klasifikasi sentimen yang akan dilakukan (Kobayashi et al., 2018).

Berikut ini merupakan beberapa tahap yang umumnya dilakukan pada proses *text preprocessing* :

1. Case Folding

Case Folding merupakan proses mengubah huruf dalam teks menjadi huruf kecil semua. Tujuannya untuk menghindari perbedaan makna kata yang sama hanya karena perbedaan huruf besar dan kecil.

2. Noise Removal

Noise Removal adalah proses menghapus karakter atau elemen yang tidak memiliki peran terhadap pemahaman konteks dalam teks. Seperti angka, tanda baca,

link, emotikon, dan simbol atau karakter khusus lainnya. Langkah ini dapat mengurangi gangguan pada proses analisis.

3. *Tokenizing*

Tokenisasi adalah proses memecah suatu kalimat atau teks menjadi satuan yang lebih kecil yang disebut sebagai token. Proses ini diperlukan agar setiap bagian teks dapat dianalisis secara lebih mendalam.

4. *Normalizing*

Normalizing merupakan tahap penyeragaman penulisan kata sehingga kata dengan makna yang sama memiliki bentuk kata yang lebih konsisten. Normalisasi mencakup perbaikan penulisan salah, mengubah bahasa gaul menjadi bahasa baku, dan memperbaiki singkatan.

5. *Stemming*

Stemming merupakan proses mengubah kata berimbuhan menjadi bentuk kata dasarnya. Dalam bahasa Inggris, stemming biasa digunakan untuk menghilangkan kata yang memiliki sifat telah terjadi (*past tense*) atau sedang terjadi. Sebagai contoh, kata '*attacking*' akan berubah menjadi kata dasarnya yaitu '*attack*'.

6. *Stopword Removal*

Stopword removal adalah proses menghilangkan kata-kata yang sering muncul pada teks namun tidak memiliki kontribusi penting dalam konteks analisis. Dalam bahasa Inggris kata-kata yang termasuk stopwords adalah "*what*", "*were*", "*it*", dan sebagainya. Dengan menghapus stopwords, model dapat lebih mudah memproses kata dengan dataset yang lebih informatif dalam menganalisis sentimen. (Steven Bird et al, 2009)

2.4 *Machine Learning*

Machine learning merupakan pendekatan berbasis data yang memungkinkan komputer mengenali pola secara otomatis tanpa perlu aturan eksplisit. Dalam analisis sentimen, algoritma *machine learning* dilatih menggunakan data teks yang telah diberi label yang kemudian memprediksi kategori sentimen dari teks baru.

Keunggulan metode ini adalah kemampuan untuk beradaptasi terhadap berbagai domain data dan memperoleh akurasi yang tinggi melalui proses pelatihan model. Namun, performanya sangat bergantung pada kualitas data dan pemilihan fitur yang tepat. (Freddy et al., 2025).

2.4.1 *Naive Bayes*

Naive Bayes adalah salah satu algoritma klasifikasi berbasis probabilistik yang sering digunakan dalam analisis sentimen karena proses perhitungannya yang sederhana dan proses komputasi yang cepat. Algoritma ini bekerja dengan menghitung probabilitas kemunculan kata dalam sebuah kelas sentimen lalu mengasumsikan bahwa setiap fitur bersifat independen. Naive Bayes dapat memberikan akurasi yang cukup baik terutama pada data teks dengan jumlah yang banyak (Kusuma Wardani & Arum Sari, 2021).

Namun, kelemahan utama dari algoritma Naive Bayes adalah algoritma yang sering mengalami bias dalam mengklasifikasi teks ketika dataset terlalu rumit dan berantakan sehingga tidak dapat mengidentifikasi teks dengan benar, dan performanya dapat bersaing dengan algoritma lain seperti SVM (Kusuma Putri et al., 2025)

2.4.2 *Support Vector Machine (SVM)*

Support Vector Machine (SVM) merupakan algoritma *supervised learning* yang berfokus pada pencarian *hyperlane* atau batas keputusan terbaik yang memisahkan kelas-kelas label. SVM banyak digunakan dalam analisis sentimen karena dapat bekerja optimal pada data teks dengan jumlah yang banyak. Algoritma SVM terbukti memiliki tingkat akurasi lebih baik dibandingkan Naive Bayes dalam mengklasifikasikan opini masyarakat, termasuk juga pada analisis sentimen, dengan data yang cenderung banyak dan menggunakan bahasa alami atau bahasa manusia (Kusuma Putri et al, 2025)

2.5 *Deep Learning*

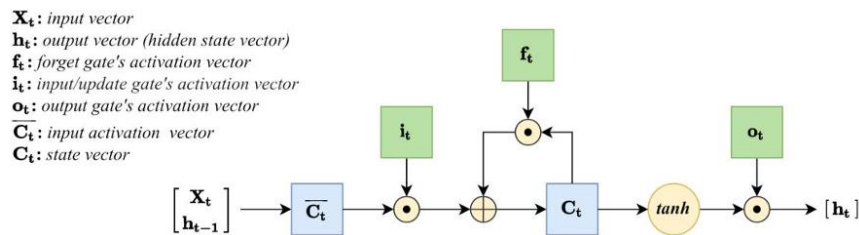
Deep learning merupakan salah satu cabang yang dikembangkan dari *machine learning* yang menggunakan arsitektur jaringan saraf dengan banyak lapisan, atau biasa disebut sebagai *deep neural network*. Lapisan jaringan ini biasa digunakan untuk melakukan proses pembelajaran secara otomatis dengan data dalam jumlah yang besar. *Deep learning* adalah algoritma yang mampu memahami pola yang lebih kompleks pada data teks, salah satunya termasuk dalam analisis sentimen (Fitroh & Hudaya, 2023).

Algoritma *deep learning* memiliki kemampuan dalam mempelajari konteks dan struktur bahasa, sehingga model yang menggunakan *deep learning* sering kali mendapatkan hasil akurasi yang lebih tinggi (Derbentsev et al., 2023)

2.5.1 *Long Short-Term Memory*

Long Short-Term Memory (LSTM) merupakan salah satu arsitektur jaringan saraf tiruan yang termasuk dalam kategori *Recurrent Neural Network (RNN)*.

LSTM dirancang secara khusus untuk mengatasi kelemahan utama RNN yaitu *vanishin gradient* ketika memproses data berurutan dalam rentang waktu yang panjang (Hochreiter & Schmidhuber, 1997).



Gambar 2.1 Arsitektur Model LSTM (Young et al., 2018)

LSTM memiliki struktur memori internal yang disebut *cell state*, serta tiga gerbang utama yaitu *input gate*, *forget gate*, dan *output gate* yang memungkinkan model untuk menyimpan, menghapus, atau meneruskan informasi tertentu pada setiap waktu (Staudemeyer & Morris, 2019).

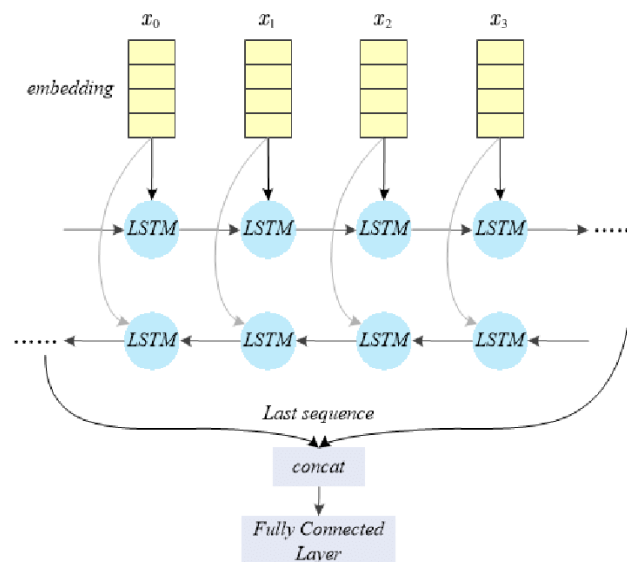
Dalam konteks pemrosesan bahasa alami, LSTM banyak digunakan untuk menangkap hubungan urutan kata dalam kalimat karena kemampuannya dalam memahami konteks jangka panjang. Salah satu penerapan paling umum adalah pada analisis sentimen, LSTM dapat mengklasifikasikan ekspresi opini dalam bentuk teks menjadi berbagai macam kategori sentimen. LSTM bekerja dengan menerima *input* berupa data teks yang sudah di *word embedding* dan kemudian memproses urutan kata tersebut untuk mengenali pola-pola yang menunjukkan arah sentimen tertentu (Zhang et al, 2018).

Keunggulan utama dari penggunaan LSTM dalam analisis sentimen adalah kemampuannya untuk mempertahankan informasi penting dalam urutan kata yang panjang. Namun, model ini tetap bergantung pada kualitas *embedding* dan *training data* yang digunakan. Model LSTM akan bekerja optimal ketika dikombinasikan

dengan teknik *word embedding* dan ketika dilatih *data training* yang digunakan adalah data yang sesuai dengan domain analisisnya (Vijayakumar, 2024)

2.5.2 Bidirectional Long Short-Term Memory (BiLSTM)

Bidirectional Long Short-Term Memory (BiLSTM) merupakan pengembangan lanjutan dari arsitektur Long Short-Term Memory (LSTM) yang dirancang untuk memproses data sekuensial secara dua arah, yaitu dari depan ke belakang (*forward*) dan dari belakang ke depan (*backward*). Pada model ini, dua lapisan LSTM dijalankan secara paralel, di mana satu lapisan mempelajari informasi masa lalu menuju masa depan, sedangkan lapisan lainnya mempelajari informasi dari masa depan menuju masa lalu. Hasil dari kedua proses tersebut kemudian digabungkan untuk menghasilkan representasi konteks yang lebih kaya dan komprehensif (Migliardi et al., 2024).



Gambar 2.2 Arsitektur BiLSTM (Zhao et al., 2020)

Dalam analisis sentimen, kemampuan BiLSTM untuk memahami konteks dua arah sangat penting karena makna suatu kata sering kali dipengaruhi oleh kata-

kata yang muncul sebelum dan sesudahnya. Oleh karena itu, BiLSTM banyak digunakan dalam klasifikasi teks dan analisis sentimen karena mampu meningkatkan pemahaman konteks kalimat serta memberikan hasil klasifikasi yang lebih stabil pada data teks beraturan (Yin et al., 2017).

Struktur *Bidirectional Long Short-Term Memory* (BiLSTM) merupakan pengembangan tingkat lanjut dari model LSTM dengan menambahkan pemrosesan dua arah, yaitu dari depan ke belakang (*forward*) dan dari belakang ke depan (*backward*). Dengan kemampuan tersebut model BiLSTM dapat memahami konteks kalimat secara lebih menyeluruh karena mempertimbangkan informasi dari yang sudah lalu dan yang akan datang.

2.6 *Unified Modeling Language* (UML)

Unified modelling language atau disingkat UML adalah notasi grafis yang digunakan untuk menggambarkan, merancang dan mendokumentasikan sistem perangkat lunak (Fowler, 2004). UML terdiri dari berbagai jenis diagram yang digunakan untuk merepresentasikan berbagai aspek dari perangkat lunak seperti struktur, aktivitas, dan interaksi.

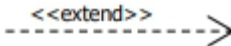
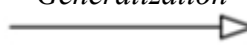
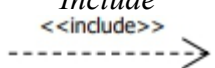
Pada umumnya UML digunakan sebagai sketsa atau *blueprint* dari sistem perangkat lunak yang dikembangkan. Diagram UML dapat digunakan sebagai media komunikasi pengembang baik dalam tim yang sama maupun berbeda. Beberapa diagram UML yang umum digunakan dalam pengembangan perangkat lunak adalah *Use Case Diagram*, *Activity Diagram*, *Sequence Diagram* dan *Class Diagram*.

2.6.1 Use Case Diagram

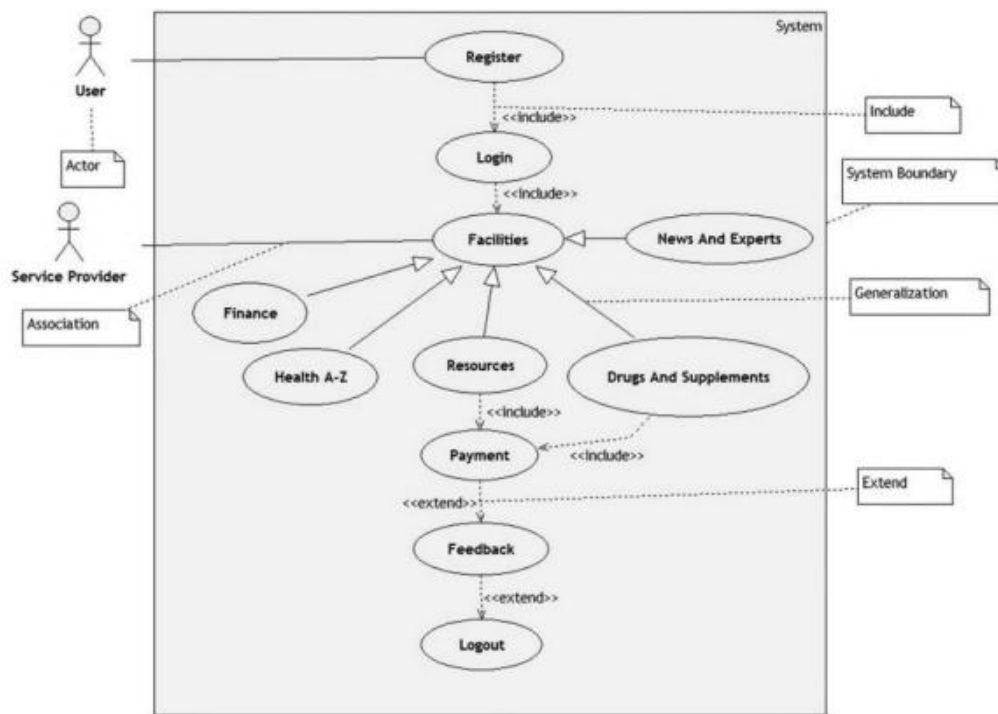
Use case diagram merupakan tampilan grafis yang menjelaskan alur *use case* (fitur/kasus penggunaan). Diagram ini menunjukkan batasan fitur dan interaksi yang disediakan aplikasi. *Use case diagram* memiliki 3 komponen yaitu aktor, *use cases*, dan hubungan antara aktor dan *use case*.

Use case diagram digunakan untuk memudahkan pemahaman kebutuhan fungsional aplikasi. *Use case diagram* sebaiknya dibuat tampilan yang sederhana dan tidak terlalu rumit untuk memudahkan ketika membaca dan memahaminya. Masing-masing simbol dalam *use case diagram* dijelaskan pada Tabel 2.1.

Tabel 2.1 Simbol-simbol pada *Use Case Diagram* (Sundaramoorthy, 2022)

No	Simbol	Keterangan
1		Fungsionalitas yang disediakan sistem sebagai unit-unit yang saling bertukar pesan antar unit atau aktor
2		Orang atau sistem lain yang berinteraksi dengan sistem informasi yang akan dibuat. Biasanya dinyatakan menggunakan kata benda atau nama aktor.
3		Komunikasi antara aktor dengan <i>use case</i> yang berpartisipasi pada diagram
4		Relasi <i>use case</i> tambahan ke sebuah <i>use case</i> , Di mana <i>use case</i> yang ditambahkan dapat berdiri sendiri meski tanpa <i>use case</i> tambahan itu. Arah panah mengarah pada <i>use case</i> yang ditambahkan.
5		Hubungan generalisasi dan spesialisasi (umum-khusus) antar dua buah <i>use case</i> Di mana fungsi yang satu adalah fungsi yang lebih umum dari lainnya.
6		Relasi <i>use case</i> tambahan ke sebuah <i>use case</i> , Di mana <i>use case</i> yang ditambahkan memerlukan <i>use case</i> ini untuk menjalankan fungsinya atau sebagai

No	Simbol	Keterangan
		syarat dijalankan <i>use case</i> ini. Arah panah <i>include</i> mengarah pada <i>use case</i> yang dibutuhkan

Gambar 2.3 Contoh *Use Case Diagram*

Gambar 2.1 merupakan salah satu contoh penggunaan *Use case diagram* pada aplikasi. Pada diagram tersebut, terdapat dua aktor yang dapat melakukan berbagai aktivitas dengan ketentuan aturan-aturan yang terdapat pada tabel penggunaan *Use Case Diagram*.

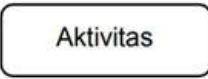
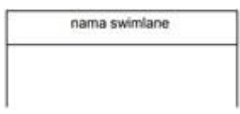
2.6.2 *Activity Diagram*

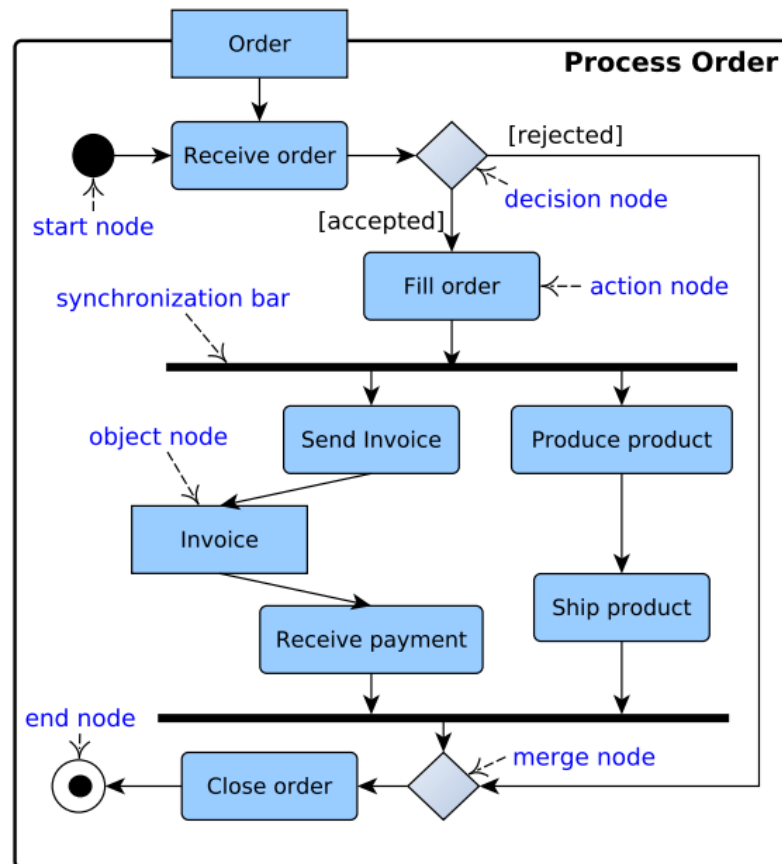
Activity Diagram adalah teknik untuk menggambarkan logika prosedural, proses bisnis dan alur kerja aplikasi, Diagram ini menampilkan rangkaian aktivitas yang terjadi dalam aplikasi. *Activity diagram* digunakan untuk membantu pengembang dan pihak yang terlibat dalam pembangunan aplikasi dalam

memahami proses atau alur kerja yang terjadi dalam pembangunan aplikasi, bagaimana interaksi antar tugas terjadi, dan bagaimana keputusan akan memengaruhi alur kerja.

Activity diagram memiliki beberapa komponen utama yang ditampilkan dalam bentuk simbol. Masing-masing simbol mewakili suatu elemen atau aktivitas yang terjadi pada sistem. Penjelasan dari masing-masing simbol pada *activity diagram* dijelaskan pada Tabel 2.2.

Tabel 2.2 Simbol-Simbol pada *Activity Diagram* (Sundaramoorthy, 2022)

No	Simbol	Keterangan
1	Status Awal 	Menandakan tindakan awal atau titik awal aktivitas untuk setiap diagram aktivitas.
2	Aktivitas 	Menunjukkan aktivitas yang dilakukan sistem
3	Percabangan / <i>decision</i> 	Asosiasi percabangan adalah keadaan Di mana terdapat pilihan aktivitas lebih dari satu.
4	Penggabungan / <i>join</i> 	Asosiasi penggabungan adalah keadaan Di mana lebih dari satu aktivitas digabungkan menjadi satu.
5	Status akhir 	Menunjukan bagian akhir dari aktivitas.
6	<i>Swimlane</i> 	<i>Swimlane</i> memisahkan organisasi bisnis yang bertanggung jawab terhadap aktivitas yang terjadi.



Gambar 2.4 Contoh *Activity Diagram* (Ahmad et al., 2019)

Pada gambar 2.2 merupakan alur *Activity Diagram* yang digunakan oleh *User* ketika melakukan transaksi pembelian produk pada suatu aplikasi. Pada *Activity Diagram* tersebut, terdapat interaksi pada *User* dan *Service Provider*. Di mana *user* akan menginputkan informasi seperti mengisi jumlah pemesanan, melakukan pembayaran, dan menerima nota transaksi untuk melakukan transaksi pembelian produk.

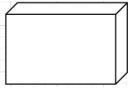
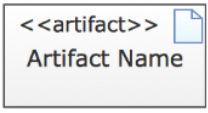
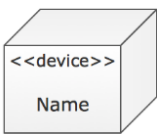
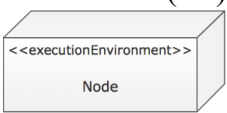
2.6.3 *Deployment Diagram*

Deployment diagram merupakan diagram yang digunakan untuk menggambarkan bagaimana komponen-komponen perangkat lunak dijalankan pada infrastruktur fisik sistem. Diagram ini menampilkan konfigurasi pada

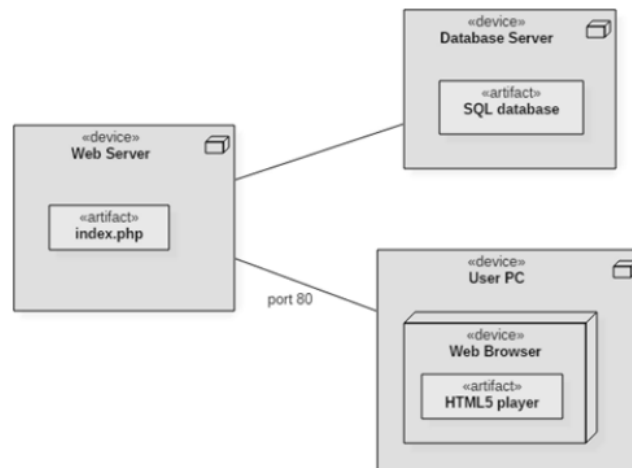
hardware, node server, jaringan, serta hubungan antar perangkat di mana sistem dideploy. Deployment diagram membantu pengembang dan pihak terkait untuk memahami arsitektur secara jelas, khususnya mengenai letak area eksekusi aplikasi, database dan lainnya.

Pada *deployment diagram*, objek utama yang digunakan adalah *node* yang merepresentasikan perangkat fisik. *Artifacts* merepresentasikan perangkat lunak yang di-deploy, serta *communication path* sebagai penghubung komunikasi antar *node*. Penjelasan setiap simbol dalam deployment diagram dapat dilihat pada Tabel 2.3.

Tabel 2.3 Simbol-simbol pada *Deployment Diagram* (Sundaramoorthy, 2022)

No	Simbol	Keterangan
1	<i>Node</i> 	Representasi perangkat fisik atau lingkungan eksekusi tempat aplikasi berjalan, seperti server, komputer klien, atau perangkat mobile.
2	<i>Artifact</i> 	File atau komponen perangkat lunak yang di-deploy pada node, seperti file aplikasi, library, script, atau basis data.
3	<i>Communication Path</i> 	Jalur komunikasi antar node yang menunjukkan koneksi jaringan atau komunikasi data antara perangkat.
4	<i>Device</i> 	Node yang berupa perangkat keras fisik seperti server hosting, smartphone, atau router jaringan.
5	Execution Environment (EE) 	Node yang merepresentasikan platform atau container eksekusi, seperti web server, JVM, browser, atau OS.

Gambar 2.3 berikut merupakan contoh *deployment diagram* yang menunjukkan interaksi antar *node* dalam sebuah sistem berbasis website.



Gambar 2.5 Contoh *Deployment Diagram* (Bhatt & Nandu, 2021)

2.7 Penelitian Terdahulu

Perkembangan *machine learning* dalam analisis berbasis teks banyak dimanfaatkan untuk memahami opini publik di media sosial. Salah satu metode yang sering digunakan adalah analisi sentimen untuk mendapatkan model paling optimal dalam mengklasifikasikan kelas sentimen. Berikut terdapat beberapa penelitian terdahulu yang melakukan analisis sentimen maupun komparasi model *machine learning* dalam klasifikasi sentimen.

Tabel 2.4 Penelitian Terdahulu

No	Penulis (Tahun)	Judul Penelitian	Metode Penelitian	Hasil Penelitian
1	Noori et al. (2025)	<i>Sentiment Analysis of the Israel-Palestine Conflict on X: Insights from the Indonesian Perspective using a Long Short-Term Memory Algorithm</i>	1.700 tweet Menggunakan model LSTM	Akurasi model sebesar 91%

2		Perbandingan Model Machine Learning dalam Analisis Sentimen Ulasan Pengunjung Keraton Yogyakarta pada Google Maps	Menggunakan Naive Bayes, SVM dan Logistic Regression	Model SVM paling optimal dengan akurasi sebesar 87,12%
3	Anthony & Barus (2025)	Perbandingan Algoritma Machine Learning dan Deep Learning Dalam Analisis Sentimen Komentar Video YouTube Berjudul Hidup Tanpa Sosial Media	Menggunakan model LSTM untuk DL Model Naive Bayes, Logistic Regression dan RNN untuk ML	Model Logistic Regression paling optimal dengan akurasi 84% mengalahkan LSTM
4	Eka Putra (2023)	Perbandingan Algoritma LSTM dan BiLSTM Untuk Analisis Sentimen Multi-Class Media Sosial Twitter.	Data Twitter multi-kelas model LSTM dan BiLSTM	LSTM akurasi 58%, BiLSTM 60%
5	G. Kusuma P., H. Sujaini, D. Isna U. (2025)	Perbandingan Algoritma Support Vector Machine (SVM) dan Naive Bayes pada Analisis Sentimen Bahasa Jawa dan Sunda	Membandingkan model Naive Bayes dan SVM	Model SVM lebih optimal dengan akurasi 80%
6	Muhammad Huda et al. (2024)	<i>Comparative Analysis of LSTM, BiLSTM, GRU, CNN, AND RNN for Depression Detection in Social Media</i>	Komparasi model LSTM, BiLSTM, GRU, CNN dan RNN	Model BiLSTM paling unggul dengan akurasi 98,45%
7	Anhsori & Shidik, (2025)	<i>A Comparative Analysis of Eight Machine Learning Models for Climate Change Sentiment Analysis</i>	Dataset dari twitter Model yang digunakan : LR, SVM, XGB, RF, NB, KNN dan GBM	Sentimen bersifat multikelas Diungguli oleh model SVM dengan akurasi 87,92%

Berdasarkan penelitian terdahulu yang dapat dilihat pada Tabel 2.4, dapat disimpulkan bahwa analisis sentimen pada umumnya hanya menggunakan satu model tertentu, ataupun melakukan perbandingan dengan algoritma dalam satu ranah yang sama. Seperti penelitian yang dilakukan oleh Anshori dan Shiddik (2025) yang melakukan analisis komparasi namun berfokus pada algoritma *machine learning* saja. Adapun penelitian Anthony & Barus (2025) yang

melakukan analisis komparasi dengan model *machine learning* dan deep learning, namun topik yang digunakan adalah komentar pada *platform* YouTube. Sedangkan pada penelitian Noori et al. (2025) terdapat kemiripan pada topik analisis sentimen namun pada penelitian tersebut hanya menggunakan satu model untuk klasifikasi sentimen.

Oleh karena itu, terdapat gap penelitian yaitu belum adanya analisis sentimen terhadap isu genosida Palestina yang membandingkan model *machine learning* dan *deep learning*. Penelitian ini bertujuan untuk membandingkan performa algoritma Naive Bayes, SVM, LSTM, dan BiLSTM serta mengembangkan aplikasi berbasis web untuk mendiseminasikan hasil klasifikasi sentimen secara informatif.

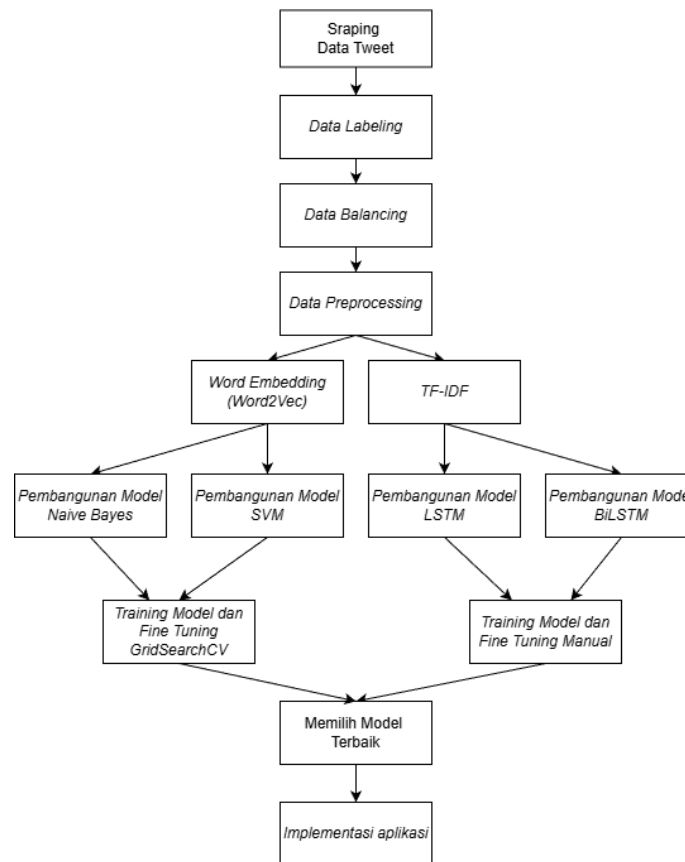
BAB III

ANALISIS DAN PERANCANGAN

Sebagaimana yang telah dijelaskan pada bab sebelumnya, telah disajikan tinjauan pustaka yang berkaitan dengan penelitian ini. Selanjutnya dilakukan proses analisis dan perancangan penelitian, mulai dari data penelitian yang digunakan, langkah-langkah penelitian, hingga evaluasi kinerja model.

3.1 Tahapan Penelitian

Penelitian ini dilakukan dengan beberapa tahapan yang meliputi pengumpulan data, pelabelan data, *data balancing*, *data preprocessing*, pengelompokan data *training* dan *validation*, pembobotan fitur menggunakan TF-IDF dan Word2vec, proses perancangan dan pembangunan empat model, dan terakhir implementasi aplikasi. Gambar 3.1 merupakan diagram alur tahapan penelitian yang dilakukan.



Gambar 3.1 Alur Tahapan Penelitian

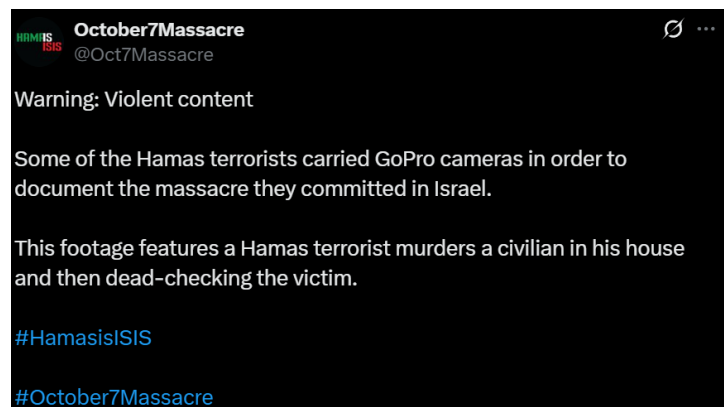
3.2 Objek Penelitian

Data dalam penelitian ini merupakan data primer dari *tweet* pengguna pada platform medial sosial X (Twitter) yang dikumpulkan dari rentang waktu 7 oktober 2023 hingga 7 April 2025. Data diperoleh dengan melakukan *scrapping* menggunakan kata kunci terkait isu genosida Isreal terhadap Palestina. Dari proses tersebut, data yang berhasil dikumpulkan dalam penelitian ini sebanyak 1702 *tweet* yang kemudian dianalisis untuk mengetahui opini publik terhadap isu tersebut.

Tweet yang telah dikumpulkan kemudian diklasifikasikan ke dalam tiga kategori sentimen. Yaitu pro, netral, dan kontra-genosida. Sentimen kategori pro-genosida berdasar pada opini yang menunjukkan dukungan terhadap tindakan

agresi Israel atau membenarkan propaganda yang disebar oleh Israel. sedangkan kategori sentimen kontra-genosida berdasar pada opini publik yang menyerukan dukungannya terhadap kemerdekaan Palestina. Dan terakhir kategori sentimen netral mencakup *tweet* yang tidak menunjukkan keberpihakan, seperti menyampaikan informasi faktual.

Sebagai ilustrasi, pada Gambar 3.2 ditampilkan contoh *tweet* yang termasuk dalam kategori sentimen pro-genosida, kemudian pada Gambar 3.3 menunjukkan contoh *tweet* kontra-genosida. Dan terakhir pada Gambar 3.4 menampilkan contoh *tweet* dengan sentimen netral.



Gambar 3.2 Contoh *Tweet* Sentimen Pro-genosida



Gambar 3.3 Contoh *Tweet* Sentimen Kontra-genosida



Gambar 3.4 Contoh *Tweet* Sentimen Netral

3.3 Pengumpulan Data

Data yang digunakan pada penelitian ini merupakan data berbentuk teks yang diambil dari media sosial X (Twitter). Media sosial X merupakan salah satu *platform* dengan kapasitas percakapan publik yang sangat tinggi dan intens. Terutama dalam mengganggapi isu-isu global secara *real-time*. Pengumpulan data dilakukan dengan menggunakan teknik *crawling* secara *online*, yaitu proses pengambilan data secara otomatis dengan bantuan bahasa pemograman bahasa Python dan menggunakan *library selenium* untuk melakukan *crawling* secara otomatis. Teknik *crawling* telah menjadi teknik umum untuk mengumpulkan data dalam waktu cepat pada topik tertentu.

Pada proses *crawling* data, peneliti menggunakan beberapa kata kunci yang telah disesuaikan dengan objek penelitian penulis. Kata kunci tersebut adalah “*genocide*”, “*gaza*”, dan “*Palestine*”. Adapun data yang dikumpulkan menggunakan kata kunci yang telah disensor atau sebagian huruf digantikan dengan simbol seperti “*gen0cide*”, “*g@z@*”, dan “*P4lestine*” untuk meloloskan konten tersebut tanpa terhapus oleh media sosial X yang mendeteksi konteks tersebut sebagai konteks yang sensitif atau tidak sesuai dengan kebijakan penggunaan media sosial X. Data yang dikumpulkan merupakan data *tweet* berupa teks berbahasa Inggris.

Tabel 3.1 Hasil Sampel Data dari *Crawling Tweet*

<i>No</i>	<i>Created_at</i>	<i>Favorite_count</i>	<i>Full_text</i>	<i>Id_str</i>
1.	Sun Apr 06 23:39:20 +0000 2025	43	@sentdefender The whole world stands with Israel and the IDF. End the Islamic regime of Iran.	1909029 5603534 97600

<i>No</i>	<i>Created_at</i>	<i>Favorite_count</i>	<i>Full_text</i>	<i>Id_str</i>
2.	Sun Apr 06 23:39:20 +0000 2025	0	<i>I agree. Israel's war is with Hamas NOT the Palestinian people. Can never understand why the UK won't stand with Palestine. The state of Isreal didn't exist until 1948 when Britain thought it was a good idea to resettle them in Palestine.</i>	1909028 1782214 70200
3.	Sun Apr 06 23:39:20 +0000 2025	0	<i>@EliAfriatISR I stand with Isreal.</i>	1909031 4420399 35500
4.	Sun Apr 06 23:39:20 +0000 2025	1032	<i>We the people stand with Isreal. Our Allies.</i>	1909030 0466791 42700
5.	Sun Apr 06 23:39:20 +0000 2025	690	<i>America must stand with Israel as it takes action against Iranian-backed terrorists.</i>	1909029 6351211 64500
6.	Sun Apr 06 23:39:20 +0000 2025	4	<i>Fact Just because you don't stand with Isreal or Forever wars don't make you antisemitic !!! It's time we end this notion that people are jew hating antisemitic because they refute Isreal. God bless America</i>	1909030 0769535 83900
7.	Sun Apr 06 23:39:20 +0000 2025	0	<i>And yet you arseholes stand with the murderous isreal</i>	1909030 9890173 46300
8.	Sun Apr 06 23:39:20 +0000 2025	0	<i>The UK must stand with Israel. Her enemies are ours. In a region of autocracy she shines as a rare democracy. We cannot abandon her. https://t.co/3ZngtSTj9L</i>	1909032 2200245 20700
...	...	...	...	...
1702	Sun Apr 06 23:39:20 +0000 2025	268	<i>The I Stand with Israel I support: 1. Israel has a right to exist</i>	1909031 1831212 60500

Tabel 3.1 merupakan potongan hasil dari *crawling tweet*, Di mana pada data tersebut menampilkan tanggal *tweet* diunggah pada kolom *created_at*. Kemudian jumlah *like* yang dimiliki pengunggah *tweet* tersebut terdapat pada kolom

favorite_count. Pada kolom *full_text* merupakan data teks *tweet* yang diunggah oleh pengguna. Dan terakhir pada kolom *id_str* merupakan data *id* pengguna.

Berikut potongan *source code* yang digunakan untuk melakukan crawling pada dataset penelitian ini.

```
import time
import random
import pandas as pd
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import TimeoutException,
NoSuchElementException

KEYWORDS = ["genocide", "israel", "gaza", "hamas", "palestine"]
try:
    # Navigate to explore page
    driver.get("https://x.com/explore")
    time.sleep(2)

    # Find and use search input
    search =
wait.until(EC.presence_of_element_located((By.XPATH,
'//input[@enterkeyhint="search" or @placeholder="Search" or
@aria-label="Search query"]'))))
    search.clear()
    query = f'{keyword} since:2023-10-07 until:2025-04-07
lang:en'
    search.send_keys(query)
    search.send_keys(Keys.RETURN)
    time.sleep(3)
```

3.4 Pelabelan Data

Setelah proses pengambilan data selanjutnya dilakukan pelabelan data. Seperti yang telah dijelaskan pada sub-bab 3.2 terkait objek penelitian ini merupakan analisis sentimen yang membutuhkan proses klasifikasi sentimen berdasarkan kategori sentimen tersebut. Oleh karena itu, pada tahap ini penulis melakukan pelabelan secara manual terhadap data *tweet* yang telah dikumpulkan sebelumnya.

Tabel 3.2 Hasil Label Data Manual

No	Created at	Full text	Label
1.	Sun Apr 06 23:39:20 +0000 2025	@sentdefender The whole world stands with Israel and the IDF. End the Islamic regime of Iran.	2
2.	Sun Apr 06 23:39:20 +0000 2025	I agree. Israel's war is with Hamas NOT the Palestinian people. Can never understand why the UK won't stand with Palestine. The state of Isreal didn't exist until 1948 when Britain thought it was a good idea to resettle them in Palestine.	0
3.	Sun Apr 06 23:39:20 +0000 2025	@EliAfriatISR I stand with Isreal.	2
4.	Sun Apr 06 23:39:20 +0000 2025	We the people stand with Isreal. Our Allies.	2
5.	Sun Apr 06 23:39:20 +0000 2025	America must stand with Israel as it takes action against Iranian-backed terrorists.	2
6.	Sun Apr 06 23:39:20 +0000 2025	Fact Just because you don't stand with Isreal or Forever wars don't make you antisemitic !!! It's time we end this notion that people are jew hating antisemitic because they refute Isreal. God bless America	1
7.	Sun Apr 06 23:39:20 +0000 2025	And yet you arseholes stand with the murderous isreal	0
8.	Sun Apr 06 23:39:20 +0000 2025	The UK must stand with Israel. Her enemies are ours. In a region of autocracy she shines as a rare democracy. We cannot abandon her. https://t.co/3ZngtSTj9L	2

No	Created_at	Full_text	Label
9.	Sun Apr 06 23:39:20 +0000 2025	@VividProwess You can't deal with terrorism they need wiping out. I stand with isreal https://t.co/jsBIST4wSQ	2
10.	Sun Apr 06 23:39:20 +0000 2025	The I Stand with Israel I support: 1. Israel has a right to exist	2

Tabel 3.2 menampilkan hasil pelabelan sentimen berdasarkan kategori sentimen yang telah ditetapkan. Pada tahap ini, pelabelan menggunakan sistem *encoding*, Di mana setiap kategori label dipresentasikan menggunakan angka. Untuk tahap pelabelan ini sentimen pada kategori pro-genosida menggunakan nilai 2, sedangkan untuk kategori sentimen netral menggunakan nilai 1, dan kategori sentimen kontra-genosida menggunakan nilai 0.

Dalam mengurangi label yang bersifat ambigu ataupun dinilai secara sepihak saja oleh penulis, maka terdapat proses pelabelan bersama *annotator* sebagai pihak ketiga dalam proses pelabelan data tersebut. Pada tahapan ini terdapat dua *annotator* yang ikut serta terlibat dalam proses pelabelan data *tweet*. Di mana label akhir pada data *tweet* ditentukan berdasarkan jumlah kategori terbanyak yang ditentukan oleh *annotator* dan penulis.

Tabel 3.3 Hasil Akhir Data Label

No	Created_at	Full_text	Penulis	Anno 1	Anno 2	Hasil Akhir
1.	Sun Apr 06 23:39:20 +0000 2025	@sentdefender The whole world stands with Israel and the IDF. End the Islamic regime of Iran.	2	1	2	2
2.	Sun Apr 06 23:39:20 +0000 2025	I agree. Israel's war is with Hamas NOT the Palestinian people. Can never understand why the UK won't stand with Palestine. The state of Isreal didn't exist until 1948	0	0	1	0

No	Created_at	Full_text	Penulis	Anno 1	Anno 2	Hasil Akhir
		<i>when Britain thought it was a good idea to resettle them in Palestine.</i>				
3.	Sun Apr 06 23:39:20 +0000 2025	<i>@EliAfriatISR I stand with Isreal.</i>	2	1	2	2
4.	Sun Apr 06 23:39:20 +0000 2025	<i>We the people stand with Isreal. Our Allies.</i>	2	1	2	2
5.	Sun Apr 06 23:39:20 +0000 2025	<i>America must stand with Israel as it takes action against Iranian-backed terrorists.</i>	2	2	2	2
6.	Sun Apr 06 23:39:20 +0000 2025	<i>Fact Just because you don't stand with Isreal or Forever wars don't make you antisemitic !!! It's time we end this notion that people are jew hating antisemitic because they refute Isreal. God bless America</i>	2	1	1	1
7.	Sun Apr 06 23:39:20 +0000 2025	<i>And yet you arseholes stand with the murderous isreal</i>	0	0	0	0
8.	Sun Apr 06 23:39:20 +0000 2025	<i>The UK must stand with Israel. Her enemies are ours. In a region of autocracy she shines as a rare democracy. We cannot abandon her. https://t.co/3ZngtSTj9L</i>	2	2	2	2
9.	Sun Apr 06 23:39:20 +0000 2025	<i>@VividProwess You can't deal with terrorism they need wiping out. I stand with isreal https://t.co/jsBIST4wSQ</i>	2	2	2	2
10.	Sun Apr 06 23:39:20 +0000 2025	<i>The I Stand with Israel I support: 1. Israel has a right to exist</i>	2	2	2	2

Tabel 3.3 merupakan hasil akhir dari pelabelan yang dilakukan bersama dua *annotator* dengan label terbanyak menjadi hasil akhir label pada data *tweet* tersebut.

3.5 Data Balancing

Setelah melakukan pelabelan pada data *tweet* selanjutnya merupakan proses memastikan data seimbang dengan cara menghitung jumlah data tweet berdasarkan label untuk menentukan apakah data sudah memenuhi jumlah yang dibutuhkan pada analisis sentimen.

Tabel 3.4 Jumlah Dataset Berdasarkan Kategori Label

Kategori Sentimen	Kode Sentimen	Jumlah
Pro-genosida	2	180
Netral	1	1408
Kontra-genosida	0	405

Tabel 3.4 menunjukkan bahwa data berdasarkan label belum seimbang, dapat dilihat pada label kategori pro-genosida yang hanya terdapat data sebanyak 180 sedangkan label pada kategori netral mencapai 1408. Dengan jumlah tersebut data menjadi *imbalanced*, Di mana data tidak seimbang sehingga dapat mempengaruhi program analisis sentimen ketika melakukan *training*. Model akan mengalami *biased learning* dikarenakan kurangnya preferensi pada salah satu label, selain itu model dapat mengalami *misleading accuracy* Di mana model tidak dapat memprediksi berdasarkan label yang sesuai.

Oleh karena itu, dilakukan proses *data balancing* di mana penulis melakukan *oversampling* pada data *tweet* untuk kategori label pro-genosida dan kontra-genosida. *Oversampling* dilakukan dengan mengumpulkan data *tweet* yang terdapat pada label pro-genosida dan kontra-genosida. Sehingga hasil akhir dataset yang digunakan pada program menjadi seimbang. Kemudian dilakukan

undersampling pada kategori sentimen netral, yaitu mengurangi dataset pada kelas tersebut agar menyesuaikan dengan jumlah kelas sentimen pro dan kontra-genosida.

Tabel 3.5 Jumlah Data Berdasarkan Kategori Label setelah *Oversampling*

Kategori Sentimen	Kode Sentimen	Jumlah
Pro-genosida	2	490
Netral	1	492
Kontra-genosida	0	490

Tabel 3.5 merupakan hasil dari *oversampling* dan *undersampling* data, pada tabel tersebut menampilkan jumlah label pro-genosida dan kontra-genosida bertambah secara signifikan sehingga data menjadi seimbang dan dapat digunakan untuk tahapan penelitian selanjutnya.

3.6 Data Preprocessing

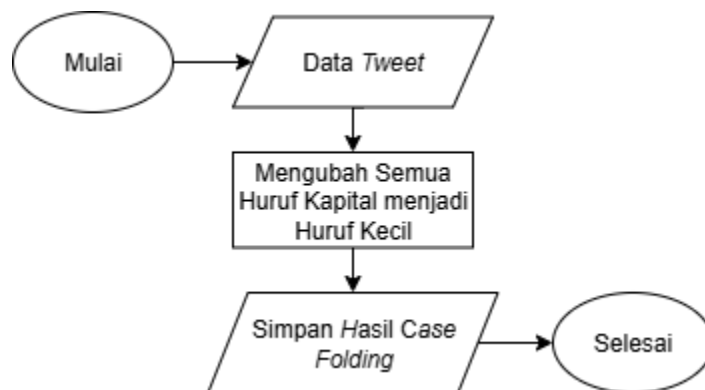
Data Preprocessing merupakan tahap di mana data dilakukan pembersihan dari elemen-elemen yang tidak dibutuhkan pada data dalam memproses data dalam model *machine learning*. Pada tahapan ini terdapat beberapa elemen yang akan dihapus maupun diubah diantaranya adalah mengubah huruf besar menjadi kecil semua (*case folding*), menghapus *link*, simbol, tanda baca dan emotikon, kemudian dilanjut dengan menghapus *stopword* dan melakukan lematisasi pada data tersebut. Berikut adalah penjelasan *data prerocessing* pada setiap tahapan.

3.6.1 Case Folding

Tahap pertama adalah *case folding*, yaitu proses mengubah seluruh huruf kapital pada dataset menjadi huruf kecil (*lowercase*). *Case folding* digunakan untuk mengurangi redundansi pada data yang dapat memengaruhi proses

pengklasifikasian. Dikarenakan setiap karakter memiliki nilai unik yang berbeda, sehingga huruf “A” dan “a” akan dianggap sebagai dua karakter yang berbeda.

Pada tahap ini, proses *case folding* dilakukan dengan menggunakan *function lower()* yang terdapat dalam bahasa pemrograman Python. Alur dari proses *case folding* ditunjukkan pada Gambar 3.5.



Gambar 3.5 Flowchart Proses Case Folding

Tabel 3.6 Data Hasil Tahap Case Folding

Data Mentah	Hasil Casefolding
@sentdefender The whole world stands with Israel and the IDF. End the Islamic regime of Iran.	@sentdefender the whole world stands with israel and the idf. end the islamic regime of iran.

3.6.2 Noise Removal

Tahap selanjutnya adalah *noise removal*, yaitu proses pembersihan data *tweet* dari karakter-karakter yang tidak diperlukan dalam proses klasifikasi dan analisis. Karakter yang termasuk *noise* antara lain adalah tanda baca, alamat situs atau *link* URL, angka, spasi ganda, dan karakter selain huruf alfabet. *Noise removal* digunakan untuk mengurangi *noise* sehingga data menjadi lebih bersih dan fokus pada informasi yang lebih relevan.

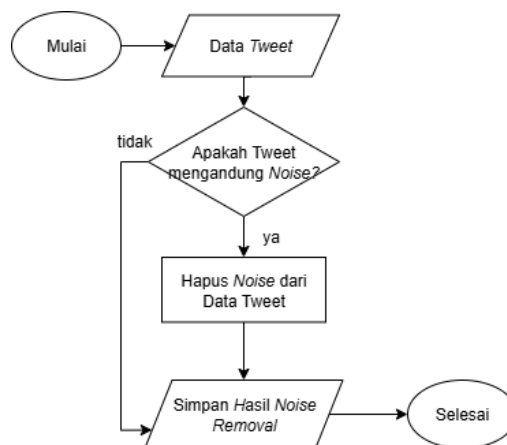
Pada penelitian ini, proses *noise removal* dilakukan dengan memanfaatkan *regular expression* untuk mendeteksi dan menghapus karakter yang dianggap

sebagai *noise*. Bentuk *regular expression* yang digunakan dapat dilihat pada Tabel 3.7.

Tabel 3.7 *Regular Expression*

No	Regex	Fungsi
1	@[\s]+	Menghapus username
2	(#[A-Za-z0-9]+)	Menghapus tanda #
3	r"[^a-zA-Z0-9]"	Mengganti setiap karakter yang bukan angka, atau karakter dalam rentang 'a ke z' atau 'A ke Z' dengan string kosong
4	r'((www\.[^\s]+) (https?://[^\s]+))'	Menghapus link situs
5	(\w*\d\w*)	Menghapus sebuah kalimat bersama angka dalam satu kata
6	r'\d+'	Menghapus angka
7	r'\s{2,}'	Menghapus spasi
8	r'\n'	Menghapus enter

Adapun alur proses *noise removal* dapat dilihat pada gambar 3.6, sedangkan Tabel 3.8 merupakan contoh hasil data *tweet* setelah melalui tahap *noise removal*.



Gambar 3.6 *Flowchart Proses Noise Removal*

Tabel 3.8 Data Hasil Tahap *Noise Removal*

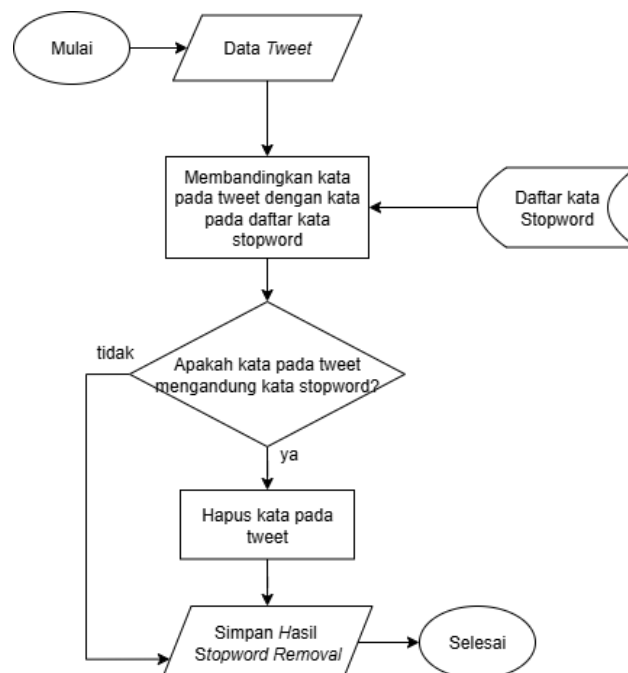
Data Casefolding	Hasil Noise Removal
@sentdefender the whole world stands with israel and the idf. end the islamic regime of iran.	the whole world stands with israel and the idf end the islamic regime of iran

3.6.3 Stopword Removal

Tahap *stopword removal* merupakan tahapan selanjutnya setelah *noise removal*. Di mana pada tahapan sebelumnya menghapus karakter yang tidak dibutuhkan dalam proses analisis, sedangkan pada tahap ini menghapuskan kata-kata yang tidak dibutuhkan dan tidak memiliki makna dalam proses analisis.

Pada penelitian ini, daftar stopwords yang digunakan berasal dari kamus stopwords bahasa Inggris yang umum dipakai dalam penelitian *Natural Language Processing (NLP)*. Sumber utama stopwords tersebut diadaptasi dari *NLTK Stopwords Corpus* (Bird et al, 2009). Daftar lengkap stopwords bahasa Inggris yang digunakan dapat dilihat pada Lampiran 1.

Proses *stopword removal* dapat dilihat pada Gambar 3.7, sedangkan Tabel 3.9 memperlihatkan contoh hasil data *tweet* setelah dilakukan *stopword removal*.



Gambar 3.7 Flowchart Proses Stopword Removal

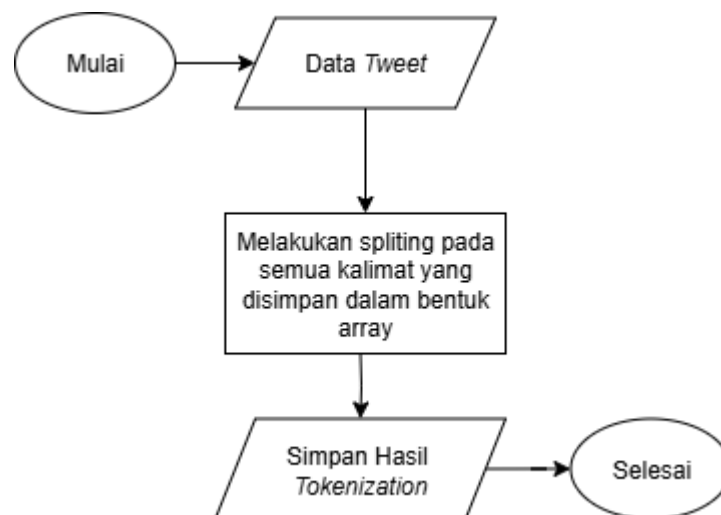
Tabel 3.9 Data Hasil Tahap *Stopword Removal*

Data <i>Noise Removal</i>	Hasil <i>Stopword Removal</i>
<i>the whole world stands with israel and the idf end the islamic regime of iran</i>	<i>whole world stands israel idf end islamic regime iran</i>

3.6.4 Tokenization

Tahap selanjutnya adalah *tokenization*, yaitu proses memecah teks menjadi unit kecil yang disebut token berupa kata atau frasa. Proses ini dilakukan agar teks yang awalnya dalam bentuk kalimat dipecah menjadi per-kata sehingga data menjadi lebih terstruktur saat diolah oleh model. Dengan melakukan tokenisasi, model dapat menganalisis setiap kata secara terpisah.

Pada penelitian ini, tokenisasi dilakukan dengan menggunakan *library NLTK* dengan *function word_tokenize*. Berikut Gambar 3.9 menunjukkan *flowchart* dari proses tokenisasi, dan para Tabel 3.10 memperlihatkan contoh hasil data *tweet* yang telah dilakukan tokenisasi.

Gambar 3.8 *Flowchart* Proses *Tokenization*Tabel 3.10 Data Hasil Tahap *Tokenization*

Data Stopword Removal	Hasil Tokenization
<i>whole world stands israel idf end islamic regime iran</i>	<i>['whole', 'world', 'stands', 'israel', 'idf', 'end', 'islamic', 'regime', 'iran']</i>

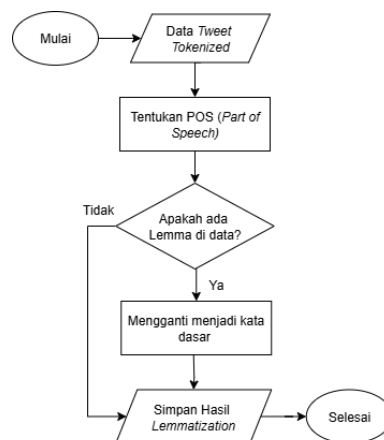
3.6.5 Lemmatization

Tahap terakhir dalam proses *text preprocessing* adalah *lemmatization*, yaitu proses mengubah kata pada data menjadi kata dasar. *lemmatization* menghasilkan kata dasar yang valid dengan cepat. Dengan demikian, variasi kata yang berbeda tetapi memiliki makna yang sama dapat dijadikan satu makna sehingga mempermudah proses analisis oleh model.

Pada penelitian ini, *lemmatization* dilakukan menggunakan *library WordNetLemmatizer* dari NLTK. Di mana *function* tersebut bekerja untuk mengubah kata pada data menjadi kata dasar yang sesuai dengan konteks. Gambar 3.8 menunjukkan flowchat dari proses *lemmatization*, sedangkan Tabel 3.10 memperlihatkan contoh hasil dari data *tweet* yang telah dilakukan *lemmatization*.

Tabel 3.11 Data Hasil Tahap *Lemmatization*

Data Tokenized	Hasil Lemmatization
<i>['whole', 'world', 'stands', 'israel', 'idf', 'end', 'islamic', 'regime', 'iran']</i>	<i>['whole', 'world', 'stand', 'israel', 'idf', 'end', 'islamic', 'regime', 'iran']</i>



Gambar 3.9 Flowchart Proses *Lemmatization*

3.7 Data Training, Validation dan Prediction

Pada tahap ini, data yang telah melalui tahapan *data balancing* dan *data preprocessing* maka selanjutnya data dibagi menjadi tiga bagian utama, yaitu *training*, *validation* dan *prediction*. Di mana data pada bagian *training* digunakan untuk melatih model dalam mempelajari pola dari data teks tersebut. Sedangkan data pada bagian *validation* digunakan untuk mengevaluasi kinerja model selama proses *training* model, data *validation* dapat mengurangi terjadinya *overfitting* pada model. Setelah model dilatih, maka tahap berikutnya adalah menggunakan data *prediction* dalam menguji model menggunakan data baru untuk mendapatkan hasil klasifikasi sentimen.

Dalam penelitian ini, dataset dibagi menjadi tiga bagian dengan rasio 70:20:10. Di mana data *training* mencakup 70% dari dataset, data *validation* 20% dari dataset, dan data *prediction* adalah 10% dari dataset. Pembagian ini dilakukan agar model mendapatkan data *training* lebih banyak untuk proses melatih model, dan tetap menyediakan data untuk menguji model tersebut dengan data *validation* dan *prediction*.

Selain menggunakan pembagian rasio, penelitian ini juga menerapkan metode *k-fold cross validation* untuk meningkatkan reliabilitas hasil evaluasi model. *k-fold cross validation* merupakan sebuah teknik membagi dataset ke dalam beberapa bagian yang memiliki ukuran yang sama. Di mana pada setiap iterasi, satu *fold* digunakan sebagai data *training* kemudian *fold* lainnya digunakan sebagai data *validation*. Proses ini diulang sebanyak *k* kali sehingga setiap *fold* berperan sebagai data *validation* satu kali. Kemudian hasil evaluasi dari setiap iterasi dirata-ratakan

untuk mendapatkan hasil akhir dari evaluasi model tersebut. Dengan metode ini, proses latihan model akan melalui berbagai skenario dengan pembagian data yang berbeda, sehingga dapat memberikan performa yang lebih akurat dan mengurangi bias dalam penilaian model.

3.8 Ekstraksi Fitur

Tahap ini merupakan proses mengubah data teks menjadi nilai numerik sehingga data dapat diproses oleh model *machine learning* maupun *deep learning*. Pada tahap ini, penulis menggunakan dua teknik ekstraksi, yaitu *Term Frequency-Inverse Document Frequency* dan *Word Embedding*.

3.8.1 *Term Frequency-Inverse Document Frequency*

Term Frequency-Inverse Document Frequency atau yang biasa disebut TF-IDF merupakan proses mengubah data teks menjadi nilai numerik berdasarkan frekuensi munculnya kata tersebut dalam dataset. Proses ini biasa digunakan pada model *machine learning* seperti model Naive Bayes dan SVM karena dapat memisahkan kata yang tergolong penting berdasarkan frekuensi.

TF-IDF dibangun menggunakan dua komponen yaitu *Term Frequency* dan *Inverse Document Frequency* yang kemudian dikalikan untuk menghasilkan bobot akhir dari setiap kata.

1. *Term Frequency* (TF)

Term Frequency berfungsi untuk menghitung besarnya frekuensi munculnya salah satu kata dalam sebuah dataset. Dengan cara menjumlahkan kemunculan kata target dan dibagi dengan total kata yang terdapat dalam dataset. Berikut persamaan 3.1 merupakan rumus menghitung frekuensi kata.

$$TF(t, d) = \frac{f_{t,d}}{\sum_k f_{k,d}} \quad (3.1)$$

Pada persamaan di atas, $TF(t, d)$ merupakan frekuensi kata target, kemudian $f_{t,d}$ merupakan total kata target dalam dataset, dan terakhir $\sum_k f_{k,d}$ merupakan jumlah kata yang dimiliki oleh dataset tersebut.

2. *Inverse Document Frequency (IDF)*

Setelah mendapatkan frekuensi pada setiap kata, maka selanjutnya adalah menghitung seberapa pentingnya suatu kata dalam dataaset. Misalkan apabila ada kata yang sering muncul dalam dataset seperti kata “yang”, “dan” dan “di” maka nilai IDF akan cenderung rendah, sedangkan kata yang jarang muncul memiliki nilai IDF yang tinggi. Persamaan 3.2 merupakan komputasi untuk menentukan bobot IDF.

$$IDF(t) = \log \left(\frac{N}{df_t} \right) \quad (3.2)$$

Di mana nilai N merupakan jumlah seluruh dokumen pada dataset dan df_t merupakan jumlah dokumen yang mengandung kata target atau (t).

3. Bobot TF-IDF

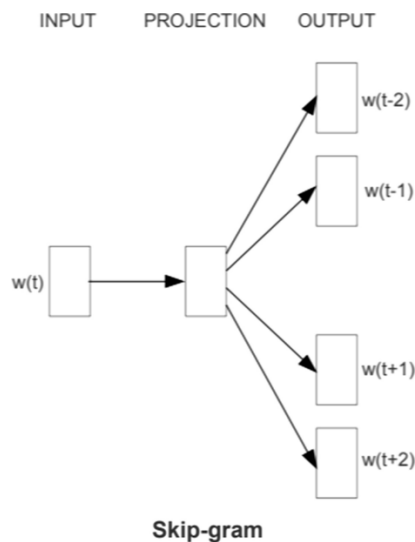
Tahap terakhir adalah mengalikan hasil dari TF dan IDF yang kemudian akan menjadi bobot TF-IDF pada kata tersebut. Sehingga setiap kata memiliki bobot yang berbeda tergantung dengan seberapa tinggi frekuensi kemunculannya dalam keseluruhan kata pada dataset dan seberapa sering kata tersebut muncul pada setiap dokumen pada dataset. Persamaan 3.3 merupakan kalkulasi akhir untuk menghasilkan bobot TF-IDF.

$$TF - IDF(t, d) = TF(t, d) \times IDF(t) \quad (3.3)$$

3.8.2 Word Embedding

Word Embedding adalah teknik dalam *Natural Language Preprocessing* (NLP) yang berfungsi untuk merepresentasikan kata dalam bentuk ruang vektor dengan nilai riil. Sehingga setiap kata memiliki vektor yang mencerminkan maknanya.

Pada penelitian ini, proses word embedding yang digunakan adalah model Word2Vec. Dan model tersebut dibangun menggunakan *library gensim* dengan bahasa pemrograman Python. Pada proses pelatihan Word2Vec dilakukan dengan menggunakan pendekatan *Skip-Gram*, yaitu salah satu algoritma Word2Vec yang digunakan untuk memperkirakan kata konteks berdasarkan kata target. *Skip-Gram* bekerja dengan menggunakan posisi kata yang ada sebelum dan sesudah target. Gambar 3.10 menampilkan ilustrasi *Skip-Gram*. Adapun tahapan pembentukan model Word2Vec dengan contoh data sederhana dijelaskan sebagai berikut.



Gambar 3.10 Arsitektur *Skip-Gram* (Mikolov et al, 2013)

1. Menyiapkan Data

Untuk memberikan gambaran perhitungan manual, digunakan sebuah korpus sederhana yang terdiri dari enam kata, yaitu “*people demand justice peace for palestine*”. Pada perhitungan ini, ditetapkan *hyperparameter* berupa ukuran *window* sebesar 2 serta dimensi vektor sebanyak 4. Kata konteks didefinisikan sebagai kata yang berada di sekitar kata target. Dengan *windows=2*, maka dua kata pada sisi kiri dan dua kata pada sisi kanan target akan dianggap sebagai kata konteks. Hasil *sliding window* dapat dilihat pada Tabel 3.12.

Tabel 3.12 *Sliding Window*

1	<i>people</i>	<i>demand</i>	<i>justice</i>	<i>peace</i>	<i>for</i>	<i>palestine</i>
	Xk	wc=1	wc=2			
2	<i>people</i>	<i>demand</i>	<i>justice</i>	<i>peace</i>	<i>for</i>	<i>palestine</i>
	wc=1	Xk	wc=1	wc=2		
3	<i>people</i>	<i>demand</i>	<i>justice</i>	<i>Peace</i>	<i>for</i>	<i>palestine</i>
	wc=1	wc=2	Xk	wc=1	wc=2	
4	<i>people</i>	<i>demand</i>	<i>justice</i>	<i>peace</i>	<i>For</i>	<i>palestine</i>
		wc=1	wc=2	Xk	wc=1	wc=2
5	<i>people</i>	<i>demand</i>	<i>justice</i>	<i>peace</i>	<i>for</i>	<i>palestine</i>
			wc=1	wc=2	Xk	wc=1
6	<i>people</i>	<i>demand</i>	<i>justice</i>	<i>peace</i>	<i>for</i>	<i>palestine</i>
				wc=1	wc=2	Xk

Tabel 3.13 *One Hot Encoding*

#token	Xk	wc=1	wc=2	One-Hot Encoding
0	<i>people</i>	<i>demand</i>	<i>justice</i>	[100000]
1	<i>demand</i>	<i>people,justice</i>	<i>peace</i>	[010000]
2	<i>justice</i>	<i>people, peace</i>	<i>demand,,for</i>	[001000]
3	<i>peace</i>	<i>demand,for</i>	<i>justice, palestine</i>	[000100]
4	<i>for</i>	<i>justice,palestine</i>	<i>peace</i>	[000010]
5	<i>palestine</i>	<i>peace</i>	<i>for</i>	[000001]

Pada Tabel 3.13 menunjukkan hasil dari proses *One Hot Encoding*, yang sebelumnya dilakukan *sliding window* dengan jumlah window sebanyak 2 buah.

<i>Hidden Layer</i>
-0,84
-0,44
-0,91
0,88
0,30
0,33
-0,28

b. Menentukan *output layer*

Output Layer merupakan hasil dari perkalian *hidden layer* dengan pembobotan W_2 , perkalian matriks dengan dimensi 1×10 dengan 10×6 menghasilkan matriks dengan dimensi 1×6 . Berikut Tabel 3.15 merupakan hasil perhitungan dari *output layer*.

Tabel 3.15 *Output Layer Word2Vec*

<i>Hidden Layer</i>		Bobot 2 (W_2)							<i>Output Layer</i>
0,52	X	-0,87	-0,41	-0,29	-0,02	-0,56	0,18	=	-2,71
0,44		-0,40	0,89	0,69	-0,33	0,22	-0,85		0,50
0,09		-0,13	0,69	-0,83	0,71	-0,42	0,06		0,23
-0,84		0,88	0,24	0,02	0,62	0,94	-0,44		-0,85
-0,44		-0,48	0,24	0,82	-0,73	0,26	-0,99		-0,75
-0,91		0,68	0,72	-0,11	0,08	-0,74	0,23		0,68
0,88		-0,69	0,91	0,46	-0,02	-0,01	0,00		
0,30		-0,05	0,40	-0,02	-0,56	-0,55	0,47		
0,33		-0,84	0,05	-0,16	-0,16	-0,67	0,14		
-0,28		0,10	-1,00	-0,31	0,88	-0,40	-0,63		

c. Menghitung *Softmax*

Setelah mendapatkan hasil *output layer* selanjutnya adalah menghitung fungsi *softmax*. Dalam model word2vec *softmax* berfungsi untuk mengubah skor *output* (logit) menjadi nilai probabilitas. Di mana *softmax* dapat menentukan nilai probabilitas pada kemunculan konteks berdasarkan kata target.

Rumus menghitung fungsi *softmax* dapat dilihat pada persamaan 3.4. Di mana $P(w_c | w_t)$ merupakan probabilitas kemunculan kata konteks w_c terhadap kata target w_t . Sedangkan y_{wc} merupakan nilai output dari kata konteks. Dan V merupakan jumlah total kosakata.

$$P(w_c | w_t) = \frac{e^{y_{wc}}}{\sum_{i=1}^V e^{y_i}} \quad (3.4)$$

d. Mencari *Error* dan *SUM of Different*

Tahapan selanjutnya adalah menghitung *error*, yaitu selisih antara nilai target *one-hot vektor* dengan nilai prediksi hasil *softmax*. Tahapan ini dilakukan untuk mengetahui seberapa jauh hasil prediksi model dengan nilai sebenarnya. Perhitungan mencari *error* dapat dilihat pada persamaan 3.5.

$$E = \hat{y} - y \quad (3.5)$$

Di mana $\hat{y}$ merupakan hasil dari *softmax* kata target dan y adalah nilai vektor target yang aktual. Setelah mendapatkan nilai *error* maka selanjutnya adalah menghitung *Sum of Difference* yaitu penjumlahan keseluruhan error pada setiap kata agar diketahui perbaikan bobot pada proses *backpropagation*.

e. *Backpropagation*

Pada tahap ini, bobot pada kata target diperbarui berdasarkan nilai *error* yang telah didapatkan sebelumnya. Proses *backpropagation* melibatkan perhitungan gradien setiap bobot sehingga model dapat menyesuaikan parameter dan meminimalisir *loss function*. Perhitungan *backpropagation* dapat dilihat pada persamaan 3.6. Di mana nilai h^T merupakan vektor dari *hidden layer* sedangkan E merupakan nilai *error* dari *output layer*.

$$\frac{\delta E}{\delta W} = h^T - E \quad (3.6)$$

f. *Update Bobot*

Tahapan terakhir adalah *update* bobot kata setelah melalui semua proses sebelumnya. *Update* bobot didapatkan melalui mengurangi nilai gradien yang telah dikalikan dengan *learning rate* (α). Perhitungan *update* bobot dapat dilakukan dengan persamaan 3.7.

$$W_{baru} = W_{lama} - \alpha \times \frac{\delta E}{\delta W} \quad (3.7)$$

Dengan demikian, hasil vektor dari proses *word embedding* dapat digunakan dalam proses *training* model LSTM dalam menganalisis dan mengklasifikasikan kelas sentimen.

3.9 Pembangunan Model

3.9.1 Naive Bayes

Model Naive Bayes merupakan salah satu algoritma *machine learning* yang dapat mengklasifikasikan berdasarkan probabilistik. Model ini menganggap setiap kata itu bersifat independen sehingga perhitungan probabilitas dapat dijalankan secara efisien. Namun sifat independen ini juga yang membuat model kurang efisien dalam mengklasifikasikan data baru.

Naive Bayes dapat memberikan performa yang baik apabila diterapkan pada dataset dengan dimensi tinggi seperti TF-IDF. Di mana proses klasifikasi pada model Naive Bayes berdasar pada Teorema Bayes yang menghitung probabilitas suatu kelas berdasarkan prior kelas tersebut. Persamaan 3.8 merupakan komputasi probabilitas pada algoritma Naive Bayes.

$$P(y|x) = \frac{P(x|y) \times P(y)}{P(x)} \quad (3.8)$$

Di mana $P(y|x)$ dihitung berdasarkan frekuensi kemunculan kata pada dokumen dengan kelas tertentu. Setelah mendapatkan probabilitas, selanjutnya model memilih kelas berdasarkan hasil probabilitas tertinggi sebagai hasil prediksi.

Pada model Naive Bayes terdapat beberapa variasi algoritma seperti Multinomial Naive Bayes yang biasa digunakan untuk klasifikasi multi kelas, kemudian Bernouli Naive Bayes untuk kelas biner atau untuk mengklasifikasikan data yang hanya memiliki dua kelas. Dan terakhir ada Gaussian Naive Bayes yang digunakan untuk data kontinu atau dataset yang tidak terbatas berdasarkan asumsi distribusi *gaussian*.

Pada penelitian ini, algoritma Naive Bayes paling efisien untuk digunakan pada klasifikasi sentimen adalah Multinomial Naive Bayes, dikarenakan dataset yang dimiliki hanya berisi teks dan kelas sentimen yang digunakan adalah sebanyak tiga kelas. Selanjutnya model tersebut dilakukan fine-tuning menggunakan beberapa komponen untuk mendapatkan model yang paling optimal. Tabel 3.16 merupakan komponen yang digunakan dalam melakukan fine-tuning pada model Naive Bayes.

Tabel 3.16 Komponen Tuning Hyperparameter Model Naive Bayes

No	Parameter	Deskripsi	Nilai
1	<i>Alpha(a)</i>	Nilai smoothing Laplace untuk mengatasi zero-frequency	0.1, 0.5, 1.0
2	<i>Fit Prior</i>	Mengatur apakah prior kelas dihitung otomatis	True, False
3	<i>Norm TF-IDF</i>	Normalisasi sebelum masuk model	L1, L2

3.9.2 Support Vector Machine

Model SVM merupakan salah satu algoritma *machine learning* yang dapat mengklasifikasikan data dengan cara memisahkan data antar kelas menggunakan *hyperplane*. Ekstraksi fitur yang paling cocok dengan SVM adalah TD-IDF dikarenakan dengan dataset dengan dimensi tinggi model SVM dapat lebih mudah mencari margin maksimal antar kelas. Di mana semakin besar margin antar kelas, semakin baik kemampuan model dalam memproses dan mengklasifikasikan data baru.

$$w \cdot x + b = 0 \quad (3.9)$$

Persamaan 3.9 merupakan komputasi untuk menemukan *hyperplane*. Di mana w adalah bobot, b adalah *bias*, dan x adalah vektor fitur. Untuk menentukan jarak atau margin antar kelas dapat dihitung menggunakan perbedaan antara *support vector* terdekat dengan *hyperplane*. Dalam mendapatkan margin yang optimal dapat dilakukan dengan komputasi menggunakan fungsi *loss hinge*, dimana fungsi *long hinge* untuk menghindari terjadinya prediksi yang bias. Persamaan 3.10 merupakan komputasi untuk optimalisasi margin.

$$L = \max (0, 1 - y_i(w \cdot x_i + b)) \quad (3.10)$$

Selanjutnya model tersebut dilakukan fine-tuning menggunakan beberapa komponen untuk mendapatkan model yang paling optimal dalam mengklasifikasi sentiment berdasarkan tiga kelas sentimen. Tabel 3.17 merupakan komponen yang digunakan dalam melakukan fine-tuning pada model SVM.

Tabel 3.17 Komponen Tuning Hyperparameter Model SVM

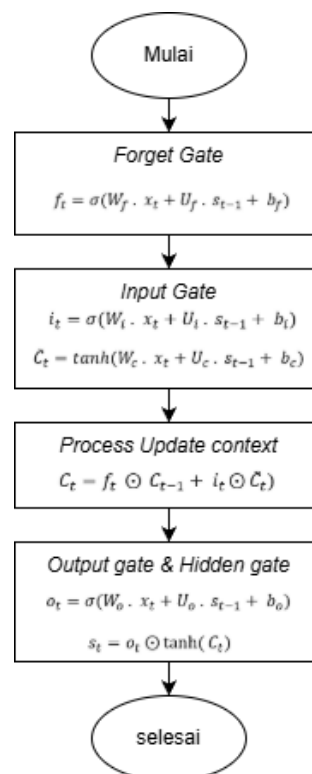
No	Parameter	Deskripsi	Nilai
1	<i>Kernel</i>	Fungsi pemisah ruang fitur	linear, RBF

2	C	Tugkat regularisasi	0.1, 1, 10, 100
3	Γ	Mengatur pengaruh data latih pada proses klasifikasi	scale, auto

3.9.3 Long Short-Term Memeory

Lapisan LSTM adalah unit dasar pada RNN yang dirancang untuk mengatasi permasalahan *vanishing gradient* dan *exploding gredient* pada jaringan RNN. LSTM memiliki kemampuan untuk menyimpan informasi dalam jangka waktu yang lebih panjang melalui mekanisme *cell state* dan beberapa *gates* yang mengatur alurnya informasi.

Proses perhitungan pada LSTM melibatkan empat komponen utama, yaitu *forget gate*, *input gate*, *cell state update*, dan *output gate*. Gambar 3.11 merupakan *flowchart* dari alur model LSTM.



Gambar 3.11 *Flowchat* Model LSTM

1. *Forget Gate*

Tahapan ini bertugas untuk menentukan informasi pada *cell state* sebelumnya (C_{t-1}) yang akan disimpan atau dilupakan. Proses komputasi dilakukan menggunakan fungsi *sigmoid* dengan rentang nilai antara 0 dan 1. Di mana nilai 0 menandakan informasi dilupakan, sedangkan nilai 1 menandakan informasi disimpan. Komputasi *forget gate* dapat dilihat pada Persamaan 3.11.

$$f_t = \sigma(W_f \cdot x_t + U_f \cdot s_{t-1} + b_f) \quad (3.11)$$

Di mana x_t adalah input pada waktu ke- t , kemudian s_{t-1} adalah *hidden state* sebelumnya, dan W_f, U_f, b_f masing-masing merupakan bobot dan bias pada *forget gate*.

2. *Input Gate dan Candidate Cell State*

Input gate berfungsi untuk menentukan informasi baru yang akan ditambahkan ke dalam *cell state*. Proses ini melibatkan dua langkah, yaitu aktivasi *sigmoid* untuk menentukan bagian informasi baru yang akan diperbarui (i_t), dan fungsi *tanh* untuk menghasilkan kandidat nilai baru pada *cell state* ($\tilde{C}_t$). *Input gate* dapat dilihat pada Persamaan 3.12 dan 3.13.

$$i_t = \sigma(W_i \cdot x_t + U_i \cdot s_{t-1} + b_i) \quad (3.12)$$

$$\tilde{C}_t = \tanh(W_c \cdot x_t + U_c \cdot s_{t-1} + b_c) \quad (3.13)$$

Kedua hasil ini kemudian digabung untuk memperbarui *cell state*.

3. *Update Cell State*

Cell state diperbarui dengan menggabungkan informasi lama yang telah disimpan dan informasi baru dari *input gate*. Persamaan 3.14 adalah komputasi untuk memperbarui *cell state*.

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (3.14)$$

Di mana simbol $\odot$ menunjukkan perkalian elemen per elemen (*Hadamard product*). Nilai C_t ini akan digunakan untuk menentukan *hidden state* berikutnya.

4. *Output Gate* dan *Hidden State*

Tahapan akhir adalah menentukan *output gate* untuk menghasilkan *hidden state* baru. *Output gate* menggunakan fungsi sigmoid untuk memfilter informasi yang akan menjadi output, sedangkan *hidden state* dihitung dengan mengalikan hasil *output gate* dengan fungsi tanh dari *cell state*. Kompurasi *output layer* dapat dilihat pada persamaan 3.15 dan 3.16.

$$o_t = \sigma(W_o \cdot x_t + U_o \cdot s_{t-1} + b_o) \quad (3.15)$$

$$s_t = o_t \odot \tanh(C_t) \quad (3.16)$$

Nilai s_t inilah yang menjadi *hidden state* pada waktu ke- t dan juga menjadi masukan bagi langkah berikutnya.

Setelah proses *forward pass* selesai dilakukan, jaringan akan memasuki tahap *backward pass* untuk memperbarui bobot dan bias melalui algoritma *backpropagation through time (BPTT)*. Kesalahan dihitung menggunakan fungsi *loss*, seperti *Mean Squared Error (MSE)* dapat dilihat pada persamaan 3.17.

$$E(x, \hat{x}) = \frac{(x - \hat{x})^2}{2} \quad (3.17)$$

Nilai gradien dari setiap gerbang kemudian dihitung secara bertahap dari *output gate* hingga *forget gate*. Setiap gradien yang dihasilkan akan digunakan untuk memperbarui bobot dengan metode *Stochastic Gradient Descent (SGD)* yang dapat dilihat pada Persamaan 3.18

$$W_{baru} = W_{lama} - \eta \cdot \Delta W \quad (3.18)$$

dengan η adalah *learning rate*. Melalui proses *forward* dan *backward pass* tersebut, jaringan LSTM dapat menyesuaikan bobot untuk meminimalkan kesalahan prediksi secara bertahap pada setiap *epoch* pelatihan.

Pada penelitian ini, dilakukan proses pembuatan, *tuning*, dan pelatihan model LSTM untuk menghasilkan model terbaik dalam melakukan prediksi atau klasifikasi data teks. Proses ini dimulai dari penentuan parameter model, pembentukan arsitektur LSTM, hingga pelatihan menggunakan data latih yang telah melalui tahap preprocessing.

Model LSTM dibangun dengan menggunakan beberapa lapisan utama, yaitu *embedding layer*, *LSTM layer*, *dropout layer*, dan *dense layer*. Selebihnya dapat dilihat pada Tabel 3.18

Tabel 3.18 Komponen Tuning *Hyperparameter* Model LSTM

No	Parameter	Deskripsi	Nilai
1	Jumlah <i>Neuron</i>	Banyaknya unit dalam lapisan LSTM yang menentukan kapasitas pembelajaran model	64, 128, 256
2	<i>Learning Rate</i>	Kecepatan pembaruan bobot selama proses training	0.001, 0.0005, 0.01
3	<i>Batch Size</i>	Jumlah data yang diproses sebelum pembaruan bobot	32, 64
4	Jumlah <i>Epoch</i>	Banyaknya iterasi penuh terhadap seluruh dataset	10, 20, 30, 40, 50
5	<i>Dropout Rate</i>	Persentase neuron yang dinonaktifkan sementara untuk mencegah overfitting	0,2 – 0,5

3.9.4 Bidirectional Long Short-Term Memory

Bidirectional Long Short-Term Memory (BiLSTM) merupakan model yang dikembangkan dari arsitektur LSTM, model ini dapat memproses seperti LSTM

namun ditambah dengan memproses data secara dua arah sekaligus. Yaitu dari kiri ke kanan (forward) dan kiri ke kanan (backward). Dengan adanya proses dua arah BiLSTM dapat menangkap konteks lebih dalam dan menyeluruh dibandingkan LSTM.

BiLSTM memiliki struktur dasar yang sama dengan LSTM, tetapi terdiri dari dua buah lapisan LSTM yang bekerja secara paralel, yaitu LSTM *forward* yang berfungsi untuk membaca urutan dataset dari kiri ke kanan, dan LSTM *backward* yang berfungsi untuk membaca dataset dari kanan ke kiri. Kedua arah tersebut menghasilkan dua buah *hidden state* yang berbeda, kemudian digabungkan untuk membentuk representasi yang lebih baik terhadap konteks kata. Proses komputasi lapisan LSTM *forward* dan *backward* dapat dilihat pada persamaan 3.19 dan 3.20.

$$h_t = LSTM_{forward}(x_t) \quad (3.19)$$

$$h_t = LSTM_{backward}(x_t) \quad (3.20)$$

Untuk membangun model BiLSTM yang optimal dalam mengklasifikasikan sentimen, diperlukan beberapa tahapan fine-tuning pada model. Pada penelitian ini, fine-tuning model BiLSTM menggunakan beberapa komponen dari hyperparameter fine-tuning LSTM, namun ditambah beberapa komponen untuk mengoptimalkan model tersebut. Berikut Tabel 3.19 menunjukkan komponen yang digunakan dalam proses fine-tuning model BiLSTM.

Tabel 3.19 Komponen Tuning Hyperparameter Model BiLSTM

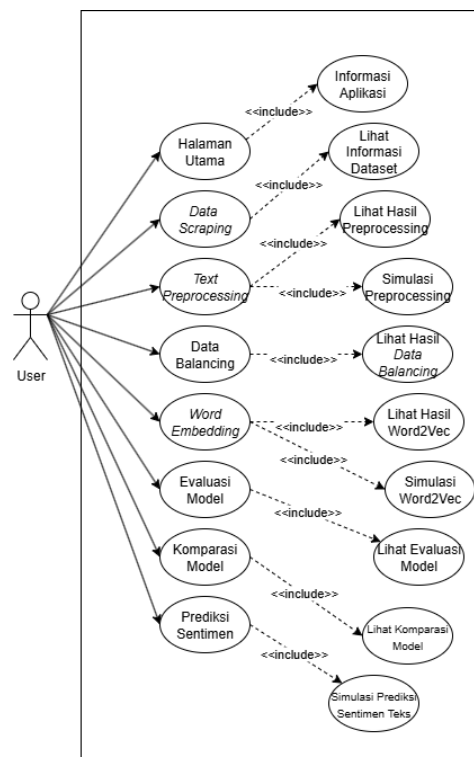
No	Parameter	Deskripsi	Nilai
1	Jumlah <i>Neuron</i>	Banyaknya unit dalam lapisan LSTM yang menentukan kapasitas pembelajaran model	64, 128, 256
2	<i>Learning Rate</i>	Kecepatan pembaruan bobot selama proses training	0.001, 0.0005, 0.01

3	<i>Batch Size</i>	Jumlah data yang diproses sebelum pembaruan bobot	32, 64
4	<i>Jumlah Epoch</i>	Banyaknya iterasi penuh terhadap seluruh dataset	10, 20, 30, 40, 50
5	<i>Dropout Rate</i>	Persentase neuron yang dinonaktifkan sementara untuk mencegah overfitting	0,2 – 0,5
6	<i>Output Units</i>	Jumlah Kelas	3 Kelas
7	<i>Merge Mode</i>	Cara menggabungkan output forward dan backward	Concat, sum, average

3.10 Perancangan Aplikasi

Perancangan sistem merupakan proses pembuatan rancangan dari sistem yang akan dibangun dalam bentuk aplikasi. Pada penelitian ini, perancangan desain sistem dibuat dalam bentuk diagram UML yang terbagi menjadi tiga macam yaitu *use case diagram*, *activity diagram* dan *deployment diagram*.

3.10.1 Use Case Diagram



Gambar 3.12 *Use Case Diagram* Aplikasi

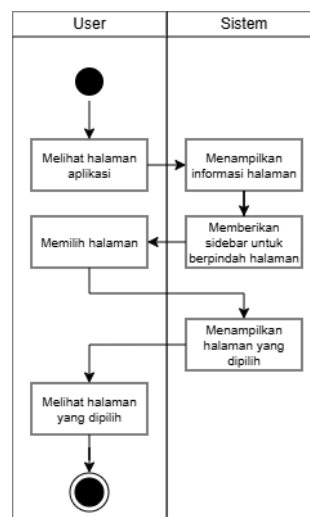
Use Case Diagram digunakan dalam mengvisualisasikan aktivitas yang dapat dilakukan oleh pengguna sistem. Pada sistem ini terdapat menu utama, yaitu halaman depan, halaman *data scraping*, halaman *data preprocessing*, halaman ekstraksi fitur, halaman evaluasi model, halaman komparasi model, dan halaman prediksi sentimen. Seluruh fitur tersebut digambarkan pada *use case diagram* yang dapat dilihat pada gambar 3.12 yang dijabarkan sebagai berikut.

1. Halaman Utama menampilkan deskripsi terkait aplikasi yang dibangun beserta informasi tim pengembang program dan profil *annotator*.
2. Halaman *Data Scraping* menampilkan deskripsi terkait data yang digunakan serta metode pengumpulannya. Pada halaman ini pengguna dapat melihat sampel dataset yang telah dikumpulkan peneliti.
3. Halaman *Data Preprocessing* menampilkan tahapan-tahapan text preprocessing yang digunakan pada penelitian ini secara berurutan. Pengguna dapat melihat hasil text preprocessing maupun melakukan simulasi *text preprocessing*.
4. Halaman *Data Balancing* menampilkan tahapan proses menjadikan data seimbang dengan menggunakan metode *oversampling*.
5. Halaman Ekstraksi Fitur menampilkan hasil dari komputasi TF-IDF dan Word2vec. Pengguna dapat melihat hasil dari TD-IDF dan word2vec yang dilakukan pada dataset.
6. Halaman *Model Evaluation* menampilkan hasil dari evaluasi setiap model yang diuji coba pada penelitian ini.

7. Halaman *Model Comparison* menampilkan komparasi model yang diuji pada penelitian ini dan memberikan opsi model terbaik dengan tingkat akurasi tertinggi yang digunakan pada penelitian ini.
8. Halaman Prediksi Sentimen merupakan halaman Di mana pengguna dapat memasukkan teks secara manual agar dapat diklasifikasi pada kelas sentimen menggunakan model yang paling optimal. Selain itu pengguna dapat memilih menggunakan berbagai macam model dalam mengklasifikasi input teks tersebut.

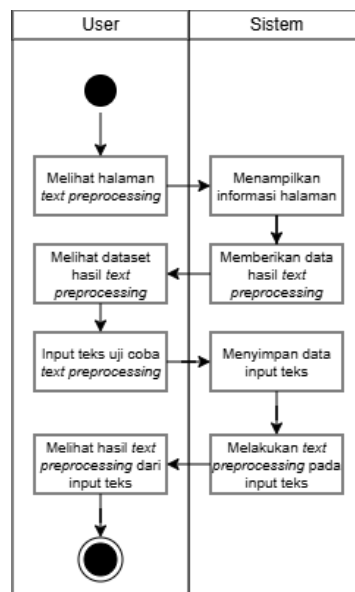
3.10.2 Activity Diagram

Activity Diagram merupakan diagram yang digunakan untuk menggambarkan proses yang terjadi dalam sistem. Pada penelitian ini, *activity diagram* digunakan untuk melakukan visualiasi tahapan interaksi antara pengguna dengan sistem sehingga dapat memudahkan proses pengembangan aplikasi tersebut.



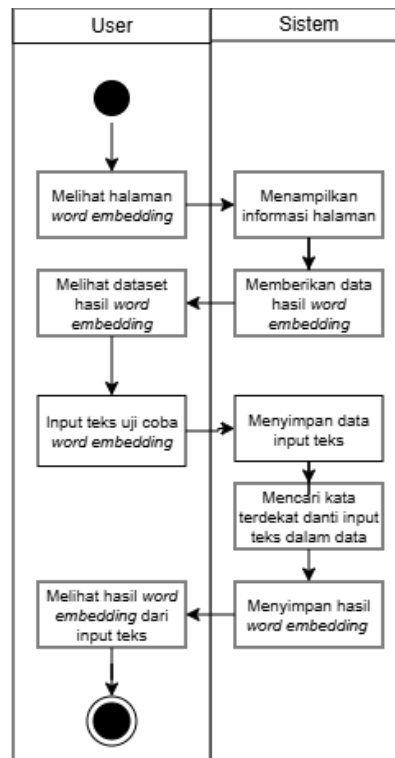
Gambar 3.13 *Activity Diagram* Halaman Utama

Gambar 3.13 menampilkan *activity diagram* pada setiap halaman dalam aplikasi. Di mana pengguna dapat melihat informasi pada halaman utama dan dapat berpindah ke halaman lainnya melalui *sidebar* yang tersedia pada halaman tersebut.



Gambar 3.14 *Activity Diagram* Halaman *Text Preprocessing*

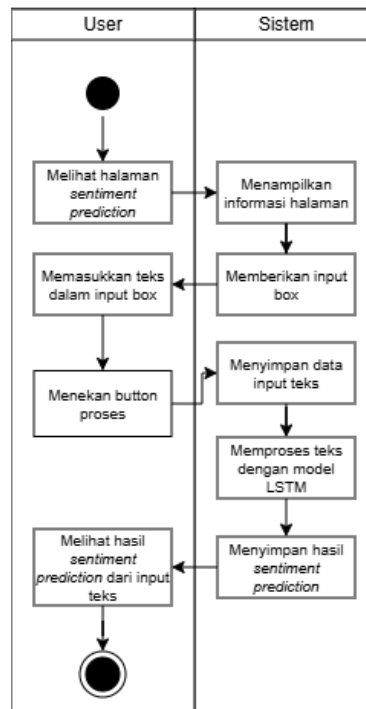
Pada gambar 3.14 merupakan *activity diagram* yang terjadi pada halaman *text preprocessing*. Di mana pengguna dapat melihat hasil dataset yang telah dilakukan *text preprocessing* secara bertahap, dimulai dari *case folding*, *noise removal*, *stopword removal* dan terakhir *lemmatization*. Selain itu, pengguna dapat memasukkan sebuah kalimat pada kolom *input teks* untuk menguji coba *text preprocessing* secara langsung. Sedangkan pada sistem akan memproses teks yang dimasukkan pengguna dan menampilkan hasil akhir dari *text preprocessing* yang dijalankan oleh sistem.



Gambar 3.15 *Activity Diagram Halaman Word Embedding*

Gambar 3.15 menampilkan *activity diagram* pada halaman *word embedding*.

Di mana pengguna dapat melihat hasil dari *word embedding* dataset yang dilakukan pada dataset penelitian. Selain itu, pengguna dapat memasukkan teks pada kolom input teks untuk menguji coba *word embedding*. Kemudian pada sisi sistem, akan memproses teks yang dimasukkan dan memberikan hasil kata kontek yang memiliki makna yang mirip.

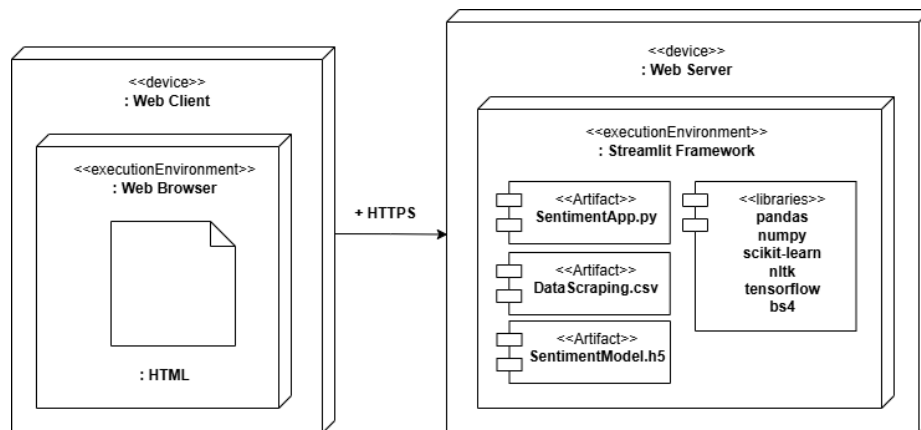


Gambar 3.16 *Activity Diagram* Halaman *Sentiment Prediction*

Pada Gambar 3.16 menampilkan *activity diagram* untuk halaman *sentiment prediction*. Pada halaman tersebut pengguna dapat memasukkan satu kalimat untuk diprediksi kelas sentimennya. Pada sistem, terdapat beberapa model dalam ekstensi .keras dan .pkl, yaitu model yang telah melalui training data sehingga dapat memproses kalimat input dari pengguna dan memberikan hasil prediksi sentimen berdasarkan model.

3.10.3 Deployment Diagram

Deployment Diagram merupakan salah satu diagram yang digunakan untuk menggambarkan struktur sistem. seperti komponen pada perangkat keras, perangkat lunak, dan hubungan antar komponen yang terlibat. *Deployment diagram* berguna untuk memberikan gambaran terkait bagaimana aplikasi berjalan baik dari sisi *client* maupun sisi server.



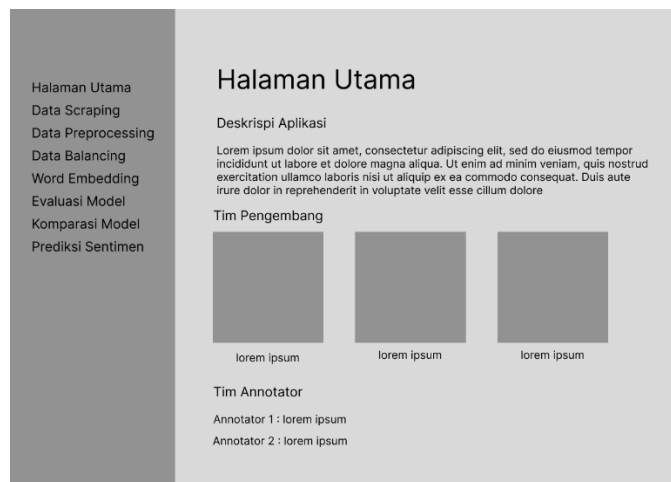
Gambar 3.17 *Deployment Diagram*

Gambar 3.17 merupakan *deployment diagram* dari aplikasi analisis sentimen. Pada diagram tersebut terdapat dua perangkat utama yaitu *Web Client* dan *Web Server*. Di mana *Web Client* berperan sebagai *interface* pengguna yang dapat diakses melalui *Web Browser*. Sedangkan *Web Server* berperan sebagai eksekutor aplikasi, Di mana *Web Server* menjalankan aplikasi menggunakan *Streamlit Framework* dengan berbagai macam *artifact* untuk menjalankan aplikasi tersebut.

3.10.4 Perancangan Antarmuka

Tahap perancangan antarmuka merupakan proses merancangan sistem berdasarkan tampilan yang interaktif dan mudah dipahami oleh pengguna. Setiap halaman dibuat sesuai dengan alur pada *use case diagram*. Adapaun perancangan antarmuka dalam sistem ini terdiri dari beberapa halaman yang dijelaskan sebagai berikut:

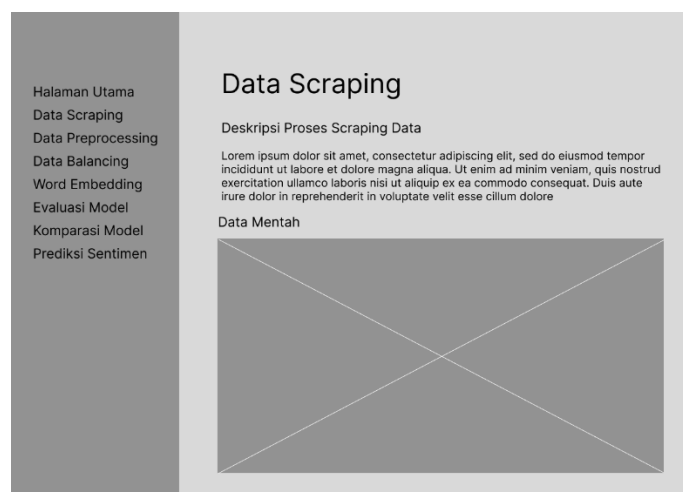
1. Halaman Utama



Gambar 3.18 Desain Halaman Utama

Gambar 3.18 menampilkan rancangan antarmuka untuk halaman utama, Di mana pada halaman tersebut ditampilkan informasi mengenai aplikasi, tim pengembang, serta tim annotator yang berkontribusi dalam penelitian ini. Adapun menu navigaasi pada sisi kiri untuk memudahkan pengguna berpindah ke halaman lainnya.

2. Halaman *Data Scraping*

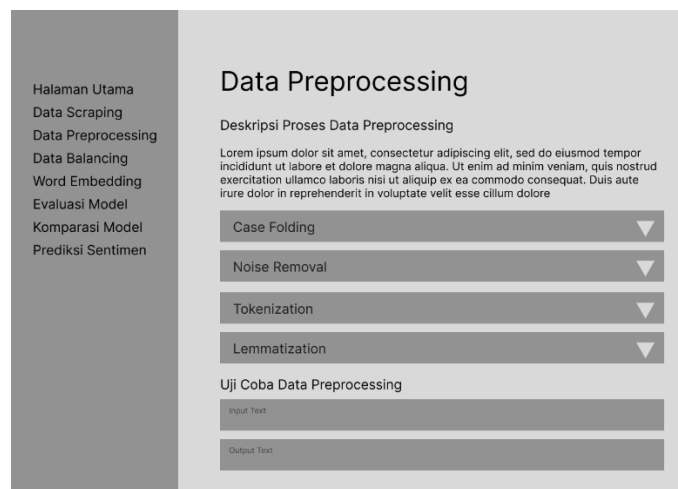


Gambar 3.19 Desain Halaman *Data Scraping*

Gambar 3.19 merupakan rancangan antarmuka halaman *data scraping*, Di mana pengguna dapat mengetahui proses pengambilan data, serta dapat melihat

detail data mentah yang digunakan dalam penelitian. Selain itu, pengguna dapat melakukan filter data berdasarkan kata kunci.

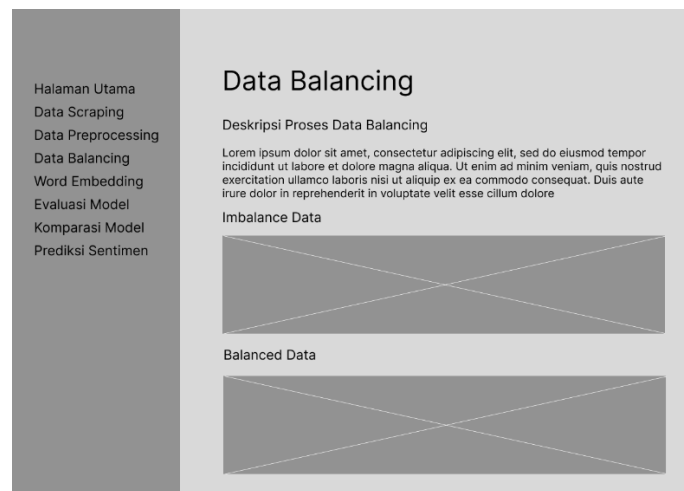
3. Halaman *Data Preprocessing*



Gambar 3.20 Desain Halaman *Data Preprocessing*

Gambar 3.20 menampilkan rancangan antarmuka halaman *data preprocessing*. Pada halaman tersebut pengguna dapat melihat proses *text preprocessing* dari setiap tahapan, mulai dari *casefolding*, kemudian dilanjutkan dengan *noise removal*, *tokenization*, dan tahap terakhir *lemmatization*. Selain itu, pengguna dapat menguji coba *text preprocessing* melalui *input text* secara manual dan aplikasi akan memberikan hasil akhir *text preprocessing* dari *input text* pengguna tersebut.

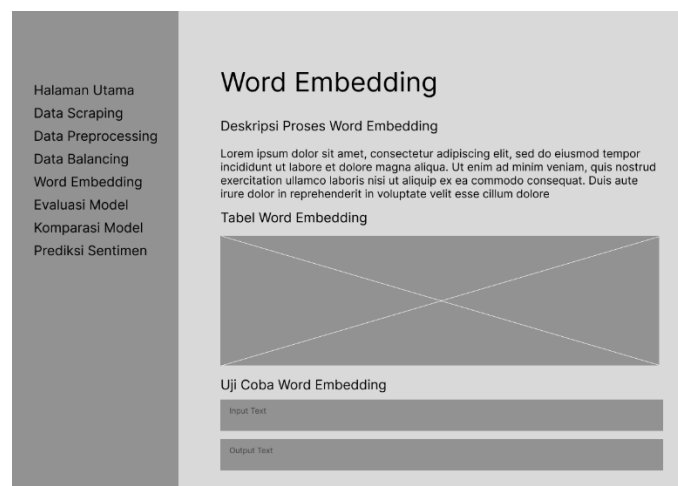
4. Halaman *Data Balancing*



Gambar 3.21 Desain Halaman *Data Balancing*

Gambar 3.21 merupakan rancangan antarmuka halaman *data balancing*. Pada halaman ini pengguna dapat melihat proses peneliti melakukan balancing data menggunakan metode *oversampling*. Halaman ini menunjukkan grafik data sebelum *oversampling* dan grafik setelah data dilakukan *oversampling*.

5. Halaman *Word Embedding*

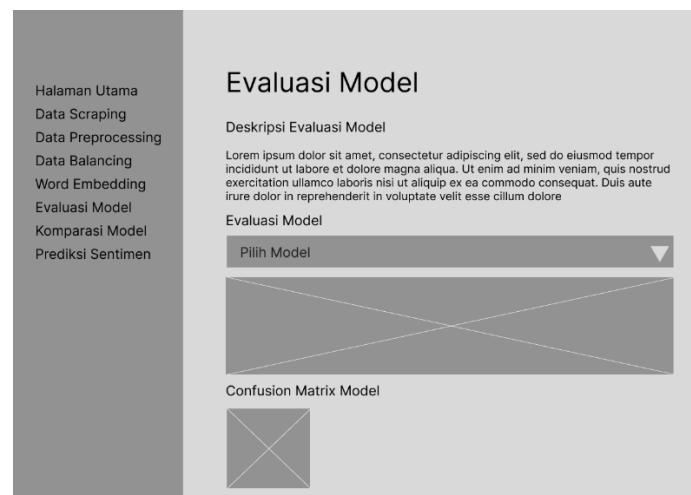


Gambar 3.22 Desain Halaman *Word Embedding*

Gambar 3.22 menunjukkan rancangan antarmuka halaman *word embedding*. Pada halaman ini pengguna dapat melihat tabel hasil *word embedding* dan pengguna dapat melakukan uji coba dengan memasukkan satu kata dalam kolom

input text untuk menghasilkan representasi kata berdasarkan input teks yang diberikan pengguna.

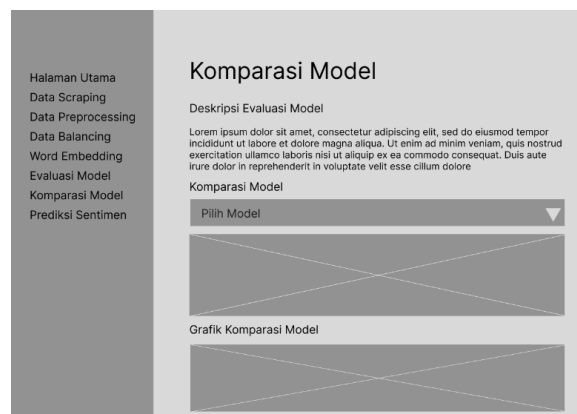
6. Halaman *Model Evaluation*



Gambar 3.23 Desain Halaman *Model Evaluation*

Gambar 3.23 menampilkan rancangan antarmuka halaman *model evaluation*. Di mana pada halaman ini pengguna dapat melihat performa model yang telah dilatih. Selain itu, pengguna dapat memilih model tertentu untuk ditampilkan. Informasi yang ditampilkan pada halaman ini berupa grafik hasil evaluasi dan *confusion matrix*.

7. Halaman *Model Comparison*



Gambar 3.24 Desain Halaman *Model Comparison*

Gambar 3.24 merupakan rancangan antarmuka halaman *model comparison*, Halaman ini merupakan visualisasi perbandingan dari setiap model yang dilatih dalam penelitian ini. Pada halaman ini pengguna dapat memilih model tertentu sehingga pengguna dapat menentukan model terbaik berdasarkan perbandingan evaluasi model tersebut.

8. Halaman *Sentiment Prediction*

Gambar 3.25 Desain Halaman *Sentiment Prediction*

Gambar 3.25 merupakan rancangan antarmuka halaman *sentiment prediction*. Pada halaman ini pengguna dapat menguji model dengan data teks baru. Di mana pengguna dapat memasukkan teks pada kolom input, lalu memilih model yang ingin digunakan, kemudian menekan tombol “proses” untuk mendapatkan hasil prediksi. Pada kolom output prediksi sentimen akan menampilkan kelas sentimen berupa Pro-genosida, Netral atau Kontra-genosida. Kemudian pada kolom Nilai Validation akan menampilkan nilai akurasi prediksi model tersebut dalam menguji input teks secara *real time*.

3.11 Analisis Kebutuhan

Analisis kebutuhan pada penelitian ini terdiri dari kebutuhan perangkat lunak (*software*) dan kebutuhan perangkat keras (*hardware*).

3.11.1 Kebutuhan Perangkat Lunak

Berikut merupakan perangkat lunak yang digunakan dalam penelitian ini, yaitu:

1. Sistem Operasi Microsoft Windows 11.
2. Visual Studio Code
3. Google Collab
4. *Browser* Google Chrome
5. Microsoft Excel
6. Bahasa Pemrograman Python
7. Streamlit

3.11.2 Kebutuhan Perangkat Keras

Dalam melakukan penelitian ini menggunakan perangkat keras yaitu sebuah *laptop* dengan spesifikasi sebagai berikut:

1. Manufaktur : Asus M415D
2. *Processor* : AMD Ryzen 3 3250U CPU 2.5 GHz
3. RAM : 16 GB
4. *Storage* : SSD 500 GB
5. *Graphic Processor* : Radeon Graphics

BAB IV

HASIL DAN PEMBAHASAN

Pada bab ini menjelaskan hasil dari implementasi model *machine learning* termasuk analisis pengujian berdasarkan *hyperparameter* pada model dan membandingkan evaluasi model untuk mendapatkan model paling optimal, selain itu, bab ini menjelaskan bagaimana aplikasi analisis sentimen diimplementasikan dalam bentuk *website*.

4.1 Persiapan Data

Tahapan ini merupakan tahap awal sebelum pelatihan model. pada penelitian ini, data berasal dari media sosial X (Twitter) yang berisi opini publik terkait isu genosida yang dilakukan Isreal terhadap Palestina. Data tersebut kemudian melalui proses *preprocessing* untuk membersihkan teks dari konteks yang tidak dibutuhkan dalam proses analisis sentimen tersebut.

Setelah dilakukan *preprocessing* selanjutnya data dilabel menjadi tiga kategori yaitu pro-genosida, kontra-genosida dan netral. Proses pelabelan ini dibantu oleh pihak ke-tiga yaitu dua *annotator* sehingga pelabelan tidak cenderung bias. Selanjutnya data akan dibagi menjadi dua bagian yaitu data *training* sebanyak 70% dari keseluruhan data, lalu untuk data *validation* sebanyak 20%, dan terakhir untuk data *prediction* sebanyak 10%.

4.1.1 Term Frequency-Inverse Document Frequency

Setelah dataset dilakukan *preprocessing*, maka dilanjut dengan ekstraksi fitur, yaitu mengubah dataset teks menjadi representasi data numerik. Proses komputasi TF-IDF dilakukan secara otomatis menggunakan bahasa pemrograman python dengan *library* sklearn, yang memiliki *function* *feature_extraction* untuk text. Untuk mendapatkan hasil TF-IDF yang optimal untuk model dibutuhkan proses fine-tuning pada TF-IDF tersebut.

Parameter yang digunakan dalam proses tuning TF-IDF adalah *max_feature* dan *ngram_range*. Di mana *max_feature* berfungsi untuk membatasi jumlah fitur kata yang digunakan oleh model, sehingga yang dimasukkan dalam vektor fitur hanya kata dengan bobot tertinggi dan memiliki relevansi yang kuat. Ketika nilai *max_feature* terlalu kecil maka dapat menghilangkan sebagian informasi yang dimiliki dataset. Pada penelitian ini, pengujian *max_feature* yang digunakan adalah dari 1.000 hingga 5.000. Hasil pengujian TF-IDF dapat dilihat pada Tabel 4.2.

Tabel 4.1 Hasil Pengujian Hyperparameter Nilai *Max Feature* TF-IDF

<i>Max Feature</i>	<i>Ngram Range</i>	<i>Accuracy (%)</i>	<i>F1Score (%)</i>	Waktu Latih (detik)
1000	1,1	73,94	73,88	0,022
2000	1,1	75,52	75,49	0,041
3000	1,1	77,10	76,91	0,027
4000	1,1	77,89	77,56	0,023
5000	1,1	77,89	77,56	0,023

Pada hasil pengujian tersebut, dapat disimpulkan bahwa *max feature* dengan nilai 4.000 dan 5.000 merupakan nilai paling optimal untuk TF-IDF dengan tingkat akurasi sebesar 77,89% dan nilai *f1-score* sebesar 77,65%. Selanjutnya pengujian *ngram_range* dengan menggunakan nilai *max_feature* tertinggi pada pengujian

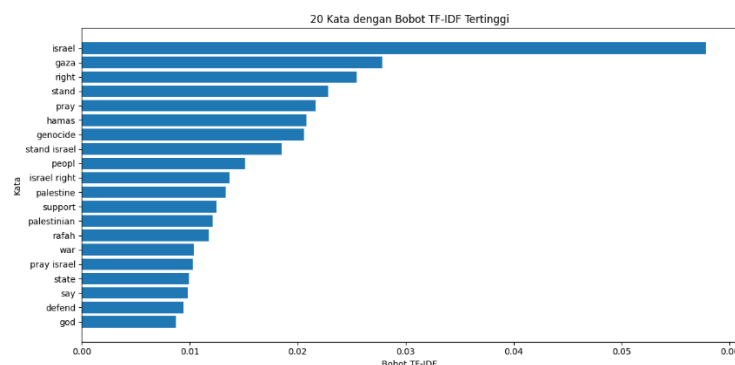
sebelumnya. *Ngram_range* merupakan rentang yang digunakan untuk menentukan panjang *n-gram* yang diproses oleh TF-IDF. Pada penelitian ini, pengujian *ngram_range* menggunakan dua rentang yaitu (1, 1) dan (1, 2) untuk menentukan rentang mana yang lebih efektif diproses TF-IDF. Berikut hasil pengujian dapat dilihat pada Tabel 4.3.

Tabel 4.2 Hasil Pengujian Hyperparameter *Ngram Range* TF-IDF

<i>Max Feature</i>	<i>Ngram Range</i>	<i>Accuracy (%)</i>	<i>F1Score (%)</i>	<i>Waktu Latih (detik)</i>
4000	1,1	77,89	77,56	0,023
4000	1,2	79,21	79,37	0,021

Berdasarkan hasil pengujian *ngram_range*, terdapat peningkatan akurasi secara signifikan dengan *ngram_range* (1, 2). Sehingga parameter akhir yang digunakan untuk proses TF-IDF adalah *max_feature* bernilai 4.000 dan *ngram_range* dengan rentang (1, 2). Hasil akhir pada pengujian menghasilkan akurasi model sebesar 79,21%.

Proses TF-IDF telah menghasilkan bobot untuk setiap kata, dimana setiap bobot menunjukkan tingkat kepentingan kata dalam dokumen. Gambar 4.1 menampilkan 20 kata dengan nilai TF-IDF tertinggi pada dataset penelitian, sedangkan Tabel 4.4 menunjukkan daftar kata dan nilai bobotnya.

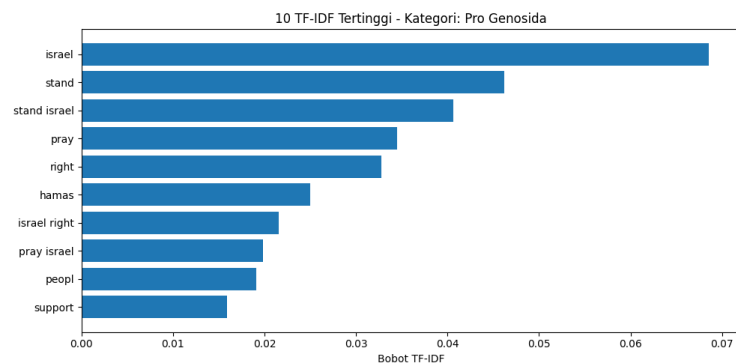


Gambar 4.1 Visualisasi Bobot TF-IDF Tertinggi

Tabel 4.3 20 Kata dengan Bobot TF-IDF Tertinggi

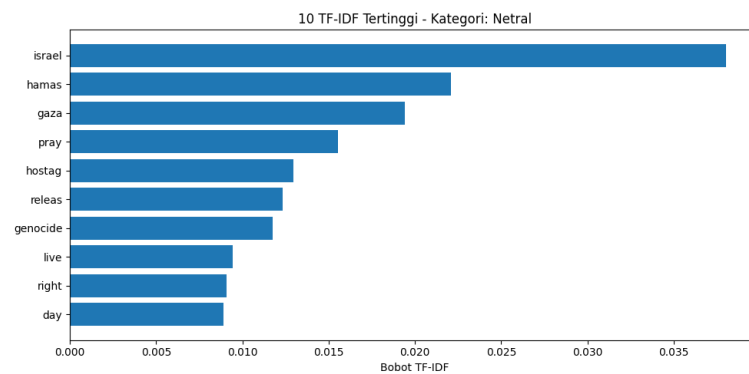
No	Kata	Bobot TF-IDF
1	<i>israel</i>	0.057804
2	<i>gaza</i>	0.027835
3	<i>right</i>	0.025456
4	<i>stand</i>	0.022806
5	<i>pray</i>	0.021681
6	<i>hamas</i>	0.020783
7	<i>genocide</i>	0.020552
8	<i>stand israel</i>	0.018484
9	<i>people</i>	0.015088
10	<i>israel right</i>	0.013651
11	<i>palestine</i>	0.013344
12	<i>support</i>	0.012463
13	<i>palestinian</i>	0.012117
14	<i>rafah</i>	0.011780
15	<i>war</i>	0.010412
16	<i>pray israel</i>	0.010281
17	<i>state</i>	0.009943
18	<i>say</i>	0.009834
19	<i>defend</i>	0.009396
20	<i>god</i>	0.008716

Selanjutnya melakukan visualisasi terhadap bobot tertinggi dari setiap kelas sentimen. Visualisasi ini bertujuan untuk melihat kata-kata yang memiliki pengaruh paling besar dalam membentuk representasi fitur pada setiap kelas. Gambar 4.2 menampilkan 10 kata dengan bobot tertinggi pada sentimen kategori pro-genosida dalam bentuk visualisasi grafik.

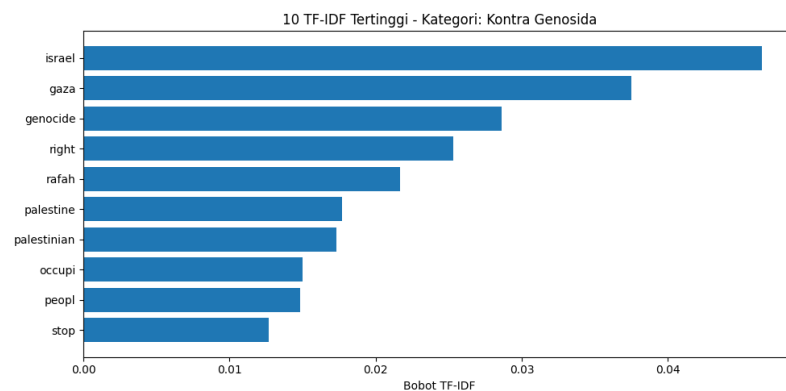


Gambar 4.2 Visualisasi 10 Kata dengan Bobot TF-IDF Tertinggi pada Kelas sentimen Pro-Genosida

Selain itu, terdapat grafik untuk menampilkan 10 kata dengan bobot tertinggi pada kelas sentimen netral dan kontra-genosida. Grafik tersebut dapat dilihat pada Gambar 4.3 dan 4.4.



Gambar 4.3 Visualisasi 10 Kata dengan Bobot TF-IDF Tertinggi pada Kelas sentimen Netral



Gambar 4.4 Visualisasi 10 Kata dengan Bobot TF-IDF Tertinggi pada Kelas sentimen Kontra-Genosida

4.1.2 Word Embedding

Pada tahap ini, dataset yang berbentuk teks diubah menjadi vektor numerik dengan menggunakan model *deep learning*. Pada proses Word2Vec ini, penulis menggunakan *library gensim* pada bahasa pemrograman Python untuk memproses Word2Vec dengan cara yang lebih efektif. Dengan menggunakan Word2Vec,

model dapat memahami makna kata berdasarkan makna dan keterdekataannya dengan kata lain.

Dalam pelatihan Word2Vec, salah satu proses untuk mendapatkan model yang optimal dalam memproses Word2Vec adalah dengan melakukan *tuning* terhadap dimensi vektor dan ukuran *window*. Di mana dimensi yang terlalu rendah dapat menyebabkan representasi kata menjadi kurang valid, dan ketika dimensi terlalu tinggi dapat mengakibatkan model menjadi *overfitting*. Berikut Tabel 4.5 merupakan hasil *tuning* model Word2Vec berdasarkan jumlah dimensi.

Tabel 4.4 Hasil Pengujian *Hyperparameter* Ukuran Dimensi Word2vec

Ukuran Dimensi	Ukuran Window	Learning Rate	Epoch	Accuracy (%)	F1Score (%)	Waktu Latih (detik)
10	2	0,001	10	79,72	78,53	130
50	2	0,001	10	78,74	78,94	197
100	2	0,001	10	80,01	79,76	156
200	2	0,001	10	79,38	77,23	134

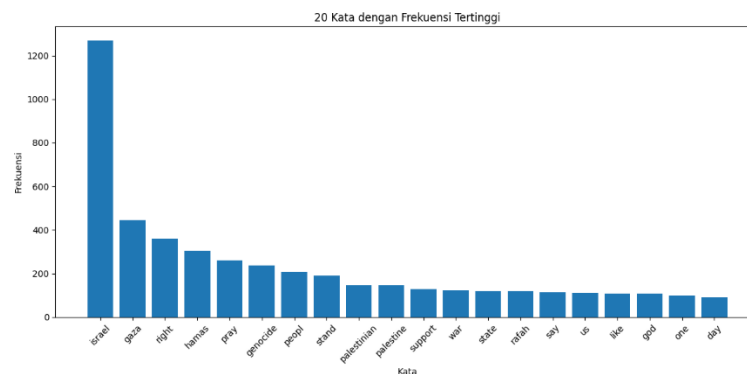
Sementara itu, ukuran *window* dapat menentukan seberapa jauh model dapat menangkap kata-kata setelah maupun sebelum kata target. Di mana ketika *windows* berjumlah kecil maka jangkauannya hanya sedikit sedangkan apabila *window* terlalu besar maka jangkauan menjadi sangat besar dan model akan mengalami bias dalam menentukan kata yang berkaitan dengan kata target.

Tabel 4.5 Hasil Pengujian *Hyperparameter* Ukuran Window Word2vec

Ukuran Dimensi	Ukuran Window	Learning Rate	Epoch	Accuracy (%)	F1Score (%)	Waktu Latih (detik)
100	2	0,001	10	80,01	79,76	135
100	5	0,001	10	79,11	73,62	140
100	10	0,001	10	78,32	76,68	140
100	15	0,001	10	78,48	77,93	142

Tabel 4.6 merupakan hasil akhir dari *tuning* model Word2vec, dimana dimensi yang paling optimal adalah 100 dan *window* yang paling optimal adalah 2 dengan akurasi tertinggi yaitu 80,01%.

Setelah mendapatkan model training Word2Vec yang optimal, selanjutnya adalah melakukan analisis terhadap frekuensi kata yang terdapat pada dataset. Analisis ini digunakan untuk mengidentifikasi kata-kata yang paling sering muncul dalam dataset. Frekuensi kata ditampilkan dalam bentuk gambar grafik dan tabel. Berikut Gambar 4.5 menampilkan grafik 20 kata dengan frekuensi tertinggi dan pada Tabel 4.7 menampilkan jumlah frekuensi dari 20 kata dengan frekuensi tertinggi.



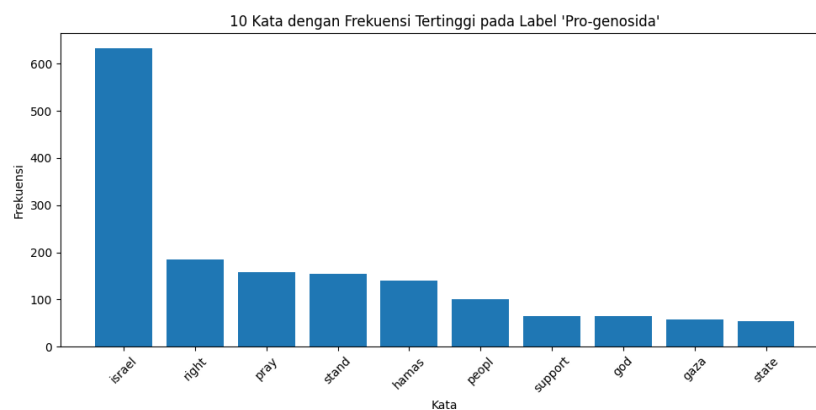
Gambar 4.5 Visualisasi Frekuensi Kata

Tabel 4.6 Frekuensi Kata Tertinggi

No	Kata	Frekuensi
1	<i>israel</i>	1269
2	<i>gaza</i>	444
3	<i>right</i>	360
4	<i>hamas</i>	304
5	<i>stand</i>	259
6	<i>genocide</i>	238
7	<i>people</i>	207
8	<i>stand</i>	189
9	<i>palestinian</i>	147
10	<i>palestine</i>	145
11	<i>support</i>	129

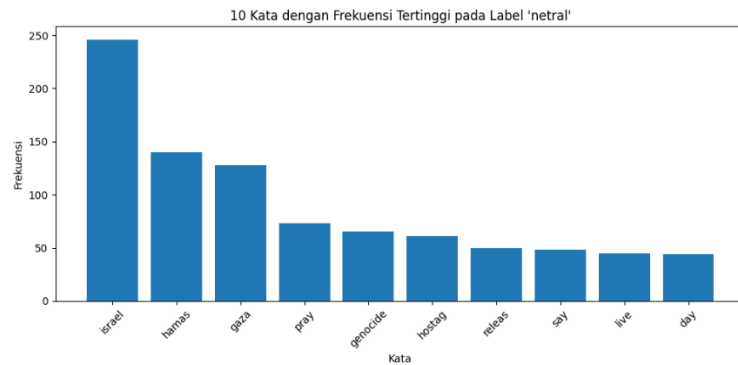
No	Kata	Frekuensi
12	<i>war</i>	123
13	<i>state</i>	120
14	<i>rafah</i>	119
15	<i>say</i>	114
16	<i>us</i>	110
17	<i>like</i>	108
18	<i>god</i>	107
19	<i>one</i>	100
20	<i>day</i>	91

Selanjutnya dilakukan visualisasi frekuensi kata tertinggi pada kelas sentimen pro-genosida. Yaitu kata yang mengandung dukungan terhadap aksi genosida. Hasil visualisasi ini dapat menunjukkan bahwa kata-kata yang memiliki kesamaan konteks berada dalam kulster yang berdekatan. Gambar 4.6 merupakan grafik 20 frekuensi kata tertinggi pada kategori pro-genosida.

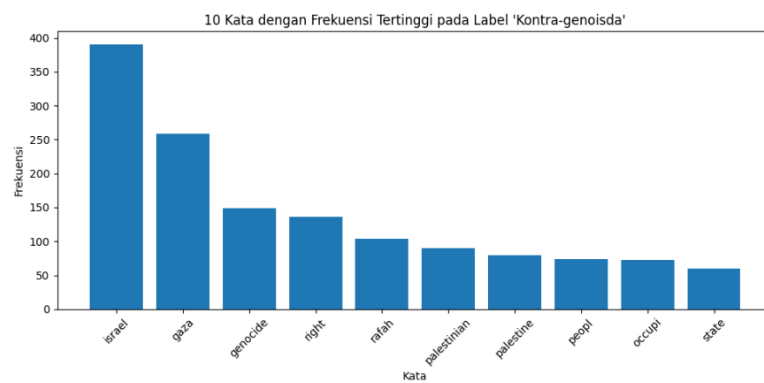


Gambar 4.6 Visualisasi Kata pada Kelas sentimen Pro-Genosida

Pada Gambar 4.7 juga menampilkan grafik frekuensi kata tertinggi pada kategori sentimen netral, dan pada Gambar 4.8 menunjukkan grafik frekuensi kata tertinggi pada kelas sentimen kontra-genosida.



Gambar 4.7 Visualisasi Kata pada Kelas sentimen Netral



Gambar 4.8 Visualisasi Kata pada Kelas sentimen Kontra-Genosida

Selain itu, *output* dari model Word2Vec dapat menghasilkan suatu vektor kata yang memiliki ukuran dimensi 50, dimana dimensi tersebut merepresentasikan vektor dari suatu kata. Gambar 4.9 merupakan contoh hasil dari vektor kata “*genocide*”. Sehingga dengan model tersebut, dapat melihat vektor kata dan dapat memberikan kata yang memiliki kemiripan antar kata. Seperti pada Tabel 4.8 merupakan hasil dari kemiripan antar kata.

```
w2v_model.wv["genocide"]
array([-0.33626577, -0.27807134, -0.21688874,  0.76550466, -0.4982979 ,
        -1.2185589 ,  0.6373322 ,  0.07259982, -2.3091702 , -0.7077824 ,
        -1.943603 , -0.12411962,  1.3606648 ,  1.1833359 , -1.6728894 ,
        1.0907639 ,  0.629384 ,  1.2672061 , -0.29119787,  0.24728204,
        1.2540293 ,  0.8019891 , -0.72826254,  1.0941672 , -1.2169772 ,
        1.8951554 ,  1.2319963 ,  1.0399315 , -0.12934421,  0.533176 ,
        -0.18376787,  1.9081496 , -1.5007813 ,  0.34651473, -1.4623224 ,
        0.87707484, -0.42900646,  0.11126696, -0.18202452, -1.1742359 ,
        -0.91311854,  0.9484639 ,  2.1294773 , -0.25427857,  1.1015408 ,
        0.69800293,  1.4475995 , -1.91323 ,  0.87470305, -0.65890193],
      dtype=float32)
```

Gambar 4.9 Vektor Kata “*genocide*” dari Model Word2Vec

Tabel 4.7 Hasil Kemiripan Antar Kata

10 Kata yang Memiliki Kemiripan dengan Kata “genocide”		
No	Kata	Jarak
1	<i>decadeslong</i>	0.7629748582839966
2	<i>sick</i>	0.735882043838501
3	<i>focus</i>	0.7349097728729248
4	<i>terrorist</i>	0.7324568629264832
5	<i>commit</i>	0.7278032898902893
6	<i>apartheid</i>	0.7250464558601379
7	<i>relief</i>	0.7210209965705872
8	<i>fascist</i>	0.7143982648849487
9	<i>assist</i>	0.7135043740272522
10	<i>colony</i>	0.7093752026557922

4.2 Pengujian Model

Pada tahap ini, penulis melakukan pembangunan serta pengujian model *machine learning* dan *deep learning* dengan cara fine-tuning *hyperparameter*. Untuk mendapatkan model berbasis *machine learning* yang optimal, penulis menggunakan *grid search*. Di mana model diuji secara sistematis dengan berbagai kombinasi parameter hingga mendapatkan performa terbaik dengan menggunakan parameter batasan yang telah ditetapkan.

Sedangkan untuk model *deep learning*, proses optimalisasi model dilakukan dengan fine-tuning manual, hal ini dikarenakan penerapan *grid search* pada model *deep learning* membutuhkan sumber daya komputasi yang jauh lebih besar sehingga dapat menguras penggunaan GPU. Selain itu, dengan penggunaan *google collab* sebagai *execution enviroentmet*, terdapat batasan dalam durasi menggunakan GPU. Oleh karena itu, fine-tuning secara manual menjadi opsi yang paling efisien dalam proses optimalisasi model *deep learning*.

4.2.1 Naive Bayes

Pada proses pengujian model Naive Bayes, penulis menggunakan ekstraksi fitur TD-IDF sebagai kombinasi dengan komputasi model tersebut. Kemudian untuk komponen *hyperparameter* yang digunakan dalam tuning model sudah dijabarkan pada Tabel 3.16, yaitu *alpha*, *fit prior*, dan *norm*. Parameter *alpha* merupakan komponen yang berfungsi untuk menghindari probabilitas yang bias, sedangkan parameter *fit prior* digunakan untuk menentukan probabilitas awal dihitung berdasarkan distribusi data latih atau diasumsikan sama, dan *norm* berfungsi untuk melakukan normalisasi vektor agar klasifikasi menjadi lebih seimbang.

Tabel 4.8 Hasil Tuning Hyperparameter Naive Bayes Menggunakan *Grid Search Cross-Validation*

No	Parameter	Nilai Optimal
1	<i>alpha</i>	0,1
2	<i>fit prior</i>	<i>False</i>
3	<i>norm TF-IDF</i>	11

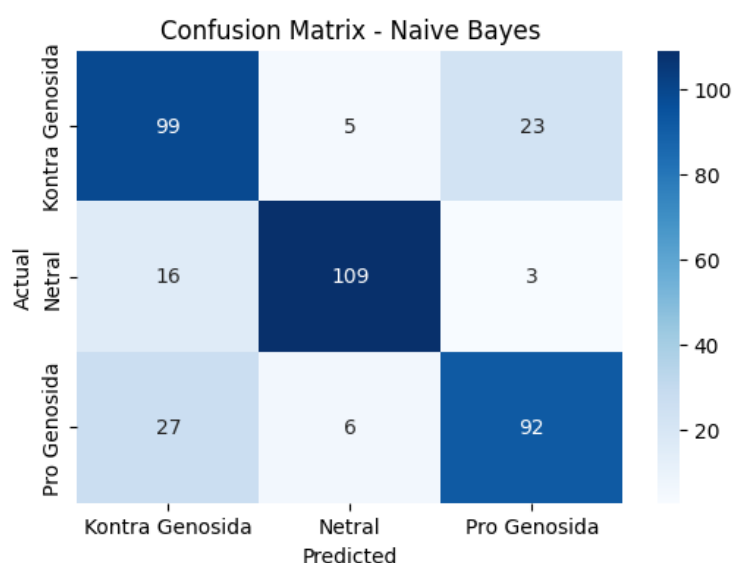
Tabel 4.9 merupakan hasil dari tuning hyperparameter menggunakan metode *grid search cross-validation* dengan *fold* sebesar 10. Hasil tuning tersebut menunjukkan bahwa kombinasi komponen terbaik adalah nilai *alpha* sebesar 0,1, *fit prior* bernilai *false* dan normalisasi TF-IDF menggunakan *norm* 11. Tabel 4.10 menunjukkan hasil evaluasi model yang dihitung berdasarkan rata-rata evaluasi model setiap fold.

Tabel 4.9 Evaluasi Model Naive Bayes

Fold	<i>Accuracy</i>	<i>F1-score</i>	<i>Precision</i>	<i>Recall</i>
1	81.05	81.57	81.03	81.08
2	77.37	78.66	77.34	77.59
3	76.32	78.13	76.23	76.55
4	80.53	81.66	80.54	80.79

Fold	<i>Accuracy</i>	<i>F1-score</i>	<i>Precision</i>	<i>Recall</i>
5	78.42	79.79	78.40	78.79
6	83.16	84.11	83.16	83.34
7	78.31	79.59	78.27	78.30
8	75.66	76.86	75.63	75.97
9	77.78	78.51	77.70	78.00
10	80.42	81.00	80.44	80.53
Rata-rata	78.90	79.09	79.99	78.87

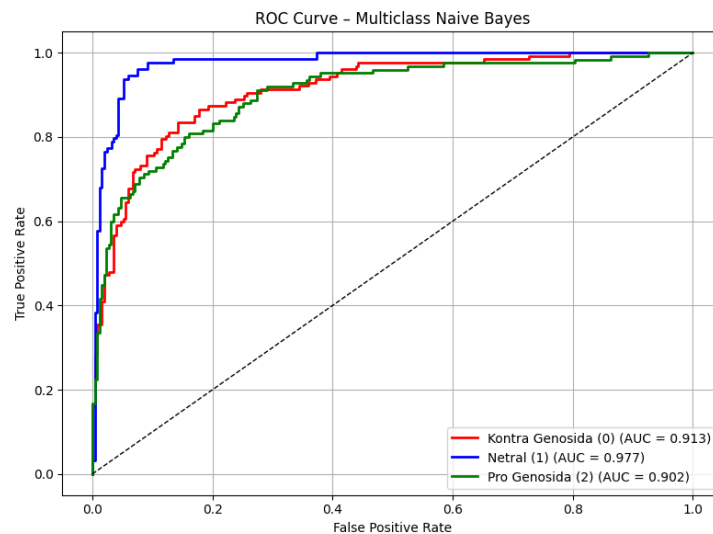
Berdasarkan hasil pengujian menggunakan *fold-10 cross-validation*, model Naive Bayes menghasilkan rata-rata akurasi sebesar 78,90%, nilai *f1-score* sebesar 79,09%, *precision* sebesar 78,87% dan nilai *recall* sebesar 78,87%. Hasil tersebut menunjukkan bahwa performa model relatif stabil pada setiap fold pengujian. Gambar 4.10 merupakan ilustrasi confusion matrix pada pengujian model Naive Bayes.



Gambar 4.10 *Confusion Matrix* Pengujian Model Naive Bayes

Berdasarkan gambar di atas, model Naive Bayes menunjukkan performa klasifikasi yang cukup baik. Pada kelas kontra-genosida terdapat 99 data yang berhasil diklasifikasikan dengan akurat, namun masih terdapat kesalahan sebanyak 23 data mengalami kesalahan dan masuk pada kelas pro-genosida. Sedangkan pada

kelas netral menunjukkan data terbanyak dengan klasifikasi akurat. Dan pada kelas pro-genosida terdapat 27 data mengalami kesalahan dan masuk pada kelas kontra-genosida. Hal ini menunjukkan bahwa adanya indikasi kemiripan pola teks pada kedua kelas tersebut.



Gambar 4.11 Grafik ROC Hasil Pengujian Model Naive Bayes

Berdasarkan grafik di atas, dapat disimpulkan bahwa model dapat membedakan tiga kelas sentimen dengan cukup optimal. Pada kurva ROC kelas netral menunjukkan performansi paling tinggi dengan nilai AUC sebesar 0,977. Sementara itu, pada kelas kontra-genosida nilai AUC yang diperoleh adalah 0,913 dan pada kelas pro-genosida memperoleh nilai AUC sebesar 0,902. Ditunjukkan kurva ROC berada diatas *baseline*, sehingga dapat disimpulkan model Naive Bayes memiliki kemampuan yang optimal dalam mengklasifikasikan kelas sentimen.

4.2.2 Support Vector Machine

Pada penelitian ini, proses tuning untuk mendapatkan model yang optimal pada model SVM dilakukan sama seperti pada model Naive Bayes, yaitu dengan cara menggunakan metode *grid search* dan *cross validation* dengan jumlah *fold*

sebanyak 10, dengan menggunakan batasan parameter yang sudah dijelaskan pada Tabel 3.17. Pada proses tuning tersebut, dihasilkan parameter berdasarkan *nilai f1-score* tertinggi. Tabel 4.11 merupakan hasil tuning parameter model SVM menggunakan grid search.

Tabel 4.10 Hasil Tuning Hyperparameter SVM Menggunakan Grid Search Cross-Validation

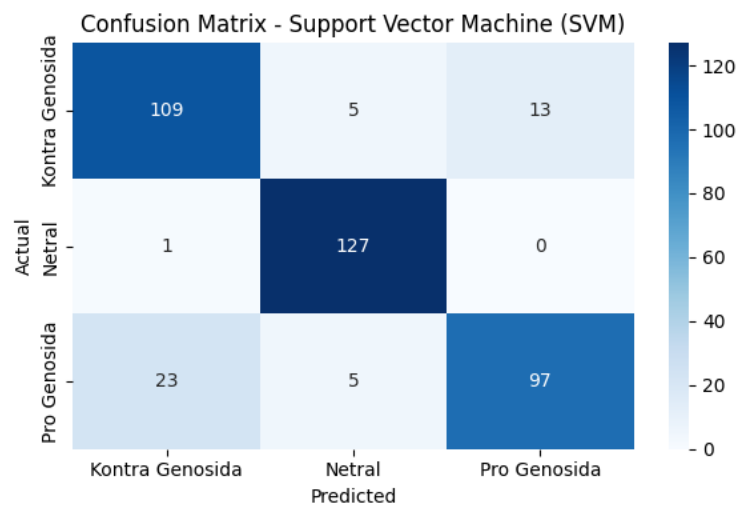
No	Parameter	Nilai Optimal
1	<i>Kernel</i>	<i>RBF</i>
2	<i>C</i>	<i>1</i>
3	<i>gamma</i>	Scale
4	<i>Norm TF-IDF</i>	12

Berdasarkan hasil *grid search* di atas, konfigurasi parameter yang paling optimal untuk model SVM adalah menggunakan kernel RBF, dengan nilai *C* sebesar 1, dengan parameter *gamma* menggunakan *scale* dan normalisasi TF-IDF 12. Seluruh parameter tersebut dipilih berdasarkan hasil nilai *f1-score* tertinggi selama proses cross validation. Tabel 4.12 merupakan hasil evaluasi model SVM paling optimal, dengan *cross validation* sebesar 10 *fold*.

Tabel 4.11 Evaluasi Model SVM

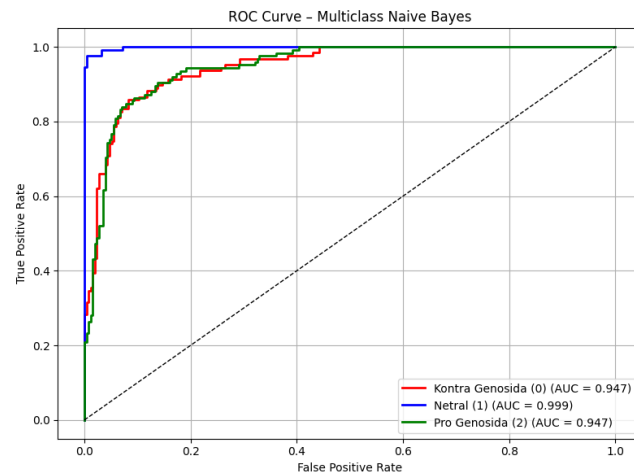
Fold	<i>Accuracy</i>	<i>F1-score</i>	<i>Precision</i>	<i>Recall</i>
1	87.89	88.58	87.77	87.72
2	87.37	87.25	87.25	87.14
3	80.00	80.36	79.89	79.92
4	88.42	88.44	88.37	88.40
5	84.74	84.77	84.67	84.72
6	90.53	90.57	90.49	90.50
7	86.77	87.23	86.64	86.53
8	82.01	82.32	81.93	81.99
9	84.66	84.66	84.54	84.59
10	87.30	87.21	87.22	87.11
Rata-rata	85.97	85.86	86.14	85.88

Berdasarkan hasil *10-fold cross validation*, model SVM menunjukkan performa yang konsisten dengan rata-rata nilai akurasi dan *f1-score* sebesar 85%. Selain menggunakan *cross validation* untuk evaluasi model, performa model SVM juga dianalisis menggunakan *confusion matrix*. Berikut visualisasi dari *confusion matrix* model SVM ditampilkan pada Gambar 4.12.



Gambar 4.12 *Confusion Matrix* Pengujian Model SVM

Dapat dilihat bahwa model SVM dapat mengklasifikasikan kelas netral dengan tingkat akurasi paling tinggi, sementara kesalahan klasifikasi paling banyak terjadi antara kelas pro-genosida dan kontra-genosida. Hal tersebut menunjukkan bahwa kedua kelas memiliki pola yang mirip. Selanjutnya untuk menganalisis lebih jauh performa model, maka evaluasi model menggunakan grafik ROC untuk melihat nilai AUC setiap kelas. Berikut grafik ROC pada evaluasi model SVM dapat dilihat pada gambar 4.23.



Gambar 4.13 Grafik ROC Hasil Pengujian Model SVM

Dari grafik tersebut, dapat disimpulkan bahwa nilai AUC pada setiap kelas berada di atas garis *baseline*, dengan performa kelas netral yang paling tinggi. Hal ini menunjukkan bahwa model SVM memiliki kemampuan klasifikais yang optimal dalam membedakan setiap kelas sentimen.

4.2.3 Long Short-Term Memory

Setelah mendapatkan model *training* yang optimal pada model Word2Vec, selanjutnya adalah tahap pengujian terhadap arsitektur model LSTM untuk mendapatkan performa terbaik dalam mengklasifikasi sentimen. Pada penelitian ini, arsitektur model LSTM berfokus hanya pada satu lapisan LSTM saja dikarenakan dataset yang relatif kurang variatif, sehingga model dapat menimbulkan *overfitting*. Dengan demikian, model LSTM yang diuji hanya menggunakan satu lapisan saja, sedangkan variabel yang diuji mencakup ukuran *embedding*, jumlah unit LSTM, nilai *dropout*, ukuran *batch size*, nilai *learning rate*, dan jumlah *epoch*.

1. Analisis Ukuran *Embedding*

Pengujian pertama yang dilakukan adalah pada lapisan *embedding dimension*. Lapisan ini digunakan untuk mengetahui representasi kata yang paling efektif dalam model LSTM. Ukuran *embedding* yang diuji pada model adalah 50, 100, dan 200 dimensi. Tabel 4.13 merupakan hasil dari pengujian berdasarkan ukuran *embedding dimension*.

Tabel 4.12 Hasil Pengujian Jumlah Ukuran *Embedding Dimension*

Lapisan	Embedding Dimension	Units	Dropout	Batch Size	Learning Rate	Epoch	Accuracy (%)
1	50	32	0,2	16	0,001	10	78,64
1	100	32	0,2	16	0,001	10	80,01
1	200	32	0,2	16	0,001	10	78,58

Berdasarkan hasil pengujian, dapat disimpulkan bahwa ukuran *embedding* sebesar 100 memberikan performa terbaik, sedangkan *embedding* berukuran 50 dan 200 dimensi memberikan performa yang kurang optimal.

2. Analisis Jumlah Unit LSTM

Pengujian berikutnya adalah menguji jumlah unit LSTM, di mana pada penelitian ini, jumlah unit LSTM yang diuji adalah 32, 64 dan 128. Hasil pengujian pada model menunjukkan bahwa jumlah unit 32 dan 64 memberikan performa yang lebih baik dibandingkan pengujian pada jumlah unit 128. Berikut Tabel 4.14 menunjukkan hasil dari pengujian model menggunakan tiga ukuran unit yang berbeda.

Tabel 4.13 Hasil Pengujian Jumlah Unit LSTM

Lapisan	Embedding Dimension	Units	Dropout	Batch Size	Learning Rate	Epoch	Accuracy (%)
1	100	32	0,2	16	0,001	10	80,01
1	100	64	0,2	16	0,001	10	77,42
1	100	128	0,2	16	0,001	10	76,16

Dapat disimpulkan bahwa model yang menggunakan ukuran unit sebesar 32 dan 64 menunjukkan performa yang lebih baik, sedangkan untuk ukuran unit sebesar 128 mengalami penurunan performa yang cukup drastis dikarenakan ketika nilai unit semakin besar maka kompleksitas model semakin meningkat dan bisa meny

3. Analisis *Dropout*

Dropout digunakan untuk mengurangi resiko terjadinya *overfitting* pada model. *Dropout* merupakan sebuah teknik regulasi yang digunakan pada model LSTM untuk memnonaktifkan sejumlah unit secara acak pada proses *training*, sehingga model tidak bergantung pada pola sebelumnya dan mampu melakukan generalisasi yang lebih baik pada data baru. Pada pengujian ini, nilai *dropout* yang digunakan adalah nilai dari 0,1 sampai dengan 0,4. Berikut Tabel 4.15 merupakan hasil dari pengujian model menggunakan nilai *dropout*.

Tabel 4.14 Hasil Pengujian *Dropout*

Lapisan	Embedding Dimension	Units	Dropout	Batch Size	Learning Rate	Epoch	Accuracy (%)
1	100	32	0,1	16	0,001	10	79,22
1	100	32	0,2	16	0,001	10	80,01
1	100	32	0,3	16	0,001	10	78,16
1	100	32	0,4	16	0,001	10	74,10

4. Analisis *Batch Size*

Setelah mendapatkan konfigurasi *dropout* yang optimal, selanjutnya adalah pengujian ukuran *batch*. Di mana *batch size* dapat berpengaruh pada stabilitas pembaruan bobot dalam proses *training*. Apabila *batch size* terlalu kecil maka dapat membuat bobot menjadi tidak stabil. Sedangkan apabila *batch size* terlalu besar maka dapat mengurangi kemampuan generalisasi pada data baru.

Pada pengujian ini, ukuran *batch* yang digunakan adalah 16, 32 dan 64. Tabel 4.8 menunjukkan hasil dari pengujian ukuran *batch size*. Di mana dapat disimpulkan bahwa *batch* ukuran 16 memberikan akurasi tertinggi dibandingkan *batch* berukuran 32 dan 63. Hal ini menunjukkan bahwa dataset pada penelitian ini membutuhkan pembaruan bobot yang lebih intens sehingga menggunakan ukuran *batch* yang paling kecil untuk membantu model LSTM untuk mempelajari pola dataset dengan lebih baik.

Tabel 4.15 Hasil Pengujian *Batch Size*

Lapisan	Embedding Dimension	Units	Dropout	Batch Size	Learning Rate	Epoch	Accuracy (%)
1	100	32	0,2	16	0,001	10	80,01
1	100	32	0,2	32	0,001	10	79,11
1	100	32	0,2	64	0,001	10	78,58

5. Analisis *Learning Rate*

Learning rate merupakan hyperparameter yang penting dalam proses training model. dikarenakan *learning rate* menjadi penentu dalam menilai sudah seberapa besar pembaruan bobot pada setiap iterasi. Sehingga apabila *learning rate* terlalu besar dapat menimbulkan model sulit memperbarui bobot, sedangkan nilai *learning rate* yang kecil dapat menyebabkan *training* berjalan lambat dan resiko terjadinya *crash* pada sistem apabila tidak memadai untuk menjalani program yang kompleks.

Pengujian ini dilakukan dengan lima *learning rate*, yaitu 0.01, 0.005, 0.001, 0,0025 dan 0.0001. Berikut Tabel 4.17 menunjukkan hasil pengujian dari *learning rate*. Di mana nilai *learning rate* 0.0001 menghasilkan model yang paling stabil dan nilai akurasi yang paling tinggi. Sedangkan untuk nilai *learning rate* yang lebih

besar seperti 0.01 menunjukkan bahwa nilai akurasi menurun dengan performa yang rendah.

Tabel 4.16 Hasil Pengujian *Learning Rate*

Lapisan	Embedding Dimension	Units	Dropout	Batch Size	Learning Rate	Epoch	Accuracy (%)
1	100	32	0,2	16	0,01	10	78,91
1	100	32	0,2	16	0,005	10	79,64
1	100	32	0,2	16	0,001	10	80,01
1	100	32	0,2	16	0,0025	10	79,59
1	100	32	0,2	16	0,0001	10	80.09

6. Analisis *Epoch*

Epoch merupakan *hyperparameter* yang mengatur berapa kali seluruh data *training* diproses oleh model. sehingga semakin besar *epoch*, semakin lama model belajar, namun dapat berisiko model mengalami *overfitting* apabila datasetnya sedikit sehingga model hanya mempelajari pola yang kurang variatif secara terus menerus.

Pengujian *epoch* pada model ini hanya menggunakan *epoch* berukuran dari 5 sampai dengan 25, dikarenakan dataset yang relatif sedikit jadi untuk menghindari terjadinya *overfitting* pada model, *epoch* yang digunakan adalah 5, 10, 15, dan 25. Berikut Tabel 4.10 merupakan hasil dari pengujian model menggunakan variasi ukuran epoch.

Tabel 4.17 Hasil Pengujian Ukuran *Epoch*

Lapisan	Embedding Dimension	Units	Dropout	Batch Size	Learning Rate	Epoch	Accuracy (%)
1	100	32	0,2	16	0,0001	5	79,59
1	100	32	0,2	16	0,0001	10	80,01
1	100	32	0,2	16	0,0001	15	79,92
1	100	32	0,2	16	0,0001	25	80.09

7. Hasil Akhir *Hyperparameter* Terbaik

Berdasarkan seluruh rangkaian pengujian *hyperparameter* yang telah diujikan pada model. Sehingga konfigurasi arsitektur model LSTM yang paling optimal dalam penelitian ini yaitu dengan menggunakan satu lapisan LSTM, *embedding dimension* sebesar 100, ukuran unit LSTM sebesar 32, nilai *dropout* 0.2, dengan ukuran *batch size* sebesar 16, nilai *learning rate* sebesar 0.0001, dan jumlah *epoch* 25. Dengan arsitektur tersebut didapatkan nilai akurasi model sebesar 80,09%. Tabel 4.19 menunjukkan hasil akhir dari pengujian *hyperparameter* pada model LSTM.

Tabel 4.18 Hasil Akhir Pengujian *Hyperparameter* Model LSTM

Lapisan	<i>Embedding Dimension</i>	<i>Units</i>	<i>Dropout</i>	<i>Batch Size</i>	<i>Learning Rate</i>	<i>Epoch</i>	<i>Accuracy (%)</i>
1	100	32	0,2	16	0,0001	25	80,09

8. Evaluasi Model LSTM

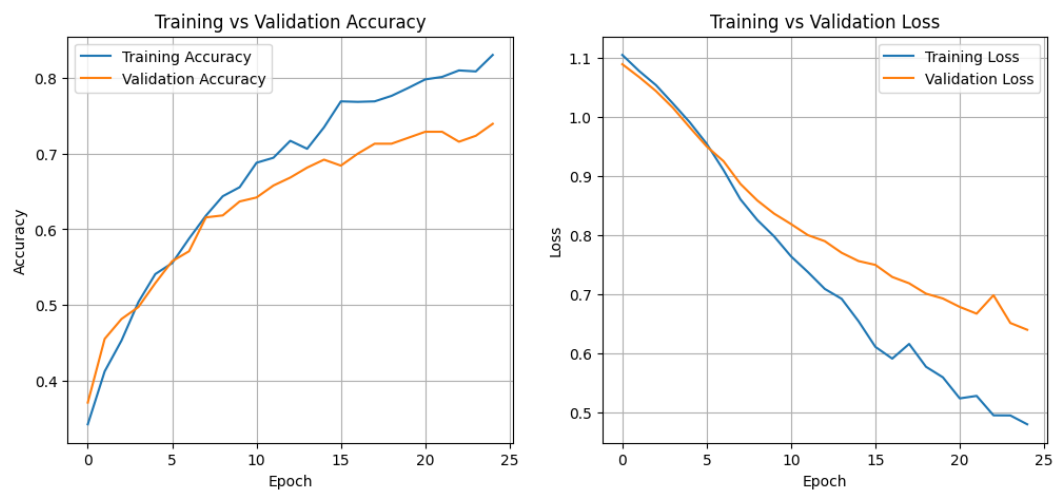
Setelah mendapatkan *hyperparameter* model LSTM paling optimal melalui proses fine-tuning manual. Selanjutnya adalah pengujian dan evaluasi model. pada proses ini, digunakan metode *stratified k-fold cross validation* agar dapat mengevaluasi kinerja model secara menyeluruh. Penggunaan *stratified cross validation* berfungsi untuk menjaga model agar tetap seimbang pada setiap *fold*. Hasil dari evaluasi model dapat dilihat pada Tabel 4.20.

Tabel 4.19 Evaluasi Model LSTM

Fold	<i>Accuracy</i>	<i>F1-score</i>	<i>Precision</i>	<i>Recall</i>
1	81,58	81,22	81,38	81,54
2	84,21	83,59	85,15	84,02
3	74,74	74,56	74,86	74,61
4	78,42	77,94	78,25	78,33
5	83,16	82,83	83,05	83,08
6	83,68	83,55	83,73	83,64
7	79,37	78,94	79,02	79,27

Fold	Accuracy	F1-score	Precision	Recall
8	78,31	78,05	78,25	78,20
9	78,31	77,99	78,19	78,19
10	83,07	82,48	83,22	82,90
Rata-rata	80,09	79,71	80,11	80,08

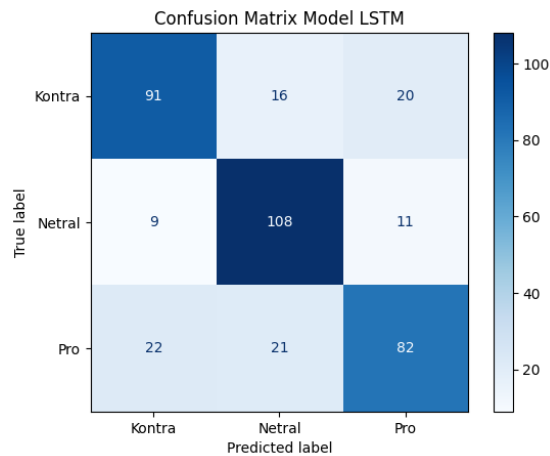
Berdasarkan tabel di atas, model LSTM menghasilkan rata-rata akurasi sebesar 80,85%, dengan *precision* sebesar 80,67%, *recall* sebesar 80,75%, dan nilai *f1-score* sebesar 80,55%. Dengan nilai *precision* dan *recall* yang relatif seimbang, hal tersebut menunjukkan bahwa modal LSTM memiliki kemampuan dalam mengklasifikasikan kelas sentimen tanpa mengalami bias yang signifikan..



Gambar 4.14 Grafik Akurasi dan Loss Model LSTM

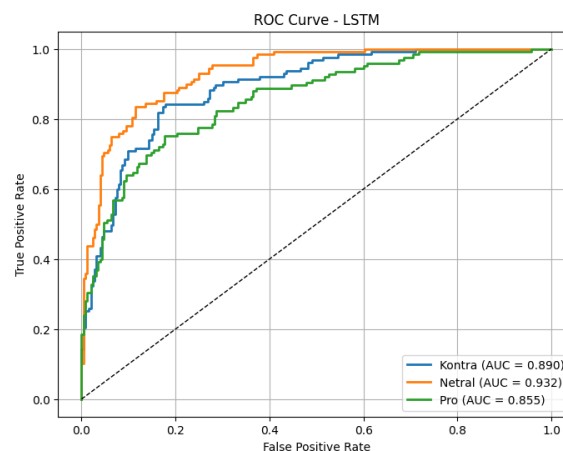
Untuk memastikan model tidak mengalami *overfitting*, ditampilkan grafik akurasi *training* dan *validation*, begitupun dengan nilai *loss* pada *training* dan *validation*. Grafik tersebut dapat dilihat pada gambar 4.14. pada grafik tersebut ditunjukkan bahwa nilai akurasi pada *training* dan *validation* mengalami peningkatan secara stabil. Sedangkan pada nilai *loss*, kedua mengalami penurunan yang konsisten juga. Hal ini menunjukkan bahwa model LSTM tidak mengalami

overfitting. selanjutnya evaluasi model menggunakan *confusion matrix* dan grafik ROC, visualisasi *confusin matrix* dapat dilihat pada gambar 4.15.



Gambar 4.15 *Confusion Matrix* Pengujian Model LSTM

Pada *confusion matrix* di atas, dapat disimpulkan bahwa kelas netral merupakan kelas yang memiliki tingkat akurasi klasifikasi tertinggi, sedangkan kelas pro-genosida dan kelas kontra-genosida memiliki data akurat yang hampir mirip. Selain itu, pada kelas pro-genosida terdapat 25 data yang mengalami kesalahan klasifikasi ke kelas kontra-genosida. Selanjutnya merupakan grafik ROC pada model LSTM yang ditampilkan pada Gambar 4.16.



Gambar 4.16 Grafik ROC Hasil Pengujian Model LSTM

Berdasarkan grafik tersebut, ditunjukkan bahwa kelas netral memiliki nilai AUC paling tinggi di antara seluruh kelas sentimen, sedangkan kelas kontra dan pro memiliki nilai AUC yang memiliki jarak yang dekat. Dan seluruh kelas sentimen berada di atas garis *baseline* sehingga dapat disimpulkan bahwa Model LSTM sudah optimal dalam mengklasifikasi sentimen yang bersifat multikelas.

4.2.4 Bidirectional Long Short-Term Memory

Hyperparameter yang digunakan pada proses fine-tuning model BiLSTM merupakan *hyperparameter* hasil akhir dari fine-tuning model LSTM. Namun pada model BiLSTM tuning ditambahkan dengan parameter *merge mode*, yaitu parameter yang digunakan untuk menggabungkan lapisan *forward* dan *backward* LSTM. *Merge mode* memiliki tiga macam teknik, yaitu *concatenate* yang berfungsi menggabungkan keseluruhan, *sum* berfungsi menjumlahkan hasil akhir, dan *average* berfungsi menghitung rata-rata. Berikut hasil akhir dari pengujian model BiLSTM menggunakan tiga variasi *merge mode*, ringkasan pengujian dapat dilihat pada tabel 4.21.

Tabel 4.20 Hasil Pengujian Hyperparameter *Merge Mode* pada Model BiLSTM

<i>Merge Mode</i>	<i>Embedding Dimension</i>	<i>Units</i>	<i>Dropout</i>	<i>Batch Size</i>	<i>Learning Rate</i>	<i>Epoch</i>	<i>Accuracy (%)</i>
<i>Concat</i>	100	32	0,2	16	0,0001	25	77.90
<i>Sum</i>	100	32	0,2	16	0,0001	25	76,24
<i>Average</i>	100	32	0,2	16	0,0001	25	77,48

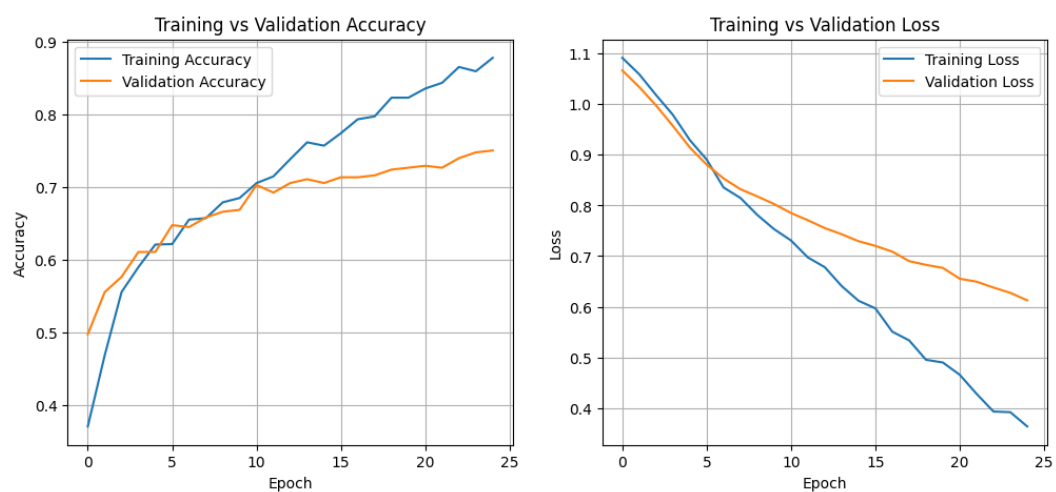
Berdasarkan hasil pengujian di atas, dapat disimpulkan bahwa *merge mode* menggunakan *concat* memiliki nilai akurasi paling tinggi, yaitu sebesar 77,90%. Selanjutnya adalah evaluasi model menggunakan *hyperparameter* paling optimal dan menggunakan *cross validation* dengan *fold* sebesar 10 untuk memastikan

model berjalan stabil. Tabel 4.22 merupakan evaluasi model BiLSTM dengan *cross validation*.

Tabel 4.21 Evaluasi Model BiLSTM

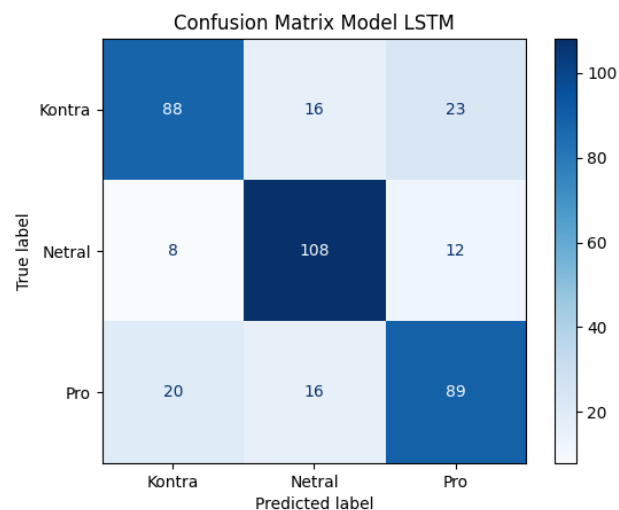
Fold	<i>Accuracy</i>	<i>F1-score</i>	<i>Precision</i>	<i>Recall</i>
1	78.95	79.35	78.85	78.69
2	82.11	82.10	82.06	82.01
3	74.74	74.28	74.65	74.14
4	74.74	74.70	74.73	74.56
5	74.74	75.05	74.69	74.82
6	82.11	82.00	82.06	81.98
7	78.31	78.21	78.19	77.94
8	73.02	73.32	72.94	72.87
9	81.48	81.26	81.37	81.17
10	78.84	79.23	78.82	78.74
Rata-rata	77.90	77.95	77.84	77.84

Untuk memastikan model tidak mengalami *overfitting*, dilakukan analisis menggunakan grafik perbandingan antara akurasi dengan *loss* pada data *training* dan *validation*. Grafik akurasi dan *loss* dapat dilihat pada Gambar 4.17. pada grafik tersebut ditunjukkan bahwa akurasi *training* dan *validation* mengalami peningkatan secara konsisten. Sedangkan pada *loss* terdapat penurunan secara signifikan. Sehingga dapat disimpulkan model tidak mengalami *overfitting*.



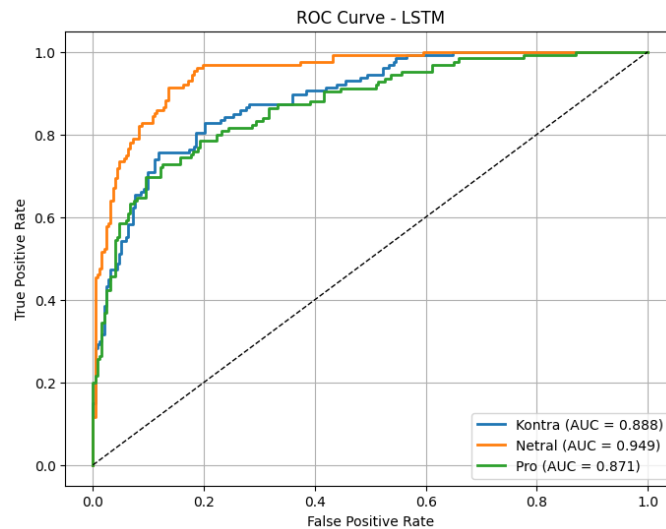
Gambar 4.17 Grafik Akurasi dan *Loss* Model BiLSTM

Selain menggunakan evaluasi metrik akurasi, *precision*, *recall* dan *f1-score*. Performa model juga dianalisis menggunakan *confusion matrix* dan grafik ROC. Visualisasi *confusion matrix* dapat dilihat pada gambar 4.18. berdasarkan gambar tersebut ditunjukkan bahwa model BiLSTM dapat mengklasifikasikan kelas netral dengan performa paling baik. Sementara itu, terjadi kesalahan klasifikasi antara kelas kontra-genosida dan pro-genosida, yang menunjukkan bahwa adanya kemiripan pola pada kedua kelas tersebut.



Gambar 4.18 *Confusion Matrix* Model BiLSTM

Evaluasi selanjutnya adalah menggunakan grafik ROC dan nilai AUC. Gambar 4.19 menunjukkan bahwa model BiLSTM memiliki nilai AUC sebesar 0,888 untuk kelas sentimen kontra-genosida, 0,949 untuk kelas netra, dan 0,871 untuk kelas sentimen pro-genosida. Dengan nilai tersebut, ditampilkan bahwa grafik setiap kelas berada di atas garis *baseline*, sehingga dapat disimpulkan bahwa model BiLSTM memiliki kemampuan klasifikasi multikelas sentimen yang baik.



Gambar 4.19 Grafik ROC Model BiLSTM

4.3 Perbandingan Evaluasi Model

Setelah mendapatkan performa terbaik dari setiap model, selanjutnya adalah analisis perbandingan evaluasi model. perbandingan ini menggunakan nilai rata-rata dari akurasi, *precision*, *recall*, dan *f1-score*. Hasil perbandingan evaluasi model ditunjukkan pada Tabel 4.23.

Tabel 4.22 Perbandingan Evaluasi Model

Model	Nilai Rata-rata			
	<i>Accuracy (%)</i>	<i>Precision (%)</i>	<i>Recall (%)</i>	<i>F1-Score (%)</i>
Naive Bayes	78.90	79.99	78.87	79.09
SVM	85.97	86.14	85.88	85.86
LSTM	80.09	80.11	80.08	79.71
BiLSTM	77.90	77.84	77.84	77.95

Berdasarkan tabel di atas, model SVM merupakan model dengan performa paling baik dengan nilai rata-rata akurasi sebesar 85,97% dan nilai *f1-score* sebesar 85,86%. Kemudian berikutnya adalah model LSTM dengan nilai akurasi sebesar 80,09% dan *f1-score* sebesar 79,71%, diikuti oleh model Naive Bayes dengan nilai yang hampir sama dengan model LSTM yaitu nilai akurasi 78,90% dan *f1-score*

79,09%. Dan performa terakhir adalah model BiLSTM dengan akurasi sebesar 77,90% dan *f1-score* sebesar 77,95%

Dengan demikian, model SVM merupakan model terbaik dalam penelitian ini dan melampaui model *deep learning* seperti LSTM dan BiLSTM yang memiliki kemampuan komputasi yang lebih kompleks.

4.4 Perbandingan Klasifikasi Model

Selain melakukan perbandingan evaluasi model, penelitian ini juga membandingkan hasil dari prediksi setiap model. perbandingan ini untuk melihat bagaimana model mengklasifikasikan data baru ke dalam tiga kelas sentimen. Hasil dari perbandingan klasifikasi dapat dilihat pada Tabel 4.24.

Tabel 4.23 Perbandingan Hasil Klasifikasi Model

No	Teks	Label Aktual	Model			
			Naive Bayes	SVM	LSTM	BiLSTM
1	<i>Israel is preventing humanitarian aid from entering Gaza, worsening the humanitarian crisis.</i>	Kontra	Kontra	Kontra	Kontra	Kontra
2	<i>Schools in Gaza have been destroyed, leaving young people already deprived of education due to COVID-19 and war, and facing a loss of future opportunities.</i>	Kontra	Kontra	Kontra	Kontra	Kontra

No	Teks	Label Aktual	Model			
			Naive Bayes	SVM	LSTM	BiLSTM
3	<i>Three Israeli hostages were handed over by Hamas in Gaza, while Palestinian prisoners were released as part of a ceasefire deal.</i>	Netral	Kontra	Kontra	Netral	Netral
4	<i>A Chicago-based surgeon has treated patients with war-related injuries from Syria, Ukraine, and Gaza.</i>	Netral	Netral	Kontra	Netral	Netral
5	<i>Supporting Israel is described as supporting humanity, while opposing Hamas terrorism is also framed as a defense of human values.</i>	Pro	Pro	Pro	Pro	Pro
6	<i>Hamas is the true terrorist.</i>	Pro	Pro	Pro	Pro	Pro

Berdasarkan tabel di atas, dapat dilihat perbandingan klasifikasi sentimen pada enam kalimat diuji menggunakan setiap model dengan hasil dari tuning paling optimal. Pada dasarnya, seluruh model dapat mengklasifikasikan sentimen yang bersifat eksplisit. Terdapat kesalahan klasifikasi pada model Naive Bayes dan SVM dalam mengklasifikasikan sentimen kelas netral. Sedangkan pada model LSTM dan BiLSTM memiliki kemampuan yang lebih baik dalam mengenali sentimen netral.

Hasil ini menunjukkan bahwa model berbasis *deep learning* lebih baik dalam mengklasifikasikan data baru meskipun secara tingkat akurasi tidak setinggi model pada *machine learning*.

4.5 Implementasi Aplikasi

Setelah melakukan pengujian pada *hyperparameter* pada model *machine learning* dan *deep learning*. Selanjutnya model-model yang optimal digunakan dalam implementasi aplikasi, sehingga pengguna dapat menguji model dengan mudah. Aplikasi tersebut dibangun menggunakan *framework* bahasa pemrograman Python yaitu Streamlit.

Aplikasi berbasis website ini dapat menampilkan dataset yang digunakan oleh model, melihat hasil dari proses *text preprocessing* dan *word embedding*, dan pengguna dapat menguji prediksi sentimen menggunakan model yang telah dilatih sebelumnya. Aplikasi tersebut dapat diakses pada link <https://skripsi-2025-sentiment-analyse.streamlit.app/>.

4.5.1 Halaman Utama

Pada halaman utama aplikasi ditampilkan deskripsi singkat tentang penelitian ini dan bagaimana proses penelitian tersebut. Kemudian pada halaman tersebut pengguna dapat melihat informasi tim pengembang dan tim *annotator* yang terlibat dalam penelitian ini.

Berikut adalah *source code* pada halaman utama. Di mana aplikasi ini menggunakan *framework* Streamlit sehingga perlu *import* library streamlit. Kemudian *library pathlib* digunakan untuk memanggil data yang berada di luar folder aplikasi.

```

import streamlit as st
from pathlib import Path
from PIL import Image

st.header('ANALISIS SENTIMEN TERHADAP ISU GENOSIDA ISRAEL KEPADA
PALESTINA')

st.write("""
    Aplikasi ini merupakan hasil dari penelitian skripsi
    penulis yang berjudul *Analisis Sentimen terhadap Isu Genosida
    Israel kepada Palestina pada Media Sosial X (Twitter)*.

    Aplikasi ini berisi proses dari penelitian ini, bermula
    dari scraping dataset, pre-processing, labelling, balancing,
    word2vec. kemudian adapun komparasi model paling efektif yang
    digunakan dalam penelitian ini. dan pengguna pada mencoba
    mengklasifikasi sentimen secara real time.
    """)
st.markdown("----")

st.subheader("👤 Tim Pengembang")

BASE_DIR = Path(__file__).resolve().parent
image_developer = BASE_DIR / "image" / "wafa.jpg"
image_dosbim = BASE_DIR / "image" / "Bu Helen.jpg"
image_dosbim2 = BASE_DIR / "image" / "Bu Mira.jpg"

col1, col2, col3 = st.columns(3)

with col1:
    if image_developer.exists():
        st.image(str(image_developer), width=100)
        st.write("*Wafa Tsabita*")
        st.write("***Developer***")
    else:
        st.error(f"Gambar tidak ditemukan: {image_developer}")

with col2:
    if image_dosbim.exists():
        st.image(str(image_dosbim), width=100)
        st.write("*Dr. Afrida Helen, S.T., M.Kom.*")
        st.write("***Pembimbing Utama***")
    else:
        st.error(f"Gambar tidak ditemukan: {image_dosbim}")

with col3:
    if image_dosbim2.exists():
        st.image(str(image_dosbim2), width=100)
        st.write("*Mira Suryani, S.Pd, M.Kom.*")
        st.write("***Co-Pembimbing***")
    else:
        st.error(f"Gambar tidak ditemukan: {image_dosbim2}")

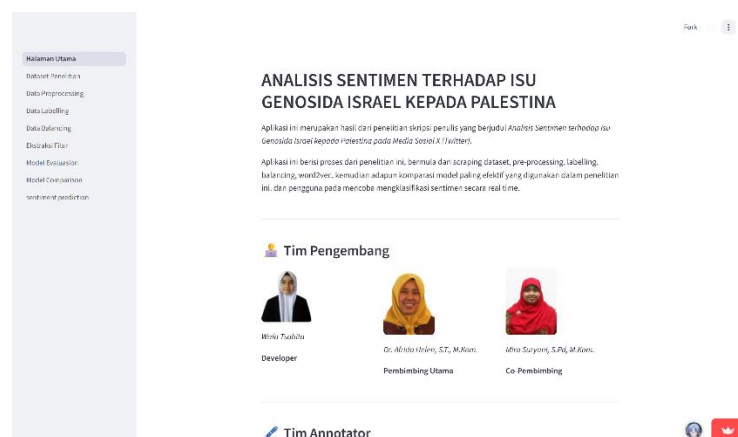
st.markdown("----")

st.subheader("✍️ Tim Annotator")

```

```
st.write("""
- Annotator 1 : Musfirah Qisthi Tardauna
- Annotator 2 : Fatimah Noor Albirkah
""")
```

Tampilan halaman utama dapat dilihat pada Gambar 4.20, pada halaman tersebut ditampilkan deskripsi singkat alur penelitian, tim pengembang dan tim *annotator*. Selain itu, terdapat *side bar* yang berfungsi sebagai link untuk pindah ke halaman lain.



Gambar 4.20 Halaman Utama

4.5.2 Halaman Dataset Penelitian

Berikutnya adalah halaman dataset penelitian, pada halaman ini terdapat informasi terkait proses *scraping* data *tweet* dengan *keyword* tertentu, halaman ini menampilkan dataset mentah yang telah dikumpulkan selama penelitian ini berlangsung. Selain itu, pengguna dapat menggunakan fitur filter untuk menampilkan dataset berdasarkan kata kunci tertentu. Tampilan halaman dataset penelitian dapat dilihat pada Gambar 4.21.

conversation_id	created_at	favorite_count	full_text
15081007095352872	Sun Apr 06 23:46:53 +0000 2025	4	Kata Israel #Gaza #GazaHocust
1508100889517345322	Sun Apr 06 23:56:26 +0000 2025	0	ISRAELI Reports confirm the c
1508100727040193723	Sun Apr 06 23:56:04 +0000 2025	0	I just pledged my support to gth
15081013001673465	Sun Apr 06 23:55:02 +0000 2025	2	Not taking a position on Gaza is to
1508101183121260551	Sun Apr 06 23:51:16 +0000 2025	266	We talked them, Shame on us #Ga
1508101351304619022	Sun Apr 06 23:51:56 +0000 2025	117	The children of Gaza stand on pro
150810121320603066	Sun Apr 06 23:47:03 +0000 2025	2	Thank you @B66644 for your e
1508102823325091102	Sun Apr 06 23:36:28 +0000 2025	3	I just donated to a family in Gaza
15081018511745211021	Sun Apr 06 23:52:26 +0000 2025	1	Quarman Noor INSTRUCTIONS: H
150810124003747782	Sun Apr 06 23:55:08 +0000 2025	0	Remember to myself if I feel like i

Gambar 4.21 Halaman Dataset Penelitian

Berikut merupakan *source code* untuk menampilkan dataset dalam bentuk tabel dan memberikan opsi filter pada pengguna.

```
st.subheader("🔍 Filter Data Berdasarkan Kata Kunci")

keywords = [
    'genocide', 'gen0cide',
    'gaza', 'g@za', 'gz@',
    'palestine', 'palestine', 'pa|estine', 'p@lestine', 'plstn',
    'israel', 'izrael', 'isr@el', 'lIsrael'
]

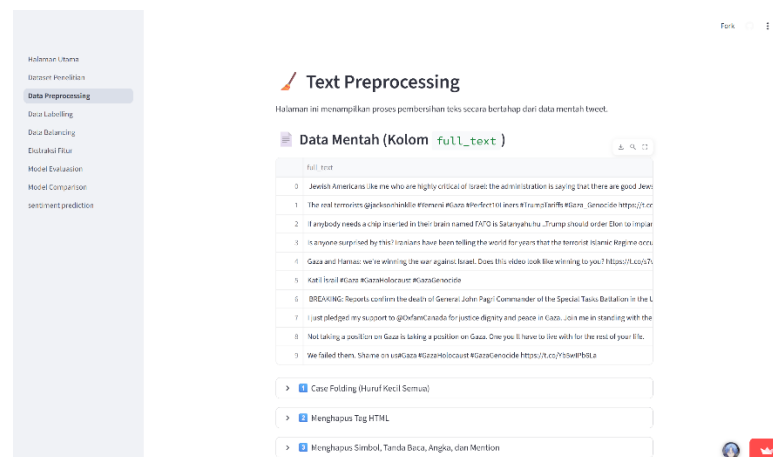
selected_keywords = st.multiselect("Pilih Kata Kunci",
sorted(set(keywords)))

if selected_keywords:
    pattern = '|'.join(selected_keywords)
    filtered = df[df['full_text'].str.contains(pattern,
case=False, na=False)]
    st.markdown(f"**Menampilkan data untuk keyword : `{',
'.join(selected_keywords)}`**")
    st.dataframe(filtered)
else:
    st.info("Silahkan pilih satu atau lebih keyword untuk
menampilkan data.")
```

4.5.3 Halaman *Data Preprocessing*

Pada halaman ini, pengguna dapat melihat proses *preprocessing* pada data secara bertahap, dimulai dari *case folding*, *noise removal*, *stopword removal*, lematisasi dan *stemming*. Selain itu, pengguna dapat menguji *preprocessing* teks secara *real time* dengan cara memasukkan teks pada kolom input dan aplikasi akan

menjalani fungsi *preprocessing* dan menampilkan hasil akhir dari teks yang telah dimasukkan pengguna. Gambar 4.22 merupakan tampilan halaman *data preprocessing*.



Gambar 4.22 Halaman *Data Preprocessing*

Proses *preprocessing* pada aplikasi ini menggunakan *library regex* dan *nltk* untuk memudahkan proses eksekusi. Berikut merupakan potongan *source code* untuk menampilkan dataset dengan *data preprocessing* yang dilakukan secara bertahap.

```
def case_folding(text):
    return text.lower()

case_folded = texts.apply(case_folding)

with st.expander("1 Case Folding (Huruf Kecil Semua)"):
    st.dataframe(pd.DataFrame({'case_folded':
    case_folded.head(10)}))

# Tahap 2: Menghapus HTML tags
def remove_html(text):
    return BeautifulSoup(text, "html.parser").get_text()

html_cleaned = case_folded.apply(remove_html)

with st.expander("2 Menghapus Tag HTML"):
    st.dataframe(pd.DataFrame({'html_cleaned':
    html_cleaned.head(10)}))

# Menghapus simbol, tanda baca, angka, dan mention
def remove_symbols_numbers_mentions(text):
```

```

text = re.sub(r'@\w+', '', text)
text = re.sub(r'^\w\s', '', text)
text = re.sub(r'\d+', '', text)

symbols_cleaned =
html_cleaned.apply(remove_symbols_numbers_mentions)

with st.expander("3 Menghapus Simbol, Tanda Baca, Angka, dan
Mention"):
    st.dataframe(pd.DataFrame({'symbols_cleaned':
symbols_cleaned.head(10)}))

```

Selain itu, berikut merupakan *source code* untuk melakukan *preprocessing* pada data teks yang dimasukkan pengguna secara *real time* melalui *interface* aplikasi. Pada proses tersebut, program melakukan *preprocessing* secara bertahap mulai dari *case folding* hingga *stemming*. Kemudian hasil akhir dari *preprocessing* tersebut dimunculkan pada *interface* aplikasi.

```

if st.button("🔄 Preprocess Teks"):
    if not input_text.strip():
        st.warning("Silakan masukkan teks terlebih dahulu.")
    else:
        # Jalankan semua tahap preprocessing
        text = case_folding(input_text)
        text = remove_html(text)
        text = remove_symbols_numbers_mentions(text)
        text = remove_stopwords(text)
        text = lemmatize_text(text)
        text = stem_text(text)

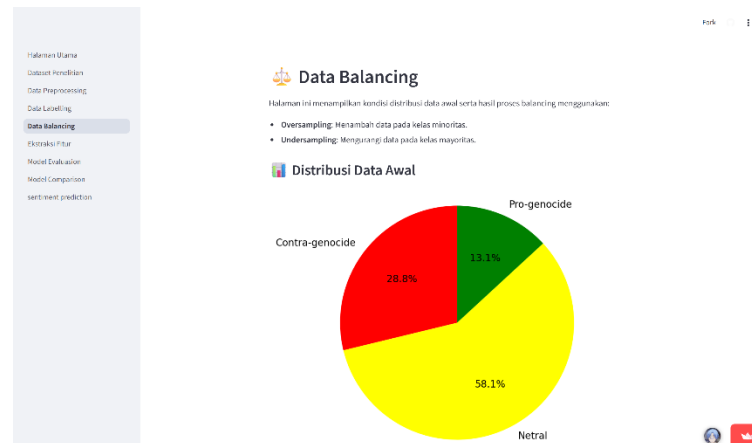
        st.success("Hasil Preprocessing:")
        st.code(text)

```

4.5.4 Halaman *Data Balancing*

Berikutnya adalah halaman *data balancing*, pada halaman ini, ditampilkan bagaimana tahapan hingga akhirnya dataset menjadi seimbang. Pada halaman ini, pengguna dapat melihat hasil dari *oversampling* dan *undersampling* yang dilakukan

hingga mendapatkan data dengan kelas sentimen yang seimbang. Gambar 4.23 merupakan tampilan halaman *data balancing*.



Gambar 4.23 Halaman *Data Balancing*

Berikut merupakan potongan *source code* yang digunakan halaman tersebut, kode tersebut berfungsi untuk merepresntasikan presentase setiap kelas sentimen menjadi diagram lingkaran.

```
try:
    df_imbalanced = pd.read_csv(path_imbalanced, delimiter=';')
    label_map = {
        'c': 'Contra-genocide', 0: 'Contra-genocide',
        'n': 'Netral',          1: 'Netral',
        'p': 'Pro-genocide',    2: 'Pro-genocide'
    }

    df_imbalanced['label'] = df_imbalanced['label'].map(label_map)

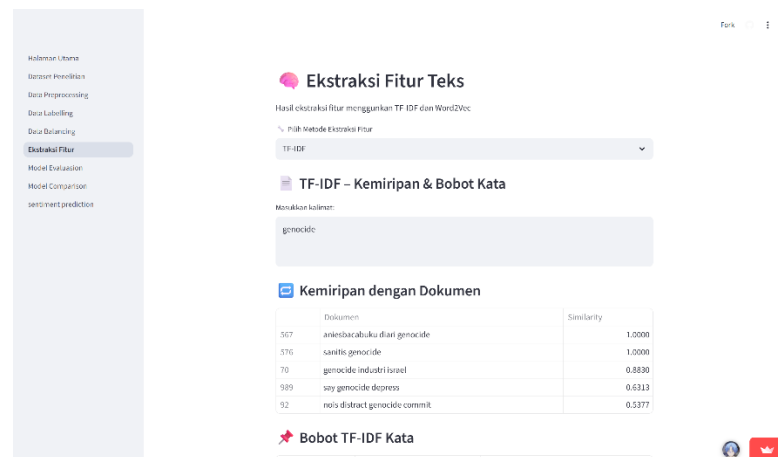
    st.subheader("📊 Distribusi Data Awal")
    show_pie_chart(df_imbalanced, label='label', title="Distribusi Label Sebelum Balancing")

except Exception as e:
    st.error(f"Gagal memuat data: {e}")
```

4.5.5 Halaman Ekstraksi Fitur

Pada halaman ini terdapat dua macam ekstraksi fitur yang ditampilkan, yang pertama adalah TF-IDF dan yang kedua adalah Word2vec. Gambar 4.24 merupakan

tampilan halaman untuk fitur TF-IDF. Pengguna dapat mengirim input berupa teks dalam bentuk kalimat maupun kata untuk menampilkan nilai bobot kata dalam kalimat tersebut. Bobot disesuaikan dengan *corpus* yang digunakan saat melakukan tuning pada TF-IDF.



Gambar 4.24 Halaman Ekstraksi Fitur TF-IDF

Berikut ini merupakan potong *source code* untuk fitur TF-IDF. Pada *code* tersebut, program melakukan vektorisasi terlebih dahulu pada input teks pengguna sebelum diproses dalam *dataframe* untuk menemukan teks yang sama pada *corpus* dan pada *function similarity* digunakan untuk menghitung seberapa besar frekuensi kata muncul pada *corpus*. Kemudian program menampilkan nilai bobot kata yang terdapat dalam *corpus*. Apabila kata yang dimasukkan pengguna tidak ada dalam *corpus* maka nilai bobotnya adalah nol.

```
if input_text:
    input_vec = tfidf_vectorizer.transform([input_text])
    similarity = cosine_similarity(input_vec,
    tfidf_matrix) [0]

    st.markdown("### 📄 Kemiripan dengan Dokumen")
    df_sim = pd.DataFrame({
        "Dokumen": corpus,
        "Similarity": similarity
    }).sort_values(by="Similarity", ascending=False)
```

```

st.table(df_sim.head(5))

st.markdown("### 🚀 Bobot TF-IDF Kata")

feature_names = tfidf_vectorizer.get_feature_names_out()
tfidf_scores = input_vec.toarray()[0]

df_weights = pd.DataFrame({
    "Kata": feature_names,
    "Bobot TF-IDF": tfidf_scores
})

df_weights = df_weights[df_weights["Bobot TF-IDF"] > 0]
df_weights = df_weights.sort_values(
    by="Bobot TF-IDF", ascending=False
)

st.table(df_weights.head(10))

```

Selanjutnya adalah halaman ekstraksi fitur pada word2vec. Pada halaman ini, pengguna dapat melakukan input kata untuk mendapatkan 10 kata yang mirip berdasarkan model word2vec. Selain itu, pengguna dapat melihat nilai vektor dari kata yang telah dimasukkan dalam kotak input. Dan pada halaman ini ditampilkan contoh bentuk vektor setiap kata. Berikut halaman word2vec dapat dilihat pada Gambar 4.25.

Ekstraksi Fitur Teks

Halal ekstraksi fitur menggunakan TF-IDF dan Word2Vec

Pilih metode ekstraksi fitur:

Word2Vec

Pilih mode:

☒ Cosine Similarity

☐ Lihat Vektor Kata

Masukkan kata:

Contoh Vektor Kata

	0	1	2	3	4	5	6	7	8	9	10
brood	-0.0027	0.0264	-0.0097	0.1918	0.2123	0.4842	0.5149	0.828	0.2428	0.2525	0.314
grin	-0.3026	0.1132	-0.1851	0.4132	0.2624	-0.4392	0.2022	0.4421	-0.3021	-0.6011	-0.212
right	-0.1947	0.0047	-0.1021	0.1201	0.2066	-0.0517	0.1553	0.5564	-0.0384	0.0396	-0.059
farms	-0.0029	0.0231	-0.2214	0.018	0.2053	-0.1152	0.1460	0.6327	0.2220	-0.2211	0.254
gray	-0.5777	0.2276	0.1356	-0.2537	0.1547	-0.2432	0.0864	1.161	0.2859	-0.2821	-0.317
genocide	-0.2058	0.1132	0.0944	0.1195	0.081	0.1851	0.1175	0.6561	0.5246	0.7943	0.271

Gambar 4.25 Halaman Ekstraksi Fitur Word2vec

Di bawah ini merupakan potongan *source code* untuk menampilkan halaman word2vec. Potongan kode tersebut berfungsi untuk mencari kata yang memiliki

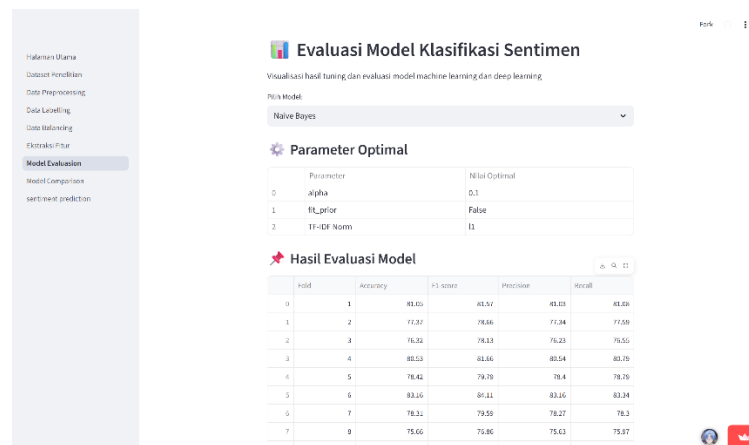
kemiripan dengan input teks pengguna dan menghitung jarak vektor dari kata yang di input oleh pengguna.

```
input_word = st.text_input("Masukkan kata:")

if input_word:
    if input_word not in w2v_model.wv:
        st.warning(f"Kata '{input_word}' tidak ditemukan di vocabulary.")
    else:
        if mode == "Cari Similarity":
            similar_words =
w2v_model.wv.most_similar(input_word, topn=10)
            df_sim = pd.DataFrame(similar_words,
columns=["Kata", "Similarity"])
            st.table(df_sim)
        if mode == "Lihat Vektor Kata":
            vector = w2v_model.wv[input_word]
            df_vector = pd.DataFrame(
                vector.reshape(1, -1),
                columns=[f"dim_{i+1}" for i in
range(len(vector))]
            )
            st.dataframe(df_vector)
```

4.5.6 Halaman *Model Evaluation*

Halaman ini menampilkan evaluasi pada setiap model menggunakan metrik nilai akurasi, *f1-score*, *precision* dan *recall*, lalu menampilkan *confusion matrix* dan grafik ROC. Sehingga pengguna dapat memilih model melalui fitur *dropdown*, kemudian evaluasi pada model tersebut ditampilkan pada laman aplikasi. Implementasi halaman evaluasi model dapat dilihat pada Gambar 4.26.



Gambar 4.26 Halaman Evaluasi Model

Sedangkan potongan *source code* di bawah ini menjabarkan bagaimana aplikasi tersebut menampilkan evaluasi model dengan memanggil *function* tertentu.

```
st.subheader("🔴 Hasil Evaluasi Model")

df_eval = pd.read_csv(EVAL_FILES[model_choice])
st.dataframe(df_eval)

st.markdown("**Rata-rata Evaluasi:**")
st.write(df_eval.mean(numeric_only=True))

st.subheader("📊 Visualisasi Model")

col1, col2 = st.columns(2)

with col1:
    st.markdown("**Confusion Matrix**")
    st.image(Image.open(IMAGE_FILES[model_choice]["cm"]))

with col2:
    st.markdown("**ROC Curve**")
    st.image(Image.open(IMAGE_FILES[model_choice]["roc"]))
```

4.5.7 Halaman *Model Comparison*

Selanjutnya adalah halaman komparasi model. pada halaman ini, pengguna dapat melihat komparasi pada setiap evaluasi model dalam bentuk tabel dan grafik. Halaman ini berfungsi untuk melakukan komparasi pada setiap model dan menentukan model yang memiliki nilai metrik evaluasi yang paling tinggi sebagai

model klasifikasi yang paling optimal. Gambar 4.27 menampilkan halaman komparasi model tersebut.



Gambar 4.27 Halaman Komparasi Model

Berikut adalah potongan *source code* yang digunakan pada halaman komparasi model.

```
df_comparison = pd.DataFrame(comparison_data)
st.subheader("★ Tabel Perbandingan Model (Rata-rata)")
st.dataframe(df_comparison)

st.subheader("📊 Perbandingan Metric - Bar Chart")
metrics = ["Accuracy", "F1-score", "Precision", "Recall"]

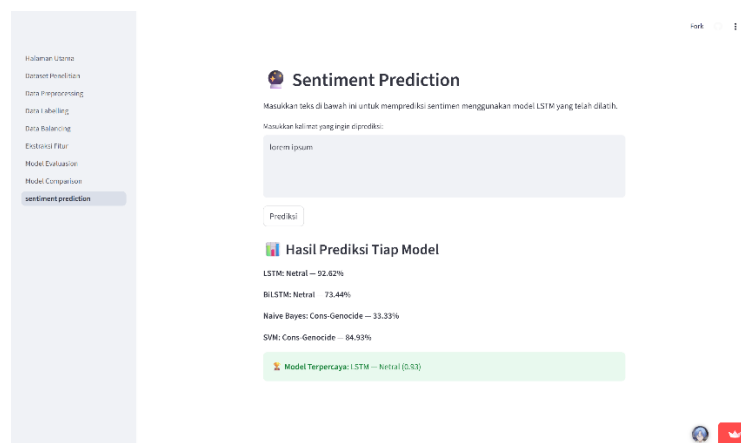
fig, ax = plt.subplots(figsize=(10,6))
x = np.arange(len(df_comparison))
width = 0.2

for i, metric in enumerate(metrics):
    ax.bar(x + i*width, df_comparison[metric], width,
    label=metric)

ax.set_xticks(x + width*1.5)
ax.set_xticklabels(df_comparison["Model"])
ax.set_ylim(0, 100)
ax.set_ylabel("Persentase (%)")
ax.set_title("Perbandingan Metric Model")
ax.legend()
st.pyplot(fig)
```

4.5.8 Halaman *Sentiment Prediction*

Pada halaman prediksi sentimen terdapat kotak *input* untuk memasukkan teks yang selanjutnya diklasifikasikan menggunakan empat model yang telah dilakukan training dan tuning sebelumnya. Hasil prediksi ditampilkan dari setiap model dengan nilai *confidence*. Dan aplikasi menampilkan model dengan nilai *confidence* paling tinggi adalah model paling optimal pada proses prediksi tersebut. Tampilan halaman prediksi sentimen dapat dilihat pada Gambar 4.28.



Gambar 4.28 Halaman Prediksi Sentimen

Berikut ini merupakan potongan *source code* yang digunakan dalam memprediksi sentimen. program memanggil file model dengan ekstensi .keras dan .pkl untuk memproses data teks yang dimasukkan pengguna. Selain itu, terdapat proses teks preprocessing sebelum data diproses model, hal tersebut untuk menghindari terjadinya prediksi yang bias karena teks tidak dikenali polanya oleh model *machine learning* maupun *deep learning*.

```
if st.button("Prediksi"):
    if text_input.strip() == "":
        st.warning("Mohon masukkan kalimat terlebih dahulu.")
    else:
        results = {}

        # Model LSTM
```

```

        label_lstm, conf_lstm = predict_lstm(text_input,
model_lstm, tokenizer)
        results["LSTM"] = (label_lstm, conf_lstm)

        # Model BiLSTM
        label_bilstm, conf_bilstm = predict_lstm(text_input,
model_bilstm, tokenizer)
        results["BiLSTM"] = (label_bilstm, conf_bilstm)

        # Model Naive Bayes
        label_nb, conf_nb = predict_sklearn(text_input,
model_nb, vectorizer)
        results["Naive Bayes"] = (label_nb, conf_nb)

        # Model SVM
        label_svm, conf_svm = predict_sklearn(text_input,
model_svm, vectorizer)
        results["SVM"] = (label_svm, conf_svm)

        # Menentukan model dengan confidence tertinggi
        best_model = max(results.items(), key=lambda x: x[1][1])

        # Tampilkan hasil
        st.subheader("📊 Hasil Prediksi Tiap Model")
        for model_name, (label, conf) in results.items():
            st.write(f"**{model_name}**: ** {label}** -
**{conf*100:.2f}%**")

        st.success(f"🏆 **Model Terpercaya:** {best_model[0]} -
{best_model[1][0]} ({best_model[1][1]:.2f})")

```

4.6 Limitasi Penelitian

Selama proses penelitian ini, terdapat beberapa limitasi yang dialami penulis, diantaranya adalah sebagai berikut.

1. Adanya keterbatasan terhadap dataset sehingga memengaruhi kinerja model, terutama pada model *deep learning* seperti LSTM dan BiLSTM yang membutuhkan dataset lebih besar dan variatif.
2. Hasil fine-tuning menunjukkan bahwa model BiLSTM memiliki nilai akurasi yang lebih rendah dibandingkan model LSTM. Sehingga dapat disimpulkan bahwa model BiLSTM tidak efisien digunakan pada kasus ini yang memiliki dataset yang kecil dan terbatas.

3. Berdasarkan hasil evaluasi model, nilai metrik pada model SVM merupakan nilai paling tinggi dari model LSTM, BiLSTM dan Naive Bayes. Namun, sentimen model *deep learning* cenderung lebih akurat dalam memprediksi kelas sentimen.
4. Penelitian lanjutan dapat menggunakan dataset yang lebih luas dan menggunakan pendekatan yang lebih optimal untuk meningkatkan performa model.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil analisis dan pengujian yang telah dilakukan terhadap model *machine learning* dan *deep learning* dalam klasifikasi sentimen isu genosida Palestina pada media sosial X, maka dapat disimpulkan sebagai berikut:

1. Penelitian ini berhasil mengembangkan empat model klasifikasi sentimen, yaitu Naive Bayes, SVM, LSTM, dan BiLSTM, yang berfungsi untuk mengklasifikasikan sentimen *tweet* terkait isu genosida Palestina.
2. Berdasarkan hasil evaluasi menggunakan metrik akurasi, *precision*, *recall*, dan *f1-score*, model SVM menunjukkan nilai rata-rata nilai akurasi tertinggi sebesar 85,97%. Sementara itu, model *deep learning*, khususnya LSTM dan BiLSTM, mampu memahami konteks sentimen lebih baik dibandingkan model *machine learning* seperti Naive Bayes dan SVM.
3. Dataset penelitian berhasil dikumpulkan dari media sosial X menggunakan kata kunci “*genocide*”, “Gaza”, “Israel”, dan “*Palestine*” dalam bahasa Inggris dengan jangka waktu dari 7 Oktober 2023 sampai dengan 7 April 2025.
4. Penelitian ini juga berhasil membangun sebuah aplikasi berbasis web yang dikembangkan menggunakan *framework* Streamlit, yang berfungsi untuk menampilkan hasil pengujian model dan klasifikasi sentimen. dan

penggunaan *Unified Modelling Language* (UML) dalam merancang alur dan struktur sistem,

5.1 Saran

Berdasarkan penelitian yang dilakukan, penulis memberikan saran yang dapat diimplementasikan pada penelitian selanjutnya, yaitu sebagai berikut:

1. Menggunakan dataset yang lebih besar dan variatif sehingga dapat meningkatkan performa model *deep learning*.
2. Penelitian selanjutnya dapat menggunakan model *pre-trained* agar mendapatkan konfigurasi model yang lebih optimal.
3. Aplikasi berbasis web dapat dikembangkan lebih lanjut dengan fitur prediksi sentimen dengan dataset yang lebih banyak dan dashboard yang dapat menampilkan analisis tren sentimen secara real-time untuk pengguna.

DAFTAR PUSTAKA

- Ahmad, T., Iqbal, J., Ashraf, A., Truscan, D., & Porres, I. (2019). Model-based testing using UML activity diagrams: A systematic mapping study. In *Computer Science Review* (Vol. 33, pp. 98–112). Elsevier Ireland Ltd. <https://doi.org/10.1016/j.cosrev.2019.07.001>
- Bhatt, B., & Nandu, M. (2021). An Overview of Structural UML Diagrams. *International Research Journal of Engineering and Technology*. www.irjet.net
- Derbentsev, V. D., Bezkorovainyi, V. S., Matviychuk, A. V, Pomazun, O. M., Hrabariiev, A. V, & Hostryk, A. M. (2023). *A comparative study of deep learning models for sentiment analysis of social media texts* ★.
- Eka Putra, R. (2024). Perbandingan Algoritma LSTM dan BiLSTM Untuk Analisis Sentimen Multi-Class Media Sosial Twitter. *Journal of Informatics and Computer Science*, 06.
- Fitroh, F., & Hudaya, F. (2023). Systematic Literature Review: Analisis Sentimen Berbasis Deep Learning. *Jurnal Nasional Teknologi Dan Sistem Informasi*, 9(2), 132–140. <https://doi.org/10.25077/teknosi.v9i2.2023.132-140>
- Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed., Vol. 3). Addison-Wesley.
- Jędrzejewska, K. Z. (2020). HASBARA: PUBLIC DIPLOMACY WITH ISRAELI CHARACTERISTICS. *Torun International Studies*, 1(13), 105. <https://doi.org/10.12775/tis.2020.008>
- Junaedy, F. Z., Hidayat Insani, R., Santoso, I., Tinggi, S., Komputer, I., Karya Informatika, C., & Jakarta, U. (n.d.). *KOMPARASI ALGORITMA SUPPORT VECTOR MACHINE DAN NAÏVE BAYES PADA ANALISIS SENTIMEN FORMULA-E JAKARTA TAHUN 2022*. <https://youtu.be/mhAEiHuVFxY>
- Jurafsky, D., & Martin, J. H. (n.d.). *Speech and Language Processing An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models Third Edition draft*.
- Kobayashi, V. B., Mol, S. T., Berkers, H. A., Kismihók, G., & Den Hartog, D. N. (2018). Text Mining in Organizational Research. *Organizational Research Methods*, 21(3), 733–765. <https://doi.org/10.1177/1094428117722619>
- Kusuma Putri, G., Sujaini, H., Isna Ulumi, D., Hadari Nawawi, J. H., & Barat, K. (n.d.). *Perbandingan Algoritma Support Vector Machine (SVM) dan Naive Bayes pada Analisis Sentimen Bahasa Jawa dan Sunda*. <https://doi.org/10.26418/justin.v13i2.88285>

- Kusuma Wardani, S., & Arum Sari, Y. (2021). *Analisis Sentimen menggunakan Metode Naïve Bayes Classifier terhadap Review Produk Perawatan Kulit Wajah menggunakan Seleksi Fitur N-gram dan Document Frequency Thresholding* (Vol. 5, Issue 12). <http://j-ptiik.ub.ac.id>
- Liu, B. (n.d.). *Sentiment Analysis Mining Opinions, Sentiments, and Emotions*.
- Migliardi, M., Dongarra, J., Geist, A., Sunderam, V., Ridge, O., & Laboratories, N. (n.d.). *Dynamic Reconfiguration and Virtual Machine Management in the Harness Metacomputing System*.
- Natural Language Processing with Python*. (n.d.).
- Noori, M. T., Rahman, M. A., Purnomo, A., & Aripin, A. (2025). Sentiment Analysis of the Israel-Palestine Conflict on X: Insights from the Indonesian Perspective using a Long Short-Term Memory Algorithm. *Journal of Informatics and Web Engineering*, 4(2), 417–429. <https://doi.org/10.33093/jiwe.2025.4.2.27>
- Prakoso, C., & Hermawan, A. (2023). KLIK: Kajian Ilmiah Informatika dan Komputer Perbandingan Model Machine Learning dalam Analisis Sentimen Ulasan Pengunjung Keraton Yogyakarta pada Google Maps. *Media Online*, 4(3), 1292–1302. <https://doi.org/10.30865/klik.v4i3.1419>
- Ryandi, F. A., Pratiwi, D., & Sari, S. (2025). Analisis Sentimen Masyarakat Di Media Sosial X Terhadap Kemenkes Dengan Naive Bayes dan SVM. *Jurnal Sains Dan Teknologi*, 7(1), 1–6. <https://doi.org/10.55338/saintek.v7i1.4615>
- Staudemeyer, R. C., & Morris, E. R. (2019). *Understanding LSTM -- a tutorial into Long Short-Term Memory Recurrent Neural Networks*. <http://arxiv.org/abs/1909.09586>
- Sundaramoorthy, S. (2022). *UML Diagramming: A Case Study Approach*.
- Vijayakumar, V. (2654). SENTIMENT ANALYSIS USING LSTM. *Www.Irjmets.Com @International Research Journal of Modernization in Engineering*. <https://doi.org/10.56726/IRJMETS65428>
- Yin, W., Kann, K., Yu, M., & Schütze, H. (2017). *Comparative Study of CNN and RNN for Natural Language Processing*. <http://arxiv.org/abs/1702.01923>
- Young, T., Hazarika, D., Poria, S., & Cambria, E. (2018). *Recent Trends in Deep Learning Based Natural Language Processing*. <http://arxiv.org/abs/1708.02709>
- Zhang, L., Wang, S., & Liu, B. (n.d.). *Deep Learning for Sentiment Analysis : A Survey*.
- Zhao, W., Zhang, G., Yuan, G., Liu, J., Shan, H., & Zhang, S. (2020). The Study on the Text Classification for Financial News Based on Partial Information. *IEEE Access, PP*, 1. <https://doi.org/10.1109/ACCESS.2020.2997969>

LAMPIRAN

Lampiran 1 Daftar Stopword

i, me, my, myself, we, our, ours, ourselves, you, your, yours, yourself, yourselves, he, him, his, himself, she, her, hers, herself, it, its, itself, they, them, their, theirs, themselves, what, which, who, whom, this, that, these, those, am, is, are, was, were, be, been, being, have, has, had, having, do, does, did, doing, a, an, the, and, but, if, or, because, as, until, while, of, at, by, for, with, about, against, between, into, through, during, before, after, above, below, to, from, up, down, in, out, on, off, over, under, again, further, then, once, here, there, when, where, why, how, all, any, both, each, few, more, most, other, some, such, no, nor, not, only, own, same, so, than, too, very, s, t, can, will, just, don, should, now

Lampiran 2 Dataset Mentah

No	Created_at	Favorite_count	Full_text	Id_str
1.	Sun Apr 06 23:39:20 +0000 2025	43	@sentdefender The whole world stands with Israel and the IDF. End the Islamic regime of Iran.	1909029 5603534 97600
2.	Sun Apr 06 23:39:20 +0000 2025	0	I agree. Israel's war is with Hamas NOT the Palestinian people. Can never understand why the UK won't stand with Palestine. The state of Isreal didn't exist until 1948 when Britain thought it was a good idea to resettle them in Palestine.	1909028 1782214 70200
3.	Sun Apr 06 23:39:20 +0000 2025	0	@EliAfriatISR I stand with Isreal.	1909031 4420399 35500
4.	Sun Apr 06 23:39:20 +0000 2025	1032	We the people stand with Isreal. Our Allies.	1909030 0466791 42700

No	Created_at	Favorite_count	Full_text	Id_str
5.	Sun Apr 06 23:39:20 +0000 2025	690	America must stand with Israel as it takes action against Iranian-backed terrorists.	1909029 6351211 64500
6.	Sun Apr 06 23:39:20 +0000 2025	4	Fact Just because you don't stand with Isreal or Forever wars don't make you antisemitic !!! It's time we end this notion that people are jew hating antisemitic because they refute Isreal. God bless America	1909030 0769535 83900
7.	Sun Apr 06 23:39:20 +0000 2025	0	And yet you arseholes stand with the murderous isreal	1909030 9890173 46300
8.	Sun Apr 06 23:39:20 +0000 2025	0	The UK must stand with Israel. Her enemies are ours. In a region of autocracy she shines as a rare democracy. We cannot abandon her. https://t.co/3ZngtSTj9L	1909032 2200245 20700
...	...	...	...	...
1702	Sun Apr 06 23:39:20 +0000 2025	268	The I Stand with Israel I support: 1. Israel has a right to exist	1909031 1831212 60500

Selengkapnya dapat dilihat pada <https://github.com/waftsa/skripsi>

Lampiran 3 Kode Preprocessing Dataset

```
import pandas as pd
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer, WordNetLemmatizer
from google.colab import files

nltk.download('stopwords')
nltk.download('punkt')
nltk.download('punkt_tab')

# 1. Upload file manual
print("\u2191Upload file CSV `:`")
uploaded = files.upload()

# Ambil nama file
filename = next(iter(uploaded))
```

```

# 2. Load dan baca data
try:
    df = pd.read_csv(filename, delimiter=';', quoting=3,
encoding='utf-8', on_bad_lines='skip')
    print("✅ Dibaca dengan UTF-8")
except UnicodeDecodeError:
    try:
        df = pd.read_csv(filename, delimiter=';', quoting=3,
encoding='ISO-8859-1', on_bad_lines='skip')
        print("✅ Dibaca dengan ISO-8859-1")
    except UnicodeDecodeError:
        df = pd.read_csv(filename, delimiter=';', quoting=3,
encoding='latin1', on_bad_lines='skip')
        print("✅ Dibaca dengan latin1 (fallback)")

# 3. Inisialisasi tools preprocessing
stop_words = set(stopwords.words('english'))
sentiment_stopwords = {
    "the", "a", "an", "and", "or", "but", "if", "while", "of",
    "at", "by", "for", "with", "about",
    "against", "between", "into", "through", "during", "before",
    "after", "above", "below", "to",
    "from", "up", "down", "in", "out", "on", "off", "over",
    "under", "again", "further", "then",
    "once", "here", "there", "when", "where", "why", "how",
    "all", "any", "both", "each", "few",
    "more", "most", "other", "some", "such", "only", "own",
    "same", "so", "than", "too", "very",
    "can", "will", "just", "should", "now", "you", "i", "me",
    "my", "we", "us", "our", "they",
    "them", "their", "he", "him", "his", "she", "her", "it",
    "its", "this", "that", "these", "those"
}
custom_stopwords = stop_words - sentiment_stopwords

stemmer = PorterStemmer()
sensitive_words = {"israel", "palestine", "hamas", "genocide",
"gaza", "zionist"}

def custom_stem_id(word):
    word_lower = word.lower()
    if word_lower in sensitive_words:
        return word_lower
    else:
        return stemmer.stem(word_lower)

# lemmatizer = WordNetLemmatizer()

# 4. Fungsi preprocessing lengkap
def full_preprocess(text):
    text = str(text).lower()
    text = re.sub(r"http\S+|www\S+|https\S+", "", text)      #
hapus URL
    text = re.sub(r"@w+", "", text)                          #
hapus mention

```

```

        text = re.sub(r"^[^w\s]", "", text) #
hapus tanda baca
        text = re.sub(r"[\U00010000-\U0010ffff]", "", text) #
hapus emoji
        text = re.sub(r"\d+", "", text) #
hapus angka
        text = re.sub(r"\s+", " ", text).strip() #
hapus spasi berlebih
        text = re.sub(r'[^x00-\x7F]+', '', str(text)) #
hapus simbol non-ASCII

        # Tokenisasi dan stopwords removal
        words = word_tokenize(text)

        words = [word for word in words if word not in stop_words]

        # Lematisasi dan stemming
        # lemmatized = [lemmatizer.lemmatize(word) for word in
words]
        stemmed_tokens = [custom_stem_id(w) for w in words]

        # Return teks yang sudah dibersihkan
        return " ".join(stemmed_tokens)

# 5. Jalankan preprocessing
df = df[df['full_text_processed'].notna()]
df['full_text_processed'] =
df['full_text_processed'].apply(full_preprocess)

# 6. Simpan hasil
output_file = 'final_preprocessed.csv'
if 'created_at' in df.columns:
    df[['created_at', 'full_text_processed',
'label']].to_csv(output_file, index=False)
else:
    df[['full_text_processed', 'label']].to_csv(output_file,
index=False)

print(f"✅ Preprocessing selesai. File disimpan sebagai
`{output_file}`.")
files.download(output_file)

df.head()

```

Lampiran 4 Source Code GridsearchCV Model Naive Bayes

```

# cross validation + gridsearch

X_train, X_test, y_train, y_test = train_test_split(
    df["full_text_processed"].astype(str),
    df["label"],
    test_size=0.2,
    stratify=df["label"],
    random_state=42

```



```

)

pipeline_nb = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("nb", MultinomialNB())
])

param_grid_nb = {
    "tfidf__max_features": [2000, 3000, 4000],
    "tfidf__ngram_range": [(1,1), (1,2)],
    "tfidf__norm": ["l1", "l2"],
    "nb__alpha": [0.1, 0.5, 1.0],
    "nb__fit_prior": [True, False],
}

grid_nb = GridSearchCV(
    pipeline_nb,
    param_grid_nb,
    cv=10,
    n_jobs=-1,
    scoring="f1_macro",
    verbose=2
)

grid_nb.fit(X_train, y_train)

print("\n=== 🕒 Best Parameters ===")
print(grid_nb.best_params_)

print("\n=== 🏆 Best CV F1-Macro Score ===")
print(grid_nb.best_score_)

best_model = grid_nb.best_estimator_
y_pred = best_model.predict(X_test)

print("\n=== 📊 Final Evaluation on Test Set ===")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("F1-Macro:", f1_score(y_test, y_pred, average='macro'))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

```

Lampiran 5 Source Code GridsearchCV Model SVM

```

# cross validation + grid search

X_train, X_test, y_train, y_test = train_test_split(
    df["full_text_processed"].astype(str),
    df["label"],
    test_size=0.2,
    stratify=df["label"],
    random_state=42
)

```

```

pipeline_svm = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("svm", SVC(probability=True))
])

param_grid_svm = {
    "tfidf__max_features": [4000],
    "tfidf__ngram_range": [(1,2)],
    "tfidf__norm": ["l1", "l2"],

    "svm__kernel": ["linear", "rbf"],
    "svm__C": [0.1, 1, 10],
    "svm__gamma": ["scale", "auto"]
}

grid_svm = GridSearchCV(
    pipeline_svm,
    param_grid_svm,
    cv=10,
    scoring="f1_macro",
    n_jobs=-1,
    verbose=2
)

grid_svm.fit(X_train, y_train)

# Best params & score
print("\n=== 🕒 Best Parameters ===")
print(grid_svm.best_params_)

print("\n=== 🏆 Best CV F1-Macro Score ===")
print(grid_svm.best_score_)

# Final evaluation (test set)
best_model = grid_svm.best_estimator_
y_pred = best_model.predict(X_test)

print("\n=== 🇮🇩 Final Evaluation on Test Set ===")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("F1-Macro:", f1_score(y_test, y_pred, average="macro"))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

```

Lampiran 6 Source Code Training LSTM

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SpatialDropout1D,
LSTM, Flatten, Dense
from tensorflow.keras.optimizers import Adam, AdamW
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score

```

```

fold = 1
results = []

for train_index, val_index in skf.split(X_padded, labels):
    print(f"\nFold {fold} -----")

    X_train, X_val = X_padded[train_index], X_padded[val_index]
    y_train, y_val = labels[train_index], labels[val_index]

    model = Sequential()
    model.add(Embedding(input_dim=vocab_size,
                        output_dim=100,
                        weights=[embedding_matrix],
                        input_length=100))
    model.add(LSTM(32, dropout=0.2, recurrent_dropout=0.2))
    model.add(Dense(3, activation='softmax'))

    optimizer = Adam(learning_rate=0.0001)
    model.compile(optimizer=optimizer,
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

    # Callback early stopping
    early_stop = EarlyStopping(
        monitor='val_loss',
        patience=2,
        restore_best_weights=True
    )

    # Training
    model.fit(
        X_train, y_train,
        epochs=25,
        batch_size=16,
        validation_data=(X_val, y_val),
        callbacks=[early_stop],
        verbose=0
    )

    y_pred_prob = model.predict(X_val, verbose=0)
    y_pred = np.argmax(y_pred_prob, axis=1)

    # Evaluasi
    acc = accuracy_score(y_val, y_pred) * 100
    prec = precision_score(y_val, y_pred, average='macro') * 100
    rec = recall_score(y_val, y_pred, average='macro') * 100
    f1 = f1_score(y_val, y_pred, average='macro') * 100

    results.append({
        "Fold": fold,
        "Accuracy": acc,
        "F1-Score": f1,
        "Precision": prec,
        "Recall": rec
    })

```

```

        fold += 1

# === TABEL HASIL ===
df_cv_lstm = pd.DataFrame(results).round(2)

print("=== 🇮🇩 10-Fold Cross Validation Results (LSTM) ===")
print(df_cv_lstm)

print("\n=== 📊 Rata-rata Performa ===")
print(df_cv_lstm.mean(numeric_only=True).round(2))

```

Lampiran 7 Source Code Training BiLSTM

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense,
Bidirectional, GlobalMaxPooling1D
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score

fold = 1
results = []

for train_index, val_index in skf.split(X_padded, labels):
    print(f"\nFold {fold} -----")

    X_train, X_val = X_padded[train_index], X_padded[val_index]
    y_train, y_val = labels[train_index], labels[val_index]

    model = Sequential()
    model.add(Embedding(input_dim = vocab_size,
                        output_dim = 100,
                        weights=[embedding_matrix],
                        input_length= 100))
    model.add(Bidirectional(LSTM(32, dropout=0.2,
                                recurrent_dropout=0.2),
                        merge_mode='concat'))
    model.add(Dense(3, activation='softmax'))

    optimizer = Adam(learning_rate=0.0001)
    model.compile(optimizer=optimizer,
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

    # Callback early stopping
    early_stop = EarlyStopping(
        monitor='val_loss',
        patience=2,
        restore_best_weights=True
    )

    # Training

```

```

model.fit(
    X_train, y_train,
    epochs=25,
    batch_size=16,
    validation_data=(X_val, y_val),
    callbacks=[early_stop],
    verbose=0
)

y_pred_prob = model.predict(X_val, verbose=0)
y_pred = np.argmax(y_pred_prob, axis=1)

# Evaluasi
acc = accuracy_score(y_val, y_pred) * 100
prec = precision_score(y_val, y_pred, average='macro') * 100
rec = recall_score(y_val, y_pred, average='macro') * 100
f1 = f1_score(y_val, y_pred, average='macro') * 100

results.append({
    "Fold": fold,
    "Accuracy": acc,
    "F1-Score": f1,
    "Precision": prec,
    "Recall": rec
})

fold += 1

# === TABEL HASIL ===
df_cv_lstm = pd.DataFrame(results).round(2)

print("=== 🇮🇩 10-Fold Cross Validation Results (LSTM) ===")
print(df_cv_lstm)

print("\n=== 📊 Rata-rata Performa ===")
print(df_cv_lstm.mean(numeric_only=True).round(2))

```

RIWAYAT HIDUP

Nama Lengkap : Wafa Tsabita
 Tempat, Tanggal Lahir : Newcastle Upon Tyne, 28 Agustus 2003
 Alamat : Tanjungsari, Kabupaten Sumedang
 No. Telepon / HP : 087738474424
 Email : wafa20001@mail.unpad.ac.id

Riwayat Pendidikan

- 2008 – 2013 SDIT Imam Bukhari
- 2013 – 2017 SMPIT Imam Bukhari
- 2017 – 2020 SMAIT Al-Multazam
- 2020 – Sekarang, Teknik Informatika, Universitas Padjadjaran

Pengalaman Organisasi

- Himpunan Mahasiswa Teknik Informatika Unpad – Staff Departemen
- Rohis Nurul ‘Ilmi FMIPA Unpad – Kepala Departemen
- Keluarga Mahasiswa Muslim Kampus Unpad – Wakil Kepala Departemen
- Unpad Student Justice for Palestine – Kepala Departemen
- SMART171 – IT Support